Lab Manual

AIC270 -Programming for Artificial Intelligence

CUI

Department of Computer Science
Islamabad Campus

Lab Contents:

Python Data structures and functions; Introduction to Numpy; Logical programing using python: Validating primes, Parsing a family tree, Analyzing geography and building puzzle solver; Tabu search; Simulated Annealing; Genetics Algorithm; Introduction to Keras; Medical image Classification using Keras; Sentiment classification using Keras; Introduction to Open CV; Image smoothing and template matching.

Student Outcomes (SO)

S.#	Description
1	Apply knowledge of computing fundamentals, knowledge of a computing specialization, and mathematics, science, and domain knowledge appropriate for the computing specialization to the abstraction and conceptualization of computing models from defined problems and requirements
2	Identify, formulate, research literature, and solve complex computing problems reaching substantiated conclusions using fundamental principles of mathematics, computing sciences, and relevant domain disciplines
3	Design and evaluate solutions for complex computing problems, and design and evaluate systems, components, or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations
4	Create, select, adapt and apply appropriate techniques, resources, and modern computing tools to complex computing activities, with an understanding of the limitations
9	Recognize the need, and have the ability, to engage in independent learning for continual development as a computing professional

Intended Learning Outcomes

Sr.#	Description	Blooms Taxonomy Learning Level	SO
CLO -4	Implement simple AI based applications and optimization techniques using Python	*Applying*	1-4
CLO -5	Build an AI application using deep learning	*Creating*	1-4,9
CLO -6	Develop games using AI.	*Creating*	1-4,9

Lab Assessment Policy

The lab work done by the student is evaluated using rubrics defined by the course instructor, viva-voce, project work/performance. Marks distribution is as follows:

Assignments	Lab Mid Term Exam	Lab Terminal Exam	Total
25	25	50	100

Note: Midterm and Final term exams must be computer based.

List of Labs

Lab 01
Python Data structures and functions

Objective:

This lab will give you practical implementation of different types of **sequences** including **lists, tuples, sets and dictionaries**. We will use lists alongside loops in order to know about indexing individual items of these containers. This lab will also allow students to write their own **functions.**

Activity Outcomes:

This lab teaches you the following topics:

- How to use lists, tuples, sets and dictionaries How to use loops with lists
- How to write customized functions

Instructor Note:

As a pre-lab activity, read Chapters 6, 10 and 14 from the book (*Introduction to Programming Using Python - Y. Liang (Pearson, 2013)*) to gain an insight about python programming and its fundamentals.

1) Useful Concepts

Python provides different types of data structures as sequences. In a sequence, there are more than one values and each value has its own index. The first value will have an index 0 in python, the second value will have index 1 and so on. These indices are used to access a particular value in the sequence.

Python Lists:

Lists are just like dynamically sized arrays, declared in other languages (vector in C++ and ArrayList in Java). Lists need not be homogeneous always which makes it the most powerful tool in Python. A single list may contain DataTypes like Integers, Strings, as well as Objects. Lists are mutable, and hence, they can be altered even after their creation.

List in Python are ordered and have a definite count. The elements in a list are indexed according to a definite sequence and the indexing of a list is done with 0 being the first index. Each element in the list has its definite place in the list, which allows duplicating of elements in the list, with each element having its own distinct place and credibility.

Creating a List

Lists in Python can be created by just placing the sequence inside the square brackets[]. Unlike Sets, a list doesn't need a built-in function for the creation of a list.

```
# Python program to demonstrate
# Creation of List

# Creating a List
List = []
print("Blank List: ")
print(List)

# Creating a List of numbers
List = [10, 20, 14]
print("\nList of numbers: ")
print(List)

# Creating a List of strings and accessing
# using index
List = ["Geeks", "For", "Geeks"]
print("\nList Items: ")
print(List[0])
print(List[2])

# Creating a Multi-Dimensional List
# (By Nesting a list inside a List)
List = [['Geeks', 'For'], ['Geeks']]
print("\nMulti-Dimensional List: ")
print(List)
```

Output:
Blank List:
[]

List of numbers:
[10, 20, 14]

List Items
Geeks
Geeks

Multi-Dimensional List:
[['Geeks', 'For'], ['Geeks']]

Creating a list with multiple distinct or duplicate elements

A list may contain duplicate values with their distinct positions and hence, multiple distinct or duplicate values can be passed as a sequence at the time of list creation.

```
# Creating a List with
# the use of Numbers
# (Having duplicate values)
List = [1, 2, 4, 4, 3, 3, 3, 6, 5]
print("\nList with the use of Numbers: ")
print(List)

# Creating a List with
# mixed type of values
# (Having numbers and strings)
List = [1, 2, 'Geeks', 4, 'For', 6, 'Geeks']
print("\nList with the use of Mixed Values: ")
print(List)
```

Output:
List with the use of Numbers:
[1, 2, 4, 4, 3, 3, 3, 6, 5]

List with the use of Mixed Values:
[1, 2, 'Geeks', 4, 'For', 6, 'Geeks']

Knowing the size of List

```
# Creating a List
List1 = []
```

```
print(len(List1))

# Creating a List of numbers
List2 = [10, 20, 14]
print(len(List2))
```

Output:
0
3

Adding Elements to a List
Using append() method
Elements can be added to the List by using the built-in **append()** function. Only one element at a time can be added to the list by using the append() method, for the addition of multiple elements with the append() method, loops are used. Tuples can also be added to the list with the use of the append method because tuples are immutable. Unlike Sets, Lists can also be added to the existing list with the use of the append() method.

```
# Python program to demonstrate
# Addition of elements in a List

# Creating a List
List = []
print("Initial blank List: ")
print(List)

# Addition of Elements
# in the List
List.append(1)
List.append(2)
List.append(4)
print("\nList after Addition of Three elements: ")
print(List)

# Adding elements to the List
# using Iterator
for i in range(1, 4):
    List.append(i)
print("\nList after Addition of elements from 1-3: ")
print(List)

# Adding Tuples to the List
List.append((5, 6))
print("\nList after Addition of a Tuple: ")
```

```
print(List)

# Addition of List to a List
List2 = ['For', 'Geeks']
List.append(List2)
print("\nList after Addition of a List: ")
print(List)

Output:
Initial blank List:
[]

List after Addition of Three elements:
[1, 2, 4]

List after Addition of elements from 1-3:
[1, 2, 4, 1, 2, 3]

List after Addition of a Tuple:
[1, 2, 4, 1, 2, 3, (5, 6)]

List after Addition of a List:
[1, 2, 4, 1, 2, 3, (5, 6), ['For', 'Geeks']]
```

Using insert() method

append() method only works for the addition of elements at the end of the List, for the addition of elements at the desired position, insert() method is used. Unlike append() which takes only one argument, the insert() method requires two arguments(position, value).

```
# Python program to demonstrate
# Addition of elements in a List

# Creating a List
List = [1,2,3,4]
print("Initial List: ")
print(List)

# Addition of Element at
# specific Position
# (using Insert Method)
List.insert(3, 12)
List.insert(0, 'Geeks')
print("\nList after performing Insert Operation: ")
print(List)
```

Output:
Initial List:
[1, 2, 3, 4]

List after performing Insert Operation:
['Geeks', 1, 2, 3, 12, 4]

Using extend() method

Other than append() and insert() methods, there's one more method for the Addition of elements, **extend()**, this method is used to add multiple elements at the same time at the end of the list.

```
# Python program to demonstrate
# Addition of elements in a List

# Creating a List
List = [1, 2, 3, 4]
print("Initial List: ")
print(List)

# Addition of multiple elements
# to the List at the end
# (using Extend Method)
List.extend([8, 'Geeks', 'Always'])
print("\nList after performing Extend Operation: ")
print(List)
```

Output:
Initial List:
[1, 2, 3, 4]

List after performing Extend Operation:
[1, 2, 3, 4, 8, 'Geeks', 'Always']

Accessing elements from the List

In order to access the list items refer to the index number. Use the index operator [ ] to access an item in a list. The index must be an integer. Nested lists are accessed using nested indexing.

```
# Python program to demonstrate
# accessing of element from list

# Creating a List with
# the use of multiple values
```

```
List = ["Geeks", "For", "Geeks"]
 # accessing a element from the
# list using index number
print("Accessing a element from the list")
print(List[0])
print(List[2])

# Creating a Multi-Dimensional List
# (By Nesting a list inside a List)
List = [['Geeks', 'For'], ['Geeks']]

# accessing an element from the
# Multi-Dimensional List using
# index number
print("Accessing a element from a Multi-Dimensional list")
print(List[0][1])
print(List[1][0])
```

Output:

Accessing a element from the list
Geeks
Geeks
Accessing a element from a Multi-Dimensional list
For
Geeks

Negative indexing

In Python, negative sequence indexes represent positions from the end of the array. Instead of having to compute the offset as in List[len(List)-3], it is enough to just write List[-3]. Negative indexing means beginning from the end, -1 refers to the last item, -2 refers to the second-last item, etc.

```
List = [1, 2, 'Geeks', 4, 'For', 6, 'Geeks']

# accessing an element using negative indexing
print("Accessing element using negative indexing")

# print the last element of list
print(List[-1])

# print the third last element of list
print(List[-3])
```

Output:

Accessing element using negative indexing
Geeks
For

Removing Elements from the List

Using remove() method

Elements can be removed from the List by using the built-in **remove()** function but an Error arises if the element doesn't exist in the list. Remove() method only removes one element at a time, to remove a range of elements, the iterator is used. The remove() method removes the specified item.

```
# Python program to demonstrate
# Removal of elements in a List

# Creating a List
List = [1, 2, 3, 4, 5, 6,
        7, 8, 9, 10, 11, 12]
print("Initial List: ")
print(List)

# Removing elements from List
# using Remove() method
List.remove(5)
List.remove(6)
print("\nList after Removal of two elements: ")
print(List)

# Removing elements from List
# using iterator method
for i in range(1, 5):
    List.remove(i)
print("\nList after Removing a range of elements: ")
print(List)
```

Output:

Initial List:
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

List after Removal of two elements:
[1, 2, 3, 4, 7, 8, 9, 10, 11, 12]

List after Removing a range of elements:
[7, 8, 9, 10, 11, 12]

Using pop() method

Pop() function can also be used to remove and return an element from the list, but by default it removes only the last element of the list, to remove an element from a specific position of the List, the index of the element is passed as an argument to the pop() method.

```
List = [1,2,3,4,5]
 # Removing element from the
```

```
# Set using the pop() method
List.pop()
print("\nList after popping an element: ")
print(List)

# Removing element at a
# specific location from the
# Set using the pop() method
List.pop(2)
print("\nList after popping a specific element: ")
print(List)
```

Output:

```
List after popping an element:
[1, 2, 3, 4]

List after popping a specific element:
[1, 2, 4]
```

Slicing of a List

In Python List, there are multiple ways to print the whole List with all the elements, but to print a specific range of elements from the list, we use the Slice operation. Slice operation is performed on Lists with the use of a colon(:). To print elements from beginning to a range use [: Index], to print elements from end-use [:-Index], to print elements from specific Index till the end use [Index:], to print elements within a range, use [Start Index:End Index] and to print the whole List with the use of slicing operation, use [:]. Further, to print the whole List in reverse order, use [::-1].

Note – To print elements of List from rear-end, use Negative Indexes.

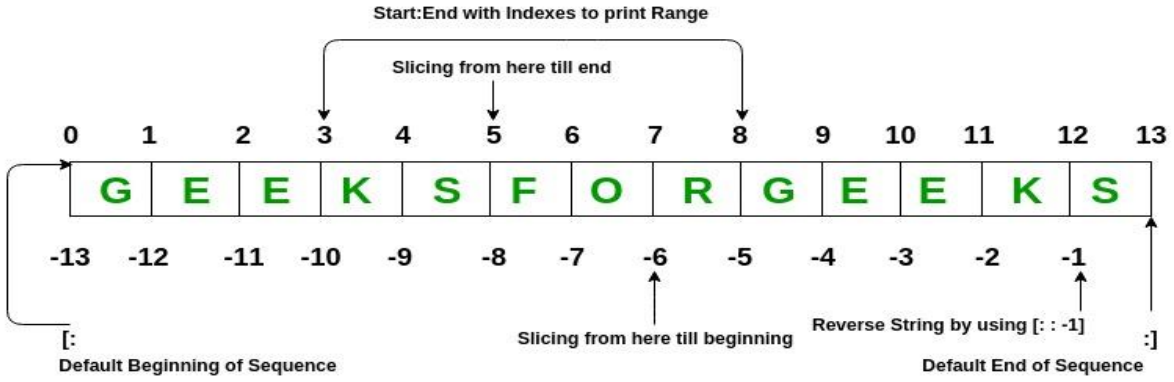


Figure 1 - List

```
# Python program to demonstrate
# Removal of elements in a List

# Creating a List
List = ['G', 'E', 'E', 'K', 'S', 'F',
        'O', 'R', 'G', 'E', 'E', 'K', 'S']
```

```
print("Initial List: ")
print(List)

# Print elements of a range
# using Slice operation
Sliced_List = List[3:8]
print("\nSlicing elements in a range 3-8: ")
print(Sliced_List)

# Print elements from a
# pre-defined point to end
Sliced_List = List[5:]
print("\nElements sliced from 5th "
      "element till the end: ")
print(Sliced_List)

# Printing elements from
# beginning till end
Sliced_List = List[:]
print("\nPrinting all elements using slice operation: ")
print(Sliced_List)
```

Output:

Initial List:
['G', 'E', 'E', 'K', 'S', 'F', 'O', 'R', 'G', 'E', 'E', 'K', 'S']

Slicing elements in a range 3-8:
['K', 'S', 'F', 'O', 'R']

Elements sliced from 5th element till the end:
['F', 'O', 'R', 'G', 'E', 'E', 'K', 'S']

Printing all elements using slice operation:
['G', 'E', 'E', 'K', 'S', 'F', 'O', 'R', 'G', 'E', 'E', 'K', 'S']

Negative index List slicing

```
# Creating a List
List = ['G', 'E', 'E', 'K', 'S', 'F',
        'O', 'R', 'G', 'E', 'E', 'K', 'S']
print("Initial List: ")
print(List)

# Print elements from beginning
# to a pre-defined point using Slice
Sliced_List = List[:-6]
```

```
print("\nElements sliced till 6th element from last: ")
print(Sliced_List)

# Print elements of a range
# using negative index List slicing
Sliced_List = List[-6:-1]
print("\nElements sliced from index -6 to -1")
print(Sliced_List)

# Printing elements in reverse
# using Slice operation
Sliced_List = List[::-1]
print("\nPrinting List in reverse: ")
print(Sliced_List)
```

Output:
Initial List:
['G', 'E', 'E', 'K', 'S', 'F', 'O', 'R', 'G', 'E', 'E', 'K', 'S']

Elements sliced till 6th element from last:
['G', 'E', 'E', 'K', 'S', 'F', 'O']

Elements sliced from index -6 to -1
['R', 'G', 'E', 'E', 'K']

Printing List in reverse:
['S', 'K', 'E', 'E', 'G', 'R', 'O', 'F', 'S', 'K', 'E', 'E', 'G']

List Comprehension

List comprehensions are used for creating new lists from other iterables like tuples, strings, arrays, lists, etc.

A list comprehension consists of brackets containing the expression, which is executed for each element along with the for loop to iterate over each element.

Syntax:

newList = [expression(element) for element in oldList if condition]

Example:

```
# Python program to demonstrate list comprehension in Python
 # below list contains square of all odd numbers from range 1
to 10

odd_square = [x ** 2 for x in range(1, 11) if x % 2 == 1]
print(odd_square)
```

Output:
[1, 9, 25, 49, 81]

For better understanding, the above code is similar to –

```
# for understanding, above generation is same as,
odd_square = []

for x in range(1, 11):
    if x % 2 == 1:
        odd_square.append(x**2)

print(odd_square)
```

Output:
[1, 9, 25, 49, 81]

Dictionary in Python is an unordered collection of data values, used to store data values like a map, which, unlike other Data Types that hold only a single value as an element, Dictionary holds **key:value** pair. Key-value is provided in the dictionary to make it more optimized.

Note – Keys in a dictionary don't allow Polymorphism.
Disclamer: *It is important to note that Dictionaries have been modified to maintain insertion order with the release of Python 3.7, so they are now ordered collection of data values.*

Creating a Dictionary

In Python, a Dictionary can be created by placing a sequence of elements within curly **{}** braces, separated by 'comma'. Dictionary holds pairs of values, one being the Key and the other corresponding pair element being its **Key:value**. Values in a dictionary can be of any data type and can be duplicated, whereas keys can't be repeated and must be *immutable*.

```
# Creating a Dictionary
# with Integer Keys
Dict = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print("\nDictionary with the use of Integer Keys: ")
print(Dict)

# Creating a Dictionary
# with Mixed keys
Dict = {'Name': 'Geeks', 1: [1, 2, 3, 4]}
print("\nDictionary with the use of Mixed Keys: ")
print(Dict)
```

Output:
Dictionary with the use of Integer Keys:
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
Dictionary with the use of Mixed Keys:
{1: [1, 2, 3, 4], 'Name': 'Geeks'}

Dictionary can also be created by the built-in function dict(). An empty dictionary can be created by just placing to curly braces{}.

```
# Creating an empty Dictionary
Dict = {}
print("Empty Dictionary: ")
print(Dict)

# Creating a Dictionary
# with dict() method
Dict = dict({1: 'Geeks', 2: 'For', 3:'Geeks'})
print("\nDictionary with the use of dict(): ")
print(Dict)

# Creating a Dictionary
# with each item as a Pair
Dict = dict([(1, 'Geeks'), (2, 'For')])
print("\nDictionary with each item as a pair: ")
print(Dict)
```

Output:

Empty Dictionary:
{}
Dictionary with the use of dict():
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
Dictionary with each item as a pair:
{1: 'Geeks', 2: 'For'}

Nested Dictionary:

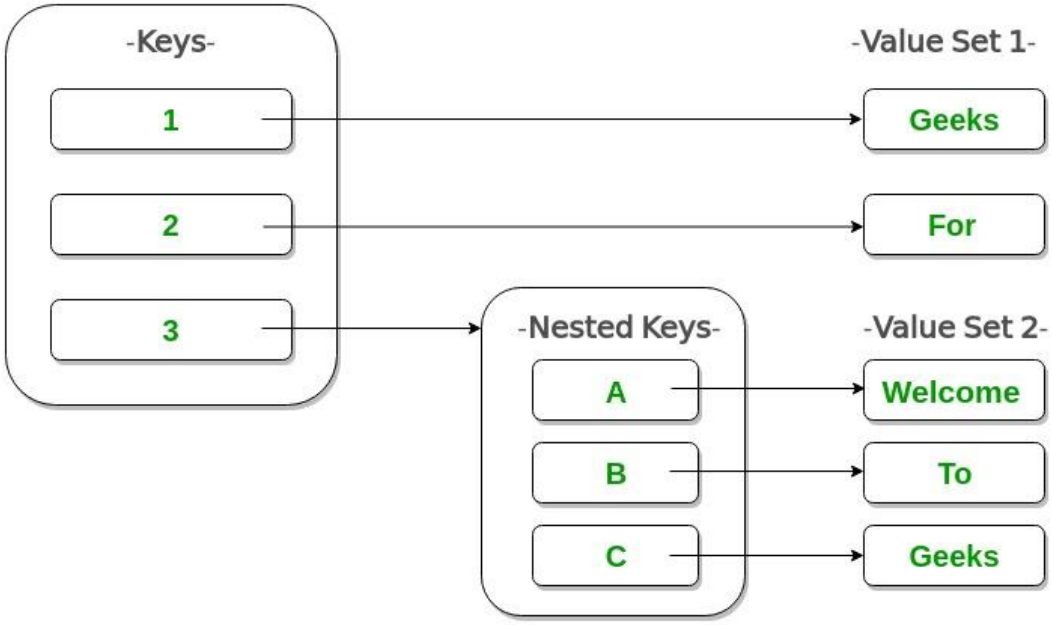


Figure 2 - Dictionary

```
# Creating a Nested Dictionary
# as shown in the below image
Dict = {1: 'Geeks', 2: 'For',
        3:{'A' : 'Welcome', 'B' : 'To', 'C' : 'Geeks'}}

print(Dict)
```

Output:

{1: 'Geeks', 2: 'For', 3: {'A': 'Welcome', 'B': 'To', 'C': 'Geeks'}}

Adding elements to a Dictionary

In Python Dictionary, the Addition of elements can be done in multiple ways. One value at a time can be added to a Dictionary by defining value along with the key e.g. Dict[Key] = 'Value'. Updating an existing value in a Dictionary can be done by using the built-in **update()** method. Nested key values can also be added to an existing Dictionary.

Note- While adding a value, if the key-value already exists, the value gets updated otherwise a new Key with the value is added to the Dictionary.

```
# Creating an empty Dictionary
Dict = {}
print("Empty Dictionary: ")
print(Dict)

# Adding elements one at a time
Dict[0] = 'Geeks'
Dict[2] = 'For'
Dict[3] = 1
print("\nDictionary after adding 3 elements: ")
print(Dict)

# Adding set of values
# to a single Key
Dict['Value_set'] = 2, 3, 4
print("\nDictionary after adding 3 elements: ")
print(Dict)

# Updating existing Key's Value
Dict[2] = 'Welcome'
print("\nUpdated key value: ")
print(Dict)

# Adding Nested Key value to Dictionary
Dict[5] = {'Nested' :{'1' : 'Life', '2' : 'Geeks'}}
print("\nAdding a Nested Key: ")
print(Dict)
```

Output:
Empty Dictionary:
{}

Dictionary after adding 3 elements:
{0: 'Geeks', 2: 'For', 3: 1}

Dictionary after adding 3 elements:
{0: 'Geeks', 2: 'For', 3: 1, 'Value_set': (2, 3, 4)}

Updated key value:
{0: 'Geeks', 2: 'Welcome', 3: 1, 'Value_set': (2, 3, 4)}

Adding a Nested Key:
{0: 'Geeks', 2: 'Welcome', 3: 1, 5: {'Nested': {'1': 'Life', '2': 'Geeks'}}, 'Value_set': (2, 3, 4)}

Accessing elements from a Dictionary
In order to access the items of a dictionary refer to its key name. Key can be used inside square brackets.

```
# Python program to demonstrate
# accessing a element from a Dictionary

# Creating a Dictionary
Dict = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}

# accessing a element using key
print("Accessing a element using key:")
print(Dict['name'])

# accessing a element using key
print("Accessing a element using key:")
print(Dict[1])
```

Output:
Accessing a element using key:
For
Accessing a element using key:
Geeks

There is also a method called **get()** that will also help in accessing the element from a dictionary.

```
# Creating a Dictionary
Dict = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}
```

```
# accessing a element using get()
# method
print("Accessing a element using get:")
print(Dict.get(3))
```

Output:

Accessing a element using get:
Geeks

Accessing an element of a nested dictionary

In order to access the value of any key in the nested dictionary, use indexing [] syntax.

```
# Creating a Dictionary
Dict = {'Dict1': {1: 'Geeks'},
        'Dict2': {'Name': 'For'}}

# Accessing element using key
print(Dict['Dict1'])
print(Dict['Dict1'][1])
print(Dict['Dict2']['Name'])
```

Output:

{1: 'Geeks'}
Geeks
For

Removing Elements from Dictionary

Using del keyword

In Python Dictionary, deletion of keys can be done by using the **del** keyword. Using the del keyword, specific values from a dictionary as well as the whole dictionary can be deleted. Items in a Nested dictionary can also be deleted by using the del keyword and providing a specific nested key and particular key to be deleted from that nested Dictionary.

```
# Initial Dictionary
Dict = { 5 : 'Welcome', 6 : 'To', 7 : 'Geeks',
        'A' : {1 : 'Geeks', 2 : 'For', 3 : 'Geeks'},
        'B' : {1 : 'Geeks', 2 : 'Life'}}
print("Initial Dictionary: ")
print(Dict)

# Deleting a Key value
del Dict[6]
print("\nDeleting a specific key: ")
print(Dict)
```

```
# Deleting a Key from
# Nested Dictionary
del Dict['A'][2]
print("\nDeleting a key from Nested Dictionary: ")
print(Dict)
```

Output:

Initial Dictionary:
{'A': {1: 'Geeks', 2: 'For', 3: 'Geeks'}, 'B': {1: 'Geeks', 2: 'Life'}, 5: 'Welcome', 6: 'To', 7: 'Geeks'}

Deleting a specific key:
{'A': {1: 'Geeks', 2: 'For', 3: 'Geeks'}, 'B': {1: 'Geeks', 2: 'Life'}, 5: 'Welcome', 7: 'Geeks'}

Deleting a key from Nested Dictionary:
{'A': {1: 'Geeks', 3: 'Geeks'}, 'B': {1: 'Geeks', 2: 'Life'}, 5: 'Welcome', 7: 'Geeks'}

Using pop() method

Pop() method is used to return and delete the value of the key specified.

```
# Creating a Dictionary
Dict = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}

# Deleting a key
# using pop() method
pop_ele = Dict.pop(1)
print('\nDictionary after deletion: ' + str(Dict))
print('Value associated to poped key is: ' + str(pop_ele))
```

Output:

Dictionary after deletion: {3: 'Geeks', 'name': 'For'}
Value associated to poped key is: Geeks

Using popitem() method

The popitem() returns and removes an arbitrary element (key, value) pair from the dictionary.

```
# Creating Dictionary
Dict = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}

# Deleting an arbitrary key
# using popitem() function
pop_ele = Dict.popitem()
print("\nDictionary after deletion: " + str(Dict))
print("The arbitrary pair returned is: " + str(pop_ele))
```

Output:
Dictionary after deletion: {3: 'Geeks', 'name': 'For'}
The arbitrary pair returned is: (1, 'Geeks')

Using clear() method

All the items from a dictionary can be deleted at once by using **clear()** method.

```
# Creating a Dictionary
Dict = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}


# Deleting entire Dictionary
Dict.clear()
print("\nDeleting Entire Dictionary: ")
print(Dict)
```

Output:
Deleting Entire Dictionary:
{}

Dictionary Methods

Methods	Description
copy()	They copy() method returns a shallow copy of the dictionary.
clear()	The clear() method removes all items from the dictionary.
pop()	Removes and returns an element from a dictionary having the given key.
popitem()	Removes the arbitrary key-value pair from the dictionary and returns it as tuple.
get()	It is a conventional method to access a value for a key.
dictionary_name.values()	returns a list of all the values available in a given dictionary.
str()	Produces a printable string representation of a dictionary.
update()	Adds dictionary dict2's key-values pairs to dict
setdefault()	Set dict[key]=default if key is not already in dict
keys()	Returns list of dictionary dict's keys
items()	Returns a list of dict's (key, value) tuple pairs
has_key()	Returns true if key in dictionary dict, false otherwise
fromkeys()	Create a new dictionary with keys from seq and values set to value.
type()	Returns the type of the passed variable.
cmp()	Compares elements of both dict.

Function

Functions break code into smaller chunks and are great for tasks that a program uses repeatedly. Instead of writing the same code each time the program needs to perform the task, just call the function!

1. First, we have to define a function. A function definition is actually the code that runs when we call the function. This would be the reusable code described above which we want to use in multiple places.
   A function does not always perform a single thing. It can perform a bunch of things in a certain category. e.g we want to write a function that converts the temperature to celsius when Fahrenheit temperature is passed and vice versa. We can do this using function parameters/arguments. We can use these to change the function behavior based on what parameters are passed.

```
def <function_name>(param1, param2, ....):
    // function body
```

2. After we have defined a function, we want to use it. Using a function is known as function calls. We do this by writing the function name and then using a pair of opening and closing round brackets.
   If we want to pass some function parameters we can write them inside the round brackets separated by commas.
   It is not mandatory to pass parameters if your function definition didnt specify them.

```
<function_name>(param1, param2, ....)
```

2) Solved Lab Activites:

Sr.No	*Allocated Time*	*Level of Complexity*	*CLO Mapping*
1	***10 mins***	***Low***	***CLO-6***
2	***10 mins***	***Low***	***CLO-6***
3	***10 mins***	***Low***	***CLO-6***
4	***15 mins***	***Medium***	***CLO-6***
5	***15 mins***	***Medium***	***CLO-6***
6	***15 mins***	***Medium***	***CLO-6***
7	***10 mins***	***Low***	***CLO-6***
8	***10 mins***	***Low***	***CLO-6***

Activity 1

Accept two lists from user and display their join.

Solution:

```
myList1=[]
print("Enter objects of first list...")
for i in range(5):
    val=input("Enter a value:")
    n=int(val)
    myList1.append(n)

myList2=[]
print("Enter objects of second list...")
for i in range(5):
    val=input("Enter a value:")
    n=int(val)
    myList2.append(n)

list3=myList1+myList2;
print(list3)
```

You will get the following output.

```
Enter objects of first list...
Enter a value:1
Enter a value:2
Enter a value:3
Enter a value:4
Enter a value:5
Enter objects of second list...
Enter a value:6
Enter a value:7
Enter a value:8
Enter a value:9
Enter a value:0
[1, 2, 3, 4, 5, 6, 7, 8, 9, 0]
>>>
```

Activity 2:

A palindrome is a string which is same read forward or backwards.
For example: "dad" is the same in forward or reverse direction. Another example is "aibohphobia" which literally means, an irritable fear of palindromes.
Write a function in python that receives a string and returns True if that string is a palindrome and False otherwise. Remember that difference between upper and lower case characters are ignored during this determination.

Solution:

```
def isPalindrome(word):
    temp=word[::-1]
    if temp.capitalize()==word.capitalize():
        return True
    else:
        return False


print(isPalindrome("deed"))
```

Activity 3:

Imagine two matrices given in the form of 2D lists as under; a = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]
b = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

Write a python code that finds another matrix/2D list that is a product of and b, i.e., C=a*b
Solution:

```
for indrow in range (3):
    c.append ([])
    for indcol in range(3):
        c[indrow].append (0)
        for indaux in range (3):
            c[indrow][indcol] += a[indrow][indaux] * b[indcol][inc

print (c)
```

Activity 4:

A closed polygon with N sides can be represented as a list of tuples of N connected coordinates, i.e., [(x_1,y_1), (x_2,y_2), (x_3,y_3), . . . , (x_N,y_N)]. A sample polygon with 6 sides (N=6) is shown below.

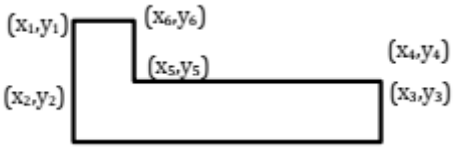


Figure 3 - Polygon

Write a python function that takes a list of N tuples as input and returns the perimeter of the polygon. Remember that your code should work for any value of N.

Hint: A perimeter is the sum of all sides of a polygon.

Solution:

```
def perimeter(listing):
    leng=len(listing)
    perimeter=0;
    for i in range(0,leng-1):
        dist = (((listing[i][0]-listing[i+1][0])**2)+
                ((listing[i][1]-listing[i+1][1])**2))**0.5
        perimeter = perimeter + dist
    perimeter = perimeter + (((listing[0][0]-listing[leng-1][0])**2)
                             +((listing[0][1]-listing[leng-1][1])**2))**0.5
    return perimeter

L = [(1,3), (2,7), (3,9), (-1,8)]
print(perimeter(L))
```

Activity 5:

Imagine two sets A and B containing numbers. Without using built-in set functionalities, write your own function that receives two such sets and returns another set C which is a symmetric difference of the two input sets. (A symmetric difference between A and B will return a set C which contains only those items that appear in one of A or B. Any items that appear in both sets are not included in C). Now compare the output of your function with the following built-in functions/operators.

- ✓ ***A.symmetric_difference(B)***
- ✓ ***B.symmetric_difference(A)***
- ✓ ***A ^ B***
- ✓ ***B ^ A***

Solution:

```
#Function defined
def symmDiff(a,b):
    e=set()   #empty set
    for i in a:   #for loop used to access in a
        if i  not in b:
            e.add(i)
    for i in b: #for loop used to access in b
        if i  not in a:
            e.add(i)
    return e


set1={0,1,2,4,5}
set2={4,5,7,8,9}
print(symmDiff(set1,set2))

#verification using inbuilt function
print(set1.symmetric_difference(set2))
print(set2.symmetric_difference(set1))
print(set1^set2)
print(set2^set1)
```

Activity 6:

Create a Python program that contains a dictionary of names and phone numbers. Use a tuple of separate first and last name values for the key field. Initialize the dictionary with at least three names and numbers. Ask the user to search for a phone number by entering a first and last name. Display the matching number if found, or a message if not found.

Solution:

```
sample={("sohaib","ali"):"0246585468445", ("aib","li"):"02465854645",
        ("sib","ai"):"0246585468445",}
firstName = input("enter first name")
lastName = input("enter last name")

searchTuple = (firstName, lastName)
if searchTuple in sample:
    print(sample[searchTuple])
else:
    print("name not found")
```

Activity 7:

Python program to illustrate a function definition

Solution:

```
def example_function():
    x = 5
    print("The value of x is:", x)
example_function()
```

Output

```
The value of x is:5
```

Activity 8:

Python program to illustrate a function definition with params

```
def example_function(num1,num2):
    x = num1 + num2
    print("The sum is:", x)
example_function(1,2)
example_function(2,10)
```

Output

```
The sum is:3
The sum is:12
```

Activity 9:

Python program to illustrate a function definition with return

```
def                                          example_function(num1,num2):
    return                    num1                   +                    num2

print(example_function(1,2))
print(example_function(3,23))
```

Output

```
3
26
```

Activity 10:

Python program to illustrate a function definition with default/optional params

```
Def example_function(num1,num2=10):
    return num1 + num2

print(example_function(1,2))
print(example_function(1))
print(example_function(8))
```

Output

```
3
11
18
```

3) Graded Lab Tasks

Note: The instructor can design graded lab activities according to the level of difficulty and complexity of the solved lab activities. The lab tasks assigned by the instructor should be evaluated in the same lab

Lab Task 1:

Create two lists based on the user values. Merge both the lists and display in sorted order.

Lab Task 2:

Repeat the above activity to find the smallest and largest element of the list. (Suppose all the elements are integer values)

Lab Task 3:

The derivate of a function f(x) is a measurement of how quickly the function f changes with respect to change in its domain x. This measurement can be approximated by the following relation,

$$\frac{d}{dx}f(x) = \frac{f(x+h) - \mathrm{f}(x)}{h}$$

Where h represents a small increment in x. You have to prove the following relation

$$\frac{d}{dx}\sin(x) = \cos(x)$$

Imagine x being a list that goes from –pi to pi with an increment of 0.001. You can approximate the derivative by using the following approximation,

$$\frac{d}{dx}\sin(x) = \frac{sin(x+h) - \sin(x)}{h}$$

*In your case, assume h = 0.001. That is at each point in x, compute the right hand side of above equation and compare whether the output value is equivalent to cos(x). Also print the corresponding values of ( ) and cos(x) for every point. Type ''from math import *'' at the start of your program to use predefined values of pi, and sin and cos functions. What happens if you increase the interval h from 0.001 to 0.01 and then to 0.1?*

Lab Task 4:

For this exercise, you will keep track of when our friend's birthdays are, and be able to find that information based on their name. Create a dictionary (in your file) of names and birthdays. When you run your program it should ask the user to enter a name, and return the birthday of that person back to them. The interaction should look something like this:

>>> Welcome to the birthday dictionary. We know the birthdays of:

Albert Einstein
Benjamin Franklin
Ada Lovelace
>>> Who's birthday do you want to look up?
Benjamin Franklin
>>> Benjamin Franklin's birthday is 01/17/1706.

Lab Task 5

Create a dictionary by extracting the keys from a given dictionary
Write a Python program to create a new dictionary by extracting the mentioned keys from the below dictionary.

Given dictionary:

```
sample_dict = {
   "name": "Kelly",
   "age": 25,
   "salary": 8000,
   "city": "New york"}

# Keys to extract
keys = ["name", "salary"]
```

Expected output:

{'name': 'Kelly', 'salary': 8000}

Lab Task 6

Write a calculator function that takes in as a param two numbers and a string specifying the operation and returns the result. It should be able to do addition, subtraction, multiplication, and division.

Lab Task 7

Write a program that takes in 4 numbers as params and returns the largest number of them all.

Lab Task 8

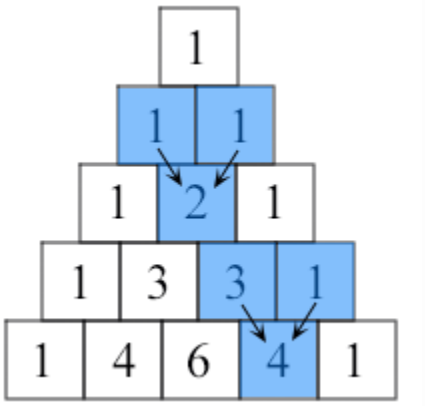
Write a function that takes in a number and returns the Fibonacci sequence till that number.
The Fibonacci Sequence is the series of numbers:
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
The next number is found by adding up the two numbers before it

Lab Task 9

Write a function that takes in params for different shape measurements and returns the area of that shape depending upon those measurements and the type of shape. If the type of shape is not specified it should by default calculate the area of a rectangle. It should be able to calculate the area of square, rectangle, circle, and triangle.

Lab Task 10

Write a Python function that prints out the first n rows of Pascal's triangle
Sample Pascal's triangle:



Each number is the two numbers above it added together

Lab 02

Numpy Introduction

Objective:

The objective of this lab is to introduce Numpy with the help of examples and learning tasks.

Activity Outcomes:

- The activities provide hands - on practice with the following topics
- Implement a array in numpy and index array in different ways.
- Implement datatypes in numpy.
- Implement array math in numpy.
- Implement array broadcasting in numpy

Instructor Note:

Read Chapter 3 from reference book "Oliphant, Travis E. *A guide to NumPy*. Vol. 1. USA: Trelgol Publishing, 2006.".

1) Useful Concepts

NumPy provides two fundamental objects: an N-dimensional array object (ndarray) and a universal function object (ufunc). An N-dimensional array is a homogeneous collection of "items" indexed using N integers. There are two essential pieces of information that define an N-dimensional array: 1) the shape of the array, and 2) the kind of item the array is composed of. The shape of the array is a tuple of N integers (one for each dimension) that provides information on how far the index can vary along that dimension. The other important information describing an array is the kind of item the array is composed of. Because every ndarray is a homogeneous collection of exactly the same data-type, every item takes up the same size block of memory, and each block of memory in the array is interpreted in exactly the same way.
For example, consider the following piece of code:

```
>>> a = array([[1,2,3],[4,5,6]])
>>> a.shape
(2, 3)
>>> a.dtype
dtype('int32')
```

This code defines an array of size 2×3 composed of 4-byte (little-endian) integer elements (on my 32-bit platform). We can index into this two-dimensional array using two integers: the first integer running from 0 to 1 inclusive and the second from 0 to 2 inclusive. For example, index (1, 1) selects the element with value 5:

```
>>> a[1,1]
5
```

All code shown in the shaded-boxes in this book has been (automatically) executed on a particular version of NumPy. The output of the code shown below shows which version of NumPy was used to create all of the output in your copy of this book.

```
>>> import numpy; print numpy._version_
1.0.2.dev3478
```

Data-Type Descriptors In NumPy, an ndarray is an N-dimensional array of items where each item takes up a fixed number of bytes. Typically, this fixed number of bytes represents a number (e.g. integer or floating-point). However, this fixed number of bytes could also represent an arbitrary record made up of any collection of other data types. NumPy achieves this flexibility through the use of a data-type (dtype) object. Every array has an associated dtype object which describes the layout of the array data. Every dtype object, in turn, has an associated Python type-object that determines exactly what type of Python object is returned when an element of the array is accessed. The dtype objects are flexible enough to contain references to arrays of other dtype objects and, therefore, can be used to define nested records. This advanced functionality will be described in better detail later as it is mainly useful for the recarray (record array) subclass that will also be defined later. However, all ndarrays can enjoy the flexibility provided by the dtype objects. Figure 3.1 provides a conceptual diagram showing the relationship between the ndarray, its associated data-type object, and an array-scalar that is returned when a single-element of the array is accessed. Note that the data-type points to the type-object of the array scalar. An array scalar is returned using the type-object and a particular element of the ndarray.

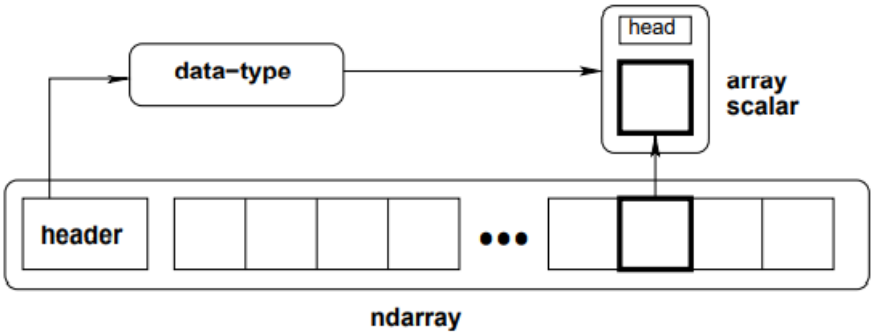


Figure 3.1: Conceptual diagram showing the relationship between the three fundamental objects used to describe the data in an array: 1) the ndarray itself, 2) the data-type object that describes the layout of a single fixed-size element of the array, 3) the array-scalar Python object that is returned when a single element of the array is accessed.

Every dtype object is based on one of 21 built-in dtype objects. These builtin objects allow numeric operations on a wide-variety of integer, floating-point, and complex data types. Associated with each data-type is a Python type object whose instances are array scalars. This type-object can be obtained using the type attribute of the dtype object. Python typically defines only one data-type of a particular data class (one integer type, one floating-point type, etc.). This can be convenient for some applications that don't

need to be concerned with all the ways data can be represented in a computer. For scientific applications, however, this is not always true. As a result, in NumPy, their are 21 different fundamental Python data-type-descriptor objects built-in. These descriptors are mostly based on the types available in the C language that CPython is written in. However, there are few types that are extremely flexible, such as str , unicode , and void. The fundamental data-types are shown in Table 3.1. Along with their (mostly) C-derived names, the integer, float, and complex data-types are also available using a bit-width convention so that an array of the right size can always be ensured (e.g. int8, float64, complex128). The C-like names are also accessible using a character code which is also shown in the table (use of the character codes, however, is discouraged). Names for the data types that would clash with standard Python object names are followed by a trailing underscore, ' '. These data types are so named because they use the same underlying precision as the corresponding Python data types. Most scientific users should be able to use the array-enhanced scalar objects in place of the Python objects. The array-enhanced scalars inherit from the Python objects they can replace and should act like them under all circumstances (except for how errors are handled in math computations).

Table 3.1: Built-in array-scalar types corresponding to data-types for an ndarray. The bold-face types correspond to standard Python types. The object type is special because arrays with dtype='O' do not return an array scalar on item access but instead return the actual object referenced in the array.

Type	Bit-Width	Character
bool_	boolXX	'?'
byte	intXX	'b'
short		'h'
intc		'i'
int_		'l'
longlong		'q'
intp		'p'
ubyte	uintXX	'B'
ushort		'H'
uintc		'I'
uint		'L'
ulonglong		'Q'
uintp		'P'
single	floatXX	'f'
float_		'd'
longfloat		'g'
csingle	complexXX	'F'
complex_		'D'
clongfloat		'G'
object_		'O'
str_		'S#'
unicode_		'U#'
void		'V#'

Three of the data types are flexible in that they can have items that are of an arbitrary size: the str type, the unicode type, and the void type. While, you can specify an arbitrary size for these types, every item in

an array is still of that specified size. The void type, for example, allows for arbitrary records to be defined as elements of the array, and can be used to define exotic types built on top of the basic ndarray

The fundamental data-types are arranged into a hierarchy of Python type objects shown in Figure 3.2. Each of the leaves on this hierarchy correspond to actual data-types that arrays can have (in other words, there is a built in dtype object associated with each of these new types). They also correspond to new Python objects that can be created. These new objects are "scalar" types corresponding to each fundamental data-type.

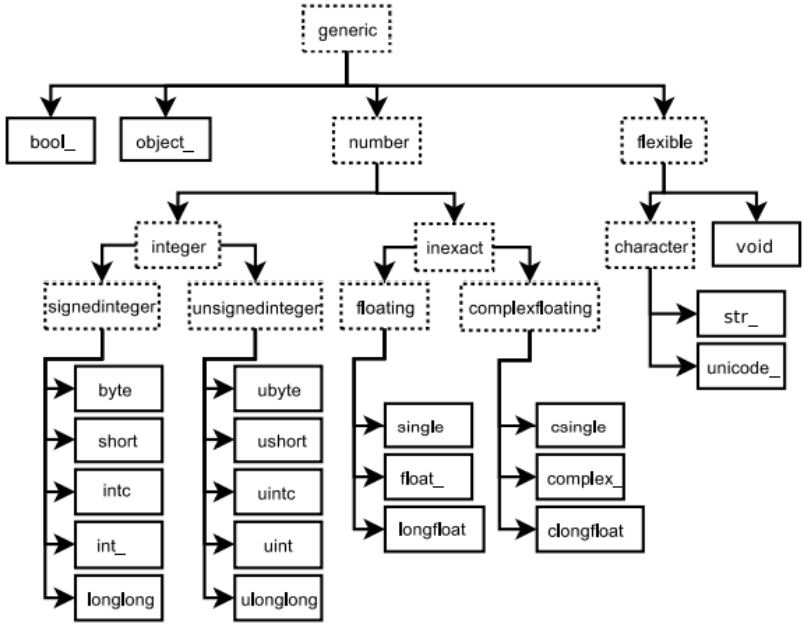


Figure 3.2: Hierarchy of type objects representing the array data types. Not shown are the two integer types intp and uintp which just point to the integer type that holds a pointer for the platform. All the number types can be obtained using bit-width names as well

2) Solved Lab Activites

Sr.no	*Allocated time*	*Level of complexity*	*CLO mapping*
Activity 1	***20 mins***	***Low***	***CLO-4***
Activity 2	***10 mins***	***Low***	***CLO-4***
Activity 3	***20 mins***	***Low***	***CLO-4***
Activity 4	***10 mins***	***Low***	***CLO-4***

Activity 1

This activity introduces array and array indexing in numpy python.

Arrays:
A numpy array is a grid of values, all of the same type, and is indexed by a tuple of nonnegative integers. The number of dimensions is the *rank* of the array; the *shape* of an array is a tuple of integers giving the size of the array along each dimension.

We can initialize numpy arrays from nested Python lists, and access elements using square brackets:

Syntax :

```
import numpy as np

a = np.array([1, 2, 3])   # Create a rank 1 array
print(type(a))            # Prints "<class
'numpy.ndarray'>"
print(a.shape)            # Prints "(3,)"
print(a[0], a[1], a[2])   # Prints "1 2 3"
a[0] = 5                  # Change an element of the array
print(a)                  # Prints "[5, 2, 3]"

b = np.array([[1,2,3],[4,5,6]])    # Create a rank 2 array
print(b.shape)                     # Prints "(2, 3)"
print(b[0, 0], b[0, 1], b[1, 0])   # Prints "1 2 4"
```

Numpy also provides many functions to create arrays:

Syntax :

```
import numpy as np

a = np.zeros((2,2))   # Create an array of all zeros
print(a)              # Prints "[[ 0.  0.]
                      #          [ 0.  0.]]"

b = np.ones((1,2))    # Create an array of all ones
print(b)              # Prints "[[ 1.  1.]]"

c = np.full((2,2), 7)  # Create a constant array
print(c)               # Prints "[[ 7.  7.]
                       #          [ 7.  7.]]"

d = np.eye(2)         # Create a 2x2 identity matrix
print(d)              # Prints "[[ 1.  0.]
                      #          [ 0.  1.]]"

e = np.random.random((2,2))  # Create an array filled with
random values
```

```
print(e)                          # Might print "[[ 0.91940167
0.08143941]
                                  #               [ 0.68744134
0.87236687]]"
```

Array indexing
Numpy offers several ways to index into arrays.

Slicing: Similar to Python lists, numpy arrays can be sliced. Since arrays may be multidimensional, you must specify a slice for each dimension of the array:

Syntax :

```
import numpy as np

# Create the following rank 2 array with shape (3, 4)
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])

# Use slicing to pull out the subarray consisting of the
first 2 rows
# and columns 1 and 2; b is the following array of shape
(2, 2):
# [[2 3]
#  [6 7]]
b = a[:2, 1:3]

# A slice of an array is a view into the same data, so
modifying it
# will modify the original array.
print(a[0, 1])   # Prints "2"
b[0, 0] = 77     # b[0, 0] is the same piece of data as
a[0, 1]
print(a[0, 1])   # Prints "77"
```

Integer array indexing: When you index into numpy arrays using slicing, the resulting array view will always be a subarray of the original array. In contrast, integer array indexing allows you to construct arbitrary arrays using the data from another array. Here is an example:

Syntax :

```
import numpy as np

a = np.array([[1,2], [3, 4], [5, 6]])

# An example of integer array indexing.
# The returned array will have shape (3,) and
print(a[[0, 1, 2], [0, 1, 0]])  # Prints "[1 4 5]"

# The above example of integer array indexing is
equivalent to this:
print(np.array([a[0, 0], a[1, 1], a[2, 0]]))  # Prints "[1
4 5]"

# When using integer array indexing, you can reuse the
same
# element from the source array:
print(a[[0, 0], [1, 1]])  # Prints "[2 2]"

# Equivalent to the previous integer array indexing
example
print(np.array([a[0, 1], a[0, 1]]))  # Prints "[2 2]"
```

Boolean array indexing: Boolean array indexing lets you pick out arbitrary elements of an array. Frequently this type of indexing is used to select the elements of an array that satisfy some condition. Here is an example:

Syntax :

```
import numpy as np

a = np.array([[1,2], [3, 4], [5, 6]])

bool_idx = (a > 2)   # Find the elements of a that are
bigger than 2;
                     # this returns a numpy array of
Booleans of the same
                     # shape as a, where each slot of
bool_idx tells
                     # whether that element of a is > 2.

print(bool_idx)      # Prints "[[False False]
                     #          [ True  True]
                     #          [ True  True]]"

# We use boolean array indexing to construct a rank 1
array
# consisting of the elements of a corresponding to the
True values
# of bool_idx
```

```
print(a[bool_idx])  # Prints "[3 4 5 6]"

# We can do all of the above in a single concise
statement:
print(a[a > 2])     # Prints "[3 4 5 6]"
```

Activity 2

***This activity introduces datatypes in numpy python*.**

Datatypes:

Every numpy array is a grid of elements of the same type. Numpy provides a large set of numeric datatypes that you can use to construct arrays. Numpy tries to guess a datatype when you create an array, but functions that construct arrays usually also include an optional argument to explicitly specify the datatype. Here is an example:

Syntax :

```
import numpy as np

x = np.array([1, 2])    # Let numpy choose the datatype
print(x.dtype)          # Prints "int64"

x = np.array([1.0, 2.0])    # Let numpy choose the datatype
print(x.dtype)              # Prints "float64"

x = np.array([1, 2], dtype=np.int64)    # Force a
particular datatype
print(x.dtype)                          # Prints "int64"
```

Activity 3

***This activity introduces array math in numpy python*.**

Array math:

Basic mathematical functions operate elementwise on arrays, and are available both as operator overloads and as functions in the numpy module:

Syntax :

```
import numpy as np

x = np.array([[1,2],[3,4]], dtype=np.float64)
y = np.array([[5,6],[7,8]], dtype=np.float64)

# Elementwise sum; both produce the array
# [[ 6.0  8.0]
#  [10.0 12.0]]
print(x + y)
print(np.add(x, y))

# Elementwise difference; both produce the array
```

```
# [[-4.0 -4.0]
#  [-4.0 -4.0]]
print(x - y)
print(np.subtract(x, y))

# Elementwise product; both produce the array
# [[ 5.0 12.0]
#  [21.0 32.0]]
print(x * y)
print(np.multiply(x, y))

# Elementwise division; both produce the array
# [[ 0.2         0.33333333]
#  [ 0.42857143  0.5       ]]
print(x / y)
print(np.divide(x, y))

# Elementwise square root; produces the array
# [[ 1.          1.41421356]
#  [ 1.73205081  2.        ]]
print(np.sqrt(x))
```

Note that unlike MATLAB, * is elementwise multiplication, not matrix multiplication. We instead use the *dot* function to compute inner products of vectors, to multiply a vector by a matrix, and to multiply matrices. *dot* is available both as a function in the numpy module and as an instance method of array objects:

Syntax :

```
import numpy as np

x = np.array([[1,2],[3,4]])
y = np.array([[5,6],[7,8]])

v = np.array([9,10])
w = np.array([11, 12])

# Inner product of vectors; both produce 219
print(v.dot(w))
print(np.dot(v, w))

# Matrix / vector product; both produce the rank 1 array
[29 67]
print(x.dot(v))
print(np.dot(x, v))

# Matrix / matrix product; both produce the rank 2 array
# [[19 22]
```

```
#  [43 50]]
print(x.dot(y))
print(np.dot(x, y))
```

Numpy provides many useful functions for performing computations on arrays; one of the most useful is *sum*:

Syntax :

```
import numpy as np

x = np.array([[1,2],[3,4]])

print(np.sum(x))  # Compute sum of all elements; prints
"10"
print(np.sum(x, axis=0))  # Compute sum of each column;
prints "[4 6]"
print(np.sum(x, axis=1))  # Compute sum of each row;
prints "[3 7]"
```

Apart from computing mathematical functions using arrays, we frequently need to reshape or otherwise manipulate data in arrays. The simplest example of this type of operation is transposing a matrix; to transpose a matrix, simply use the *T* attribute of an array object:

Syntax :

```
import numpy as np

x = np.array([[1,2], [3,4]])
print(x)    # Prints "[[1 2]
            #          [3 4]]"
print(x.T)  # Prints "[[1 3]
            #          [2 4]]"

# Note that taking the transpose of a rank 1 array does
nothing:
v = np.array([1,2,3])
print(v)    # Prints "[1 2 3]"
print(v.T)  # Prints "[1 2 3]"
```

String objects have a bunch of useful methods; for example:

Syntax :

```
s = "hello"
print(s.capitalize())  # Capitalize a string; prints
"Hello"
print(s.upper())       # Convert a string to uppercase;
prints "HELLO"
print(s.rjust(7))      # Right-justify a string, padding
with spaces; prints "  hello"
print(s.center(7))     # Center a string, padding with
```

```
spaces; prints " hello "
print(s.replace('l', '(ell)'))  # Replace all instances of
one substring with another;
                                # prints "he(ell)(ell)o"
print('  world '.strip())  # Strip leading and trailing
whitespace; prints "world"
```

Acitity 4

This activity introduces array broadcasting in numpy python.

Broadcasting is a powerful mechanism that allows numpy to work with arrays of different shapes when performing arithmetic operations. Frequently we have a smaller array and a larger array, and we want to use the smaller array multiple times to perform some operation on the larger array. For example, suppose that we want to add a constant vector to each row of a matrix. We could do it like this:

Syntax :

```
import numpy as np

# We will add the vector v to each row of the matrix x,
# storing the result in the matrix y
x = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
v = np.array([1, 0, 1])
y = np.empty_like(x)   # Create an empty matrix with the
same shape as x

# Add the vector v to each row of the matrix x with an
explicit loop
for i in range(4):
    y[i, :] = x[i, :] + v

# Now y is the following
# [[ 2  2  4]
#  [ 5  5  7]
#  [ 8  8 10]
#  [11 11 13]]
print(y)
```

This works; however when the matrix *x* is very large, computing an explicit loop in Python could be slow. Note that adding the vector *v* to each row of the matrix *x* is equivalent to forming a matrix *vv* by stacking multiple copies of *v* vertically, then performing elementwise summation of x and *vv*. We could implement this approach like this:

Syntax :

```
import numpy as np

# We will add the vector v to each row of the matrix x,
# storing the result in the matrix y
x = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
v = np.array([1, 0, 1])
vv = np.tile(v, (4, 1))   # Stack 4 copies of v on top of
each other
print(vv)                 # Prints "[[1 0 1]
                          #         [1 0 1]
                          #         [1 0 1]
                          #         [1 0 1]]"
y = x + vv  # Add x and vv elementwise
print(y)  # Prints "[[ 2  2  4
          #         [ 5  5  7]
          #         [ 8  8 10]
          #         [11 11 13]]"
```

The line *y = x + v* works even though x has shape *(4, 3)* and v has shape *(3,)* due to broadcasting; this line works as if *v* actually had shape *(4, 3)*, where each row was a copy of *v*, and the sum was performed elementwise.

3)Graded Lab Tasks

Note: The instructor can design graded lab activities according to the level of difficulty and complexity of the solved lab activities. The lab tasks assigned by the instructor should be evaluated in the same lab

Lab Task 1

Implement linear regression using numpy

Lab 03

Logic Programming in Python: Validating primes, Parsing a Family Tree

Objective:

We are going to learn how to write programs using logic programming. The students will get hands on experience of implementation of validating primes and parsing a family tree.

Activity Outcomes:

The activities provide hands - on practice with the following topics
Installing Python packages

- Validating primes
- Parsing a family tree

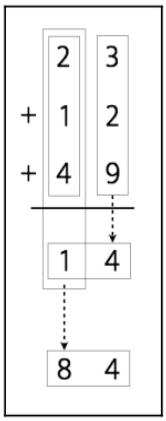
Instructor Note:

As pre-lab activity, read Chapter 06 from the text book Joshi, Prateek. *Artificial intelligence with python*. Packt Publishing Ltd, 2017.

1)Useful Concepts

Logic programming is a programming paradigm, which basically means it is a particular way to approach programming. Before we talk about what it constitutes and how it is relevant in Artificial Intelligence, let's talk a bit about programming paradigms. The concept of programming paradigms arises owing to the need to classify programming languages. It refers to the way computer programs solve problems through code. Some programming paradigms are primarily concerned with implications or the sequence of operations used to achieve the result. Other programming paradigms are concerned about how we organize the code.

We use these expressions and rules to generate the output. For example, let's say we want to compute the sum of 23, 12, and 49:



The procedure would be as follows: 23 + 12 + 49 => (2 + 1 + 4 + 1)4 => 84 On the other hand, if we want to deduce something, we need to start from a conjecture. We then need to construct a proof according to a set of rules. In essence, the process computation is mechanical, whereas the process of deduction is more creative. When we write a program in the logic programming paradigm, we specify a set of statements based on facts and rules about the problem domain and the solver solves it using this information.

Understanding the building blocks of logic programming: In programming object-oriented or imperative paradigms, we always have to specify how a variable is defined. In logic programming, things work a bit differently. We can pass an uninstantiated argument to a function and the interpreter will instantiate these variables for us by looking at the facts defined by the user. This is a powerful way of approaching the variable matching problem. The process of matching variables with different items is called unification. This is one of the places logic programming really stands apart. We need to specify something called relations in logic programming. These relations are defined by means of clauses called facts and rules. Facts are just statements that are truths about our program and the data that it's operating on. The syntax is pretty straightforward. For example, Donald is Allan's son, can be a fact whereas, Who is Allan's son? cannot be a fact. Every logic program needs facts to work with, so that it can achieve the given goal based on them. Rules are the things we have learned about how to express various facts and how to query them. They are the constraints that we have to work with and they allow us to make conclusions about the problem domain. For example, let's say you are working on building a chess engine. You need to specify all the rules about how each piece can move on the chessboard. In essence, the final conclusion is valid only if all the relations are true.

Solving problems using logic programming Logic programming looks for solutions by using facts and rules. We need to specify a goal for each program. In the case where a logic program and a goal don't contain any variables, the solver comes up with a tree that constitutes the search space for solving the problem and getting to the goal. One of the most important things about logic programming is how we treat the rules. Rules can be viewed as logical statements. Let's consider the following:

Kathy likes chocolate => Alexander loves Kathy

This can be read as an implication that says, If Kathy likes chocolate, then Alexander loves Kathy. It can also be construed as Kathy likes chocolate implies Alexander loves Kathy. Similarly, let's consider the following rule:

Crime movies, English => Martin Scorsese

It can be read as the implication If you like crime movies in English, then you would like movies made by Martin Scorsese. This construction is used in various forms throughout logic programming to solve various types of problems. Let's go ahead and see how to solve these problems in Python.

Matching mathematical expressions:

We encounter mathematical operations all the time. Logic programming is a very efficient way of comparing expressions and finding out unknown values. Let's see how to do that.

Create a new Python file and import the following packages:

```
from logpy import run, var, fact
import logpy.assoccomm as la
```

Define a couple of mathematical operations:

```
# Define mathematical operations
add = 'addition'
mul = 'multiplication'
```

Both addition and multiplication are commutative operations. Let's specify that:

Declare that these operations are commutative

using the facts system

```
fact(la.commutative, mul)
fact(la.commutative, add)
fact(la.associative, mul)
fact(la.associative, add)
```

Let's define some variables:

```
# Define some variables
a, b, c = var('a'), var('b'), var('c')
```

Consider the following expression:

```
expression_orig = 3 x (-2) + (1 + 2 x 3) x (-1)
```

Let's generate this expression with masked variables. The first expression would be:

```
expression1 = (1 + 2 x a) x b + 3 x c
```

The second expression would be:

```
expression2 = c x 3 + b x (2 x a + 1)
```

The third expression would be:

```
expression3 = (((2 x a) x b) + b) + 3 x c
```

If you observe carefully, all three expressions represent the same basic expression. Our goal is to match these expressions with the original expression to extract the unknown values:

Generate expressions

```
expression_orig = (add, (mul, 3, -2), (mul, (add, 1, (mul, 2, 3)), -1))
expression1 = (add, (mul, (add, 1, (mul, 2, a)), b), (mul, 3, c))
expression2 = (add, (mul, c, 3), (mul, b, (add, (mul, 2, a), 1)))
expression3 = (add, (add, (mul, (mul, 2, a), b), b), (mul, 3, c))
```

Compare the expressions with the original expression. The method run is commonly used in logpy. This method takes the input arguments and runs the expression. The first argument is the number of values, the second argument is a variable, and the third argument is a function:

Compare expressions

```
print(run(0, (a, b, c), la.eq_assoccomm(expression1, expression_orig)))
print(run(0, (a, b, c), la.eq_assoccomm(expression2, expression_orig)))
print(run(0, (a, b, c), la.eq_assoccomm(expression3, expression_orig)))
```

The full code is given in *expression_matcher.py*. If you run the code, you will see the following output on your Terminal

```
((3, -1, -2),)
((3, -1, -2),)
()
```

The three values in the first two lines represent the values for a, b, and c. The first two expressions matched with the original expression, whereas the third one returned nothing. This is because even though the third expression is mathematically the same, it is structurally different. Pattern comparison works by comparing the structure of the expressions.

2) Solved Lab Activites

Sr.no	***Allocated time***	***Level of complexity***	***CLO mapping***
1	***5 mins***	***Low***	***CLO-4***
2	***25 mins***	***Medium***	***CLO-4***
3	***30 mins***	***Medium***	***CLO-4***

Activity 1

Installing Python packages

Before we start logic programming in Python, we need to install a couple of packages. The package logpy is a Python package that enables logic programming in Python. We will also be using SymPy for some of the problems. So let's go ahead and install logpy and sympy using pip:

- $ pip3 install logpy
- $ pip3 install sympy

If you get an error during the installation process for logpy, you can install from source at https://github.com/logpy/logpy. Once you have successfully installed these packages, you can proceed to the next section.

Activity 2

Validating primes

Let's see how to use logic programming to check for prime numbers. We will use the constructs available in logpy to determine which numbers in the given list are prime, as well as finding out if a given number is a prime or not. Create a new Python file and import the following packages:

```
import itertools as it
import logpy.core as lc
from sympy.ntheory.generate import prime, isprime
```

Next, define a function that checks if the given number is prime depending on the type of data. If it's a number, then it's pretty straightforward. If it's a variable, then we have to run the sequential operation. To give a bit of background, the method conde is a goal constructor that provides logical *AND* and *OR* operations. The method *condeseq* is like *conde*, but it supports generic iterator of goals:

Check if the elements of x are prime

```
def check_prime(x):
if lc.isvar(x):
    return lc.condeseq([(lc.eq, x, p)] for p in map(prime,
    it.count(1)))
else: return lc.success if isprime(x) else lc.fail
```

Declare the variable x that will be used:

```
# Declate the variable
x = lc.var()
```

Define a set of numbers and check which numbers are prime. The method membero checks if a given number is a member of the list of numbers specified in the input argument:

```
# Check if an element in the list is a prime number
list_nums = (23, 4, 27, 17, 13, 10, 21, 29, 3, 32, 11, 19)
print('\nList of primes in the list:')
print(set(lc.run(0, x, (lc.membero, x, list_nums), (check_prime, x))))
```

Let's use the function in a slightly different way now by printing the first 7 prime numbers:

```
# Print first 7 prime numbers
print('\nList of first 7 prime numbers:')
print(lc.run(7, x, check_prime(x)))
```

The full code is given in prime.py. If you run the code, you will see the following output:

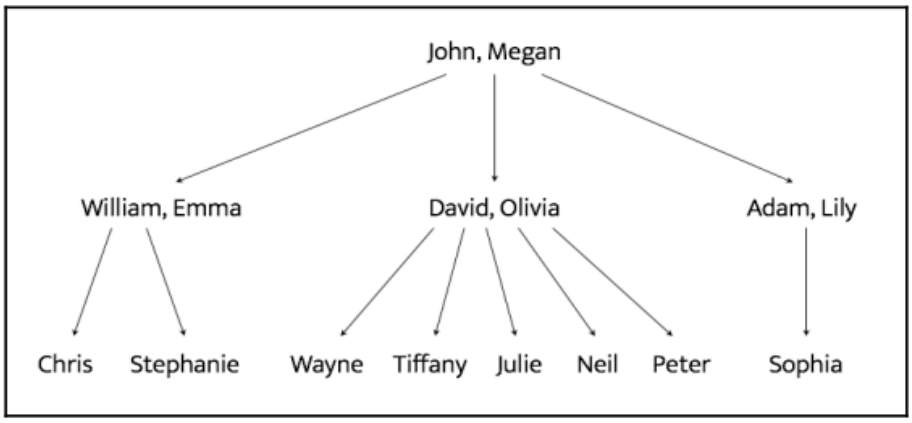
```
List of primes in the list: {3, 11, 13, 17, 19, 23, 29}
List of first 7 prime numbers: (2, 3, 5, 7, 11, 13, 17)
```

You can confirm that the output values are correct.

Activity 3:

Parsing a family tree

Now that we are more familiar with logic programming, let's use it to solve an interesting problem. Consider the following family tree:



John and Megan have three sons – William, David, and Adam. The wives of William, David, and Adam are Emma, Olivia, and Lily respectively. William and Emma have two children – Chris and Stephanie. David and Olivia have five children – Wayne, Tiffany, Julie, Neil, and Peter. Adam and Lily have one child – Sophia. Based on these facts, we can create a program that can tell us the name of Wayne's grandfather or Sophia's uncles are. Even though we have not explicitly specified anything about the grandparent or uncle relationships, logic programming can infer them. These relationships are specified in a file called *relationships.json* provided for you. The file looks like the following:

```
{
"father":
[
{"John": "William"},
{"John": "David"},
{"John": "Adam"},
{"William": "Chris"},
{"William": "Stephanie"},
{"David": "Wayne"},
{"David": "Tiffany"},
{"David": "Julie"},
{"David": "Neil"},
{"David": "Peter"},
{"Adam": "Sophia"} ],
"mother": [
{"Megan": "William"},
{"Megan": "David"},
```

```
{"Megan": "Adam"},
{"Emma": "Stephanie"},
{"Emma": "Chris"},
{"Olivia": "Tiffany"},
{"Olivia": "Julie"},
{"Olivia": "Neil"},
 {"Olivia": "Peter"},
{"Lily": "Sophia"}
]
}
```

It is a simple *json* file that specifies only the father and mother relationships. Note that we haven't specified anything about husband and wife, grandparents, or uncles.

Create a new Python file and import the following packages:

```
import json
from logpy  import Relation, facts, run, conde, var, eq
```

Define a function to check if x is the parent of y. We will use the logic that if x is the parent of y, then x is either the father or the mother. We have already defined "father" and "mother" in our fact base:

```
# Check if 'x' is the parent of 'y'
def parent(x, y):
return conde([father(x, y)], [mother(x, y)])
```

Define a function to check if *x* is the grandparent of *y*. We will use the logic that if *x* is the grandparent of *y*, then the offspring of *x* will be the parent of *y*:

```
# Check if 'x' is the grandparent of 'y'
def grandparent(x, y):
temp = var()
return conde((parent(x, temp), parent(temp, y)))
```

Define a function to check if x is the sibling of y. We will use the logic that if x is the sibling of y, then x and y will have the same parents. Notice that there is a slight modification needed here because when we list out all the siblings of x, x will be listed as well because x satisfies these conditions. So when we print the output, we will have to remove x from the list. We will discuss this in the main function:

```
# Check for sibling relationship between 'a' and 'b'
def sibling(x, y):
      temp = var()
return conde((parent(temp, x), parent(temp, y)))
```

Define a function to check if *x* is y's uncle. We will use the logic that if x is y's uncle, then *x* grandparents will be the same as y's parents. Notice that there is a slight modification needed here because when we list out all the uncles of *x*, x's father will be listed as well because x's father satisfies these conditions. So when we print the output, we will have to remove x's father from the list. We will discuss this in the main function:

```
# Check if x is y's uncle
def uncle(x, y):
      temp = var()
      return conde((father(temp, x), grandparent(temp, y)))
```

Define the main function and initialize the relations *father* and *mother*:

```
if __name__=='__main__':
father = Relation()
mother = Relation()
```

Load the data from the relationships.json file:

```
with open('relationships.json') as f:
d = json.loads(f.read())
```

Read the data and add them to our fact base:

```
for item in d['father']: facts(father, (list(item.keys())[0],
list(item.values())[0]))
for item in d['mother']: facts(mother, (list(item.keys())[0],
list(item.values())[0]))
```

Define the variable x:

```
x = var()
```

We are now ready to ask some questions and see if our solver can come up with the right answers.

Let's ask who John's children are:

```
# John's children
name = 'John'
output = run(0, x, father(name, x))
print("\nList of " + name + "'s children:")
for item in output:
print(item)
```

Who is William's mother?

```
# William's mother
name = 'William'
output = run(0, x, mother(x, name))[0]
print("\n" + name + "'s mother:\n" + output)
```

Who are Adam's parents?

```
# Adam's parents
```

```
name = 'Adam'
output = run(0, x, parent(x, name))
print("\nList of " + name + "'s parents:")
for item in output:
print(item)
```

Who are Wayne's grandparents?

```
# Wayne's grandparents
name = 'Wayne'
output = run(0, x, grandparent(x, name))
print("\nList of " + name + "'s grandparents:")
for item in output:
      print(item)
```

The full code is given in family.py at https://github.com/PacktPublishing/Artificial-Intelligence-with-Python/tree/master/Chapter%2006/code . If you run the code, you will see many things on your Terminal.

3)Graded Lab Tasks

Note: The instructor can design graded lab activities according to the level of difficulty and complexity of the solved lab activities. The lab tasks assigned by the instructor should be evaluated in the same lab

Lab Task 1

Construct your kinship knowledge base using logpy library and perform inference.

Lab 04

Logic Programming in Python: Analyzing Geography and Building Puzzle Solver

Objective:

We are going to learn more about implementing logic programming in python. Specifically, the students will get hands on experience of analyzing geography and building a puzzle solver.

Activity Outcomes:

The activities provide hands - on practice with the following topics
Installing Python packages

- Analyzing geography
- Building a puzzle solver

Instructor Note:

As pre-lab activity, read Chapter 06 from the text book Joshi, Prateek. *Artificial intelligence with python*. Packt Publishing Ltd, 2017.

1)Useful Concepts

This lab is extended version of Lab 03. Key concepts are provided in Lab 03.

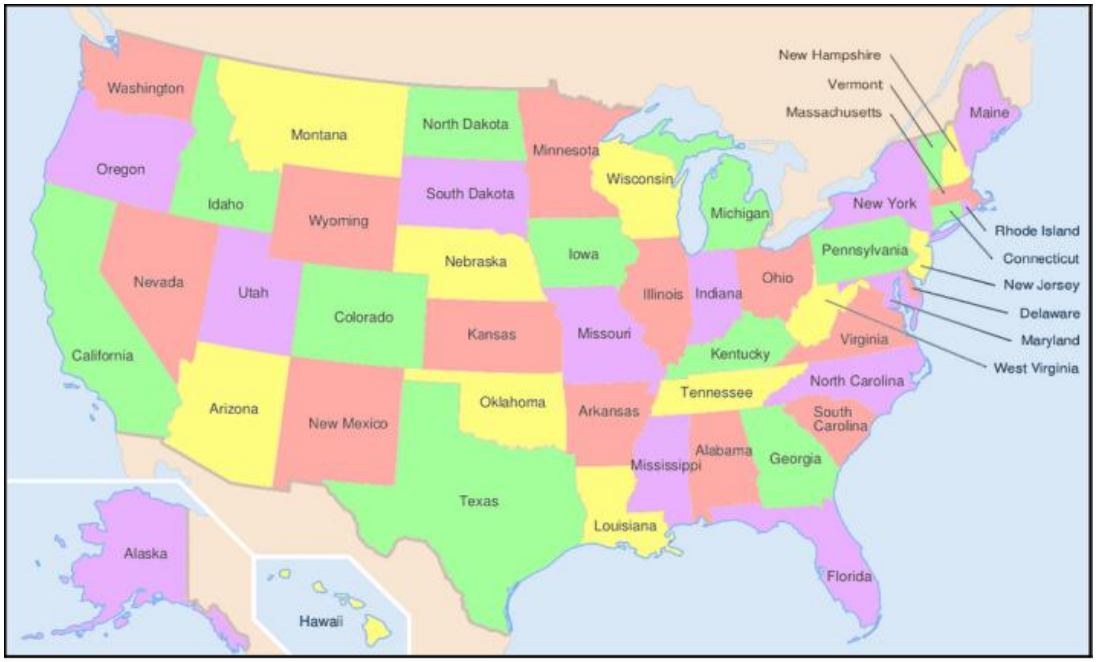
2)Solved Lab Activites

Sr.no	***Allocated time***	***Level of complexity***	***CLO mapping***
1	***25 mins***	***Medium***	***CLO-4***
2	***30 mins***	***Medium***	***CLO-4***

Activity 1

Analyzing geography

Let's use logic programming to build a solver to analyze geography. In this problem, we will specify information about the location of various states in the US and then query our program to answer various questions based on those facts and rules. The following is a map of the US:



You have been provided with two text files named *adjacent_states.txt* and *coastal_states.txt*. These files contain the details about which states are adjacent to each other and which states are coastal. Based on this, we can get interesting information like What states are adjacent to both Oklahoma and Texas? or Which coastal state is adjacent to both New Mexico and Louisiana?

Create a new Python file and import the following:

```
from logpy import run, fact, eq, Relation, var

Initialize the relations:
adjacent = Relation()
coastal = Relation()
```

Define the input files to load the data from:

```
file_coastal = 'coastal_states.txt'
file_adjacent = 'adjacent_states.txt'
```

Load the data:

```
# Read the file containing the coastal states
with open(file_coastal, 'r') as f:
      line = f.read()
      coastal_states = line.split(',')
```

Add the information to the fact base:

Add the info to the fact base

```
for state in coastal_states:
      fact(coastal, state)
```

Read the adjacency data:

Read the file containing the coastal states

```
with open(file_adjacent, 'r') as f:
      adjlist = [line.strip().split(',')
      for line in f if line and line[0].isalpha()]
```

Add the adjacency information to the fact base:

Add the info to the fact base

```
for L in adjlist:
      head, tail = L[0], L[1:]
      for state in tail:
            fact(adjacent, head, state)
```

Initialize the variables x and y:

```
      # Initialize the variables
       x = var()
      y = var()
```

We are now ready to ask some questions. Check if Nevada is adjacent to Louisiana:

```
      # Is Nevada adjacent to Louisiana?
      output = run(0, x, adjacent('Nevada', 'Louisiana'))
      print('\nIs Nevada adjacent to Louisiana?:')
      print('Yes' if len(output) else 'No')
```

Print out all the states that are adjacent to Oregon:

```
      # States adjacent to Oregon
      output = run(0, x, adjacent('Oregon', x))
      print('\nList of states adjacent to Oregon:')
```

```
for item in output:
    print(item)
```

List all the coastal states that are adjacent to Mississippi:
States adjacent to Mississippi that are coastal

```
output = run(0, x, adjacent('Mississippi', x), coastal(x))
print('\nList of coastal states adjacent to Mississippi:')
for item in output:
    print(item)
```

The full code is given in states.py at https://github.com/PacktPublishing/Artificial-Intelligence-with-Python/tree/master/Chapter%2006/code . If you run the code, you will see the following output:

```
Is Nevada adjacent to Louisiana?:
No

List of states adjacent to Oregon:
Washington
California
Nevada
Idaho

List of coastal states adjacent to Mississippi:
Alabama
Louisiana

List of 7 states that border a coastal state:
Georgia
Pennsylvania
Massachusetts
Wisconsin
Maine
Oregon
Ohio

List of states that are adjacent to Arkansas and Kentucky:
Missouri
Tennessee
```

Activity 2

Building a puzzle solver

Another interesting application of logic programming is in solving puzzles. We can specify the conditions of a puzzle and the program will come up with a solution. In this section, we will specify various bits and pieces of information about four people and ask for the missing piece of information.

In the logic program, we specify the puzzle as follows:

- Steve has a blue car
- The person who owns the cat lives in Canada

- Matthew lives in USA
- The person with the black car lives in Australia
- Jack has a cat Alfred lives in Australia
- The person who has a dog lives in France
- Who has a rabbit?

The goal is the find the person who has a rabbit. Here are the full details about the four people:

Name	Pet	Car color	Country
Steve	dog	blue	France
Jack	cat	green	Canada
Matthew	rabbit	yellow	USA
Alfred	parrot	black	Australia

Create a new Python file and import the following packages:

```
from logpy import *
from logpy.core import lall
```

Declare the variable people:

```
# Declare the variable
people = var()
```

Define all the rules using lall. The first rule is that there are four people:

```
# Define the rules
rules = lall(
# There are 4 people
(eq, (var(), var(), var(), var()), people),
```

The person named Steve has a blue car:

```
# Steve's car is blue
(membero, ('Steve', var(), 'blue', var()), people),
```

The person who has a cat lives in Canada:

```
# Person who has a cat lives in Canada
(membero, (var(), 'cat', var(), 'Canada'), people),
```

The person named Matthew lives in USA:

```
# Matthew lives in USA
(membero, ('Matthew', var(), var(), 'USA'), people),
```

The person who has a black car lives in Australia:

```
# The person who has a black car lives in Australia
(membero, (var(), var(), 'black', 'Australia'), people),
```

The person named Jack has a cat:

```
# Jack has a cat
(membero, ('Jack', 'cat', var(), var()), people),
```

The person named Alfred lives in Australia:

```
# Alfred lives in Australia
(membero, ('Alfred', var(), var(), 'Australia'), people),
```

The person who has a dog lives in France:

```
# Person who owns the dog lives in France
(membero, (var(), 'dog', var(), 'France'), people),
```

One of the people in this group has a rabbit. Who is that person?

```
# Who has a rabbit?
(membero, (var(), 'rabbit', var(), var()), people) )
```

Run the solver with the preceding constraints:

```
# Run the solver
solutions = run(0, people, rules)
```

Extract the output from the solution:

```
# Extract the output
output = [house for house in solutions[0] if 'rabbit' in house][0][0]
```

Print the full matrix obtained from the solver:

```
# Print the output
print('\n' + output + ' is the owner of the rabbit')
print('\nHere are all the details:')
attribs = ['Name', 'Pet', 'Color', 'Country']
print('\n' + '\t\t'.join(attribs))
print('=' * 57)
for item in solutions[0]:
      print('')
      print('\t\t'.join([str(x) for x in item]))
```

The full code is given in puzzle.py at https://github.com/PacktPublishing/Artificial-Intelligence-with-Python/tree/master/Chapter%2006/code . If you run the code, you will see the following output:

```
Matthew is the owner of the rabbit

Here are all the details:

Name            Pet             Color           Country
=========================================================

Steve           dog             blue            France

Jack            cat             ~_9             Canada

Matthew         rabbit          ~_11            USA

Alfred          ~_13            black           Australia
```

3)Graded Lab Tasks

Note: The instructor can design graded lab activities according to the level of difficulty and complexity of the solved lab activities. The lab tasks assigned by the instructor should be evaluated in the same lab

Lab task 1

Construct files similar to adjacent_states.txt and coastal_states.txt and perform geography analyzing on Pakistan map. For example,

Lab 05

Simulated Annealing using Python

Objective:

Students will discover the simulated annealing algorithm for function optimization in Python with the help of examples and learning tasks.

Activity Outcomes:

After completing this tutorial, you will know:

- Simulated annealing is a stochastic global search algorithm for function optimization.
- How to implement the simulated annealing algorithm from scratch in Python.
- How to use the simulated annealing algorithm and inspect the results of the algorithm.

Instructor Note:

As pre-lab activity, read Chapter 07 from the text book Joshi, Prateek. *Artificial intelligence with python*. Packt Publishing Ltd, 2017.

1)Useful concepts

Simulated Annealing Simulated Annealing is a type of local search as well as a stochastic search technique. Stochastic search techniques are used extensively in various fields such as robotics, chemistry, manufacturing, medicine, economics, and so on. We can perform things like optimizing the design of a robot, determining the timing strategies for automated control in factories, and planning traffic. Stochastic algorithms are used to solve many real-world problems. Simulated Annealing is a variation of the hill climbing technique. One of the main problems of hill climbing is that it ends up climbing false foothills. This means that it gets stuck in local maxima. So it is better to check out the whole space before we make any climbing decisions. In order to achieve this, the whole space is initially explored to see what it is like. This helps us avoid getting stuck in a plateau or local maxima. In Simulated Annealing, we reformulate the problem and solve it for minimization, as opposed to maximization. So, we are now descending into valleys as opposed to climbing hills. We are pretty much doing the same thing, but in a different way. We use an objective function to guide the search. This objective function serves as our heuristic.

The rate at which we cool the system is called the annealing schedule. The rate of cooling is important because it directly impacts the final result. In the real world case of metals, if the rate of cooling is too fast, it ends up settling for the local maximum. For example, if we take the heated metal and put it in cold water, it ends up quickly settling for the sub-optimal local

maximum. If the rate of cooling is slow and controlled, we give the metal a chance to arrive at the globally optimum state. The chances of taking big steps quickly towards any particular hill are lower in this case. Since the rate of cooling is slow, it will take its time to choose the best state. We do something similar with data in our case.

We first evaluate the current state and see if it is the goal. If it is, then we stop. If not, then we set the best state variable to the current state. We then define our annealing schedule that controls how quickly it descends into a valley. We compute the difference between the current state and the new state. If the new state is not better, then we make it the current state with a certain predefined probability. We do this using a random number generator and making a decision based on a threshold. If it is above the threshold, then we set the best state to this state. Based on this, we update the annealing schedule depending on the number of nodes. We keep doing this until we arrive at the goal.

Basic Understanding

First, we must define our objective function and the bounds on each input variable to the objective function. The objective function is just a Python function we will name objective(). The bounds will be a 2D array with one dimension for each input variable that defines the minimum and maximum for the variable. For example, a one-dimensional objective function and bounds would be defined as follows:

```
# objective function
def objective(x):
      return 0

# define range for input
bounds = asarray([[-5.0, 5.0]])
```

Next, we can generate our initial point as a random point within the bounds of the problem, then evaluate it using the objective function.

```
# generate an initial point
best = bounds[:, 0] + rand(len(bounds)) * (bounds[:, 1] - bounds[:,
0])
# evaluate the initial point
best_eval = objective(best)
```

We need to maintain the “current” solution that is the focus of the search and that may be replaced with better solutions.

```
# current working solution
curr, curr_eval = best, best_eval
```

Now we can loop over a predefined number of iterations of the algorithm defined as “n_iterations“, such as 100 or 1,000.

```
...
# run the algorithm
for i in range(n_iterations):
```

```
	.../* code comes here
```

The first step of the algorithm iteration is to generate a new candidate solution from the current working solution, e.g. take a step.

This requires a predefined "step_size" parameter, which is relative to the bounds of the search space. We will take a random step with a Gaussian distribution where the mean is our current point and the standard deviation is defined by the "step_size". That means that about 99 percent of the steps taken will be within 3 * step_size of the current point.

```
...
# take a step
candidate = solution + randn(len(bounds)) * step_size
```

We don't have to take steps in this way. You may wish to use a uniform distribution between 0 and the step size.

Next, we need to evaluate it.

```
# evaluate candidate point
candidate_eval = objective(candidate)
```

We then need to check if the evaluation of this new point is as good as or better than the current best point, and if it is, replace our current best point with this new point.

This is separate from the current working solution that is the focus of the search.

```
# check for new best solution
if candidate_eval < best_eval:
	# store new best point
	best, best_eval = candidate, candidate_eval
	# report progress
	print('>%d f(%s) = %.5f' % (i, best, best_eval))
```

Next, we need to prepare to replace the current working solution.

The first step is to calculate the difference between the objective function evaluation of the current solution and the current working solution.

```
# difference between candidate and current point evaluation
diff = candidate_eval - curr_eval
```

Next, we need to calculate the current temperature, using the fast annealing schedule, where "temp" is the initial temperature provided as an argument.

```
# calculate temperature for current epoch
t = temp / float(i + 1)
```

We can then calculate the likelihood of accepting a solution with worse performance than our current working solution.

```
# calculate metropolis acceptance criterion
metropolis = exp(-diff / t)
```

Finally, we can accept the new point as the current working solution if it has a better objective function evaluation (the difference is negative) or if the objective function is worse, but we probabilistically decide to accept it.

```
...
# check if we should keep the new point
if diff < 0 or rand() < metropolis:
	# store the new current point
	curr, curr_eval = candidate, candidate_eval
```

We can implement this simulated annealing algorithm as a reusable function that takes the name of the objective function, the bounds of each input variable, the total iterations, step size, and initial temperature as arguments, and returns the best solution found and its evaluation.

```
# simulated annealing algorithm
def simulated_annealing(objective, bounds, n_iterations, step_size,
temp):
	# generate an initial point
	best = bounds[:, 0] + rand(len(bounds)) * (bounds[:, 1] -
bounds[:, 0])
	# evaluate the initial point
	best_eval = objective(best)
	# current working solution
	curr, curr_eval = best, best_eval
	# run the algorithm
	for i in range(n_iterations):
		# take a step
		candidate = curr + randn(len(bounds)) * step_size
		# evaluate candidate point
		candidate_eval = objective(candidate)
		# check for new best solution
		if candidate_eval < best_eval:
			# store new best point
			best, best_eval = candidate, candidate_eval
			# report progress
			print('>%d f(%s) = %.5f' % (i, best, best_eval))
		# difference between candidate and current point
evaluation
		diff = candidate_eval - curr_eval
		# calculate temperature for current epoch
		t = temp / float(i + 1)
		# calculate metropolis acceptance criterion
		metropolis = exp(-diff / t)
		# check if we should keep the new point
		if diff < 0 or rand() < metropolis:
```

```
                # store the new current point
                curr, curr_eval = candidate, candidate_eval
    return [best, best_eval]
```

2)Solved Lab Activites

Sr.no	*Allocated time*	*Level of complexity*	*CLO mapping*
1	*60 mins*	*High*	*CLO-4,5*

Activity 1:

This activity introduces a working example of simulated annealing in python.

Now that we know how to implement the simulated annealing algorithm in Python, let's look at how we might use it to optimize an objective function

First, let's define our objective function. We will use a simple one-dimensional x^2 objective function with the bounds [-5, 5]. The example below defines the function, then creates a line plot of the response surface of the function for a grid of input values, and marks the optima at f(0.0) = 0.0 with a red line

```
# convex unimodal optimization function
from numpy import arange
from matplotlib import pyplot

# objective function
def objective(x):
    return x[0]**2.0

# define range for input
r_min, r_max = -5.0, 5.0
# sample input range uniformly at 0.1 increments
inputs = arange(r_min, r_max, 0.1)
# compute targets
results = [objective([x]) for x in inputs]
# create a line plot of input vs result
pyplot.plot(inputs, results)
# define optimal input value
x_optima = 0.0
# draw a vertical line at the optimal input
pyplot.axvline(x=x_optima, ls='--', color='red')
# show the plot
pyplot.show()
```

Running the example creates a line plot of the objective function and clearly marks the function optima.

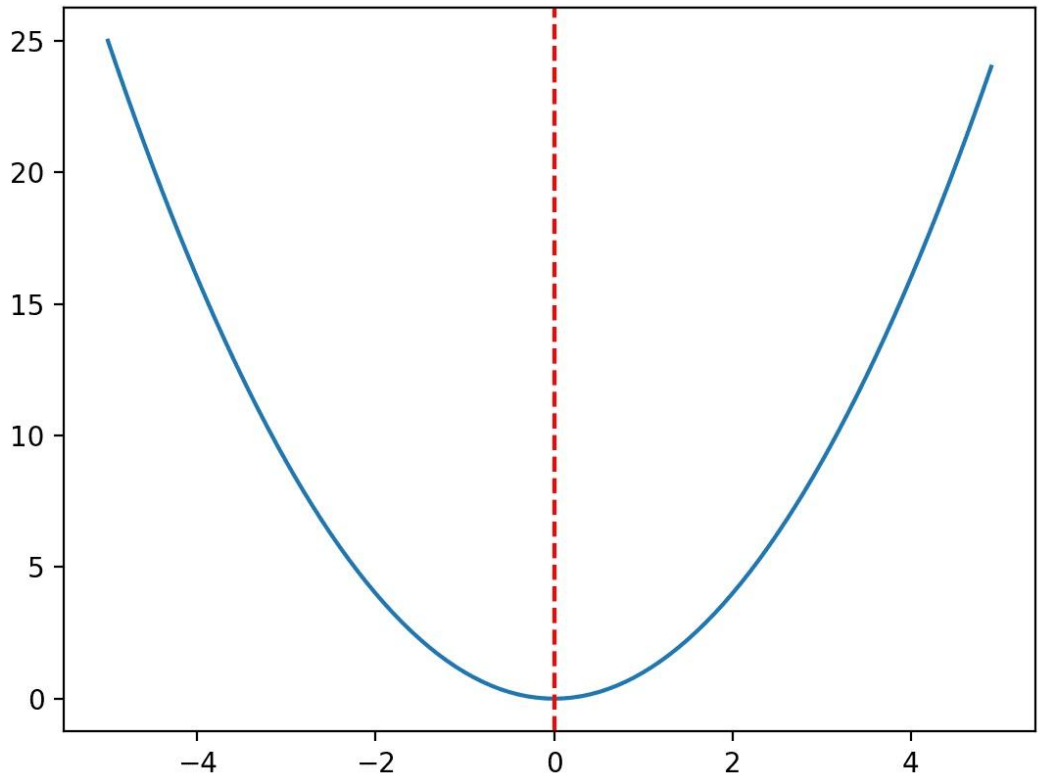


Figure 1 Line Plot of Objective Function With Optima Marked With a Dashed Red Line

Before we apply the optimization algorithm to the problem, let's take a moment to understand the acceptance criterion a little better.

First, the fast annealing schedule is an exponential function of the number of iterations. We can make this clear by creating a plot of the temperature for each algorithm iteration. We will use an initial temperature of 10 and 100 algorithm iterations, both arbitrarily chosen.

The complete example is listed below.

```
# explore temperature vs algorithm iteration for simulated annealing
from matplotlib import pyplot
# total iterations of algorithm
iterations = 100
# initial temperature
initial_temp = 10
# array of iterations from 0 to iterations - 1
iterations = [i for i in range(iterations)]
# temperatures for each iterations
temperatures = [initial_temp/float(i + 1) for i in iterations]
# plot iterations vs temperatures
pyplot.plot(iterations, temperatures)
pyplot.xlabel('Iteration')
pyplot.ylabel('Temperature')
pyplot.show()
```

Running the example calculates the temperature for each algorithm iteration and creates a plot of algorithm iteration (x-axis) vs. temperature (y-axis).

We can see that temperature drops rapidly, exponentially, not linearly, such that after 20 iterations it is below 1 and stays low for the remainder of the search.

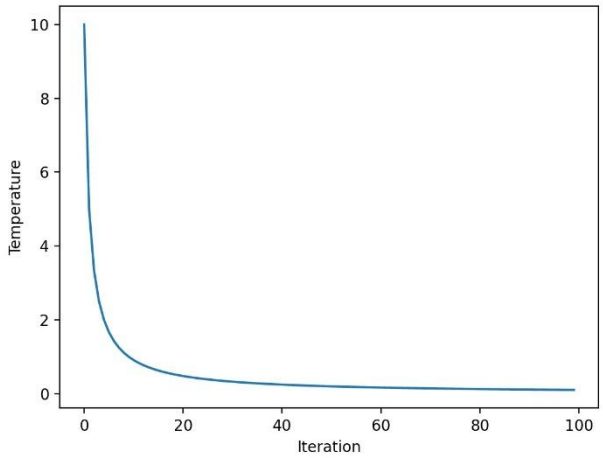


Figure 2: Line Plot of Temperature vs. Algorithm Iteration for Fast Annealing

Next, we can get a better idea of how the metropolis acceptance criterion changes over time with the temperature.

Recall that the criterion is a function of temperature, but is also a function of how different the objective evaluation of the new point is compared to the current working solution. As such, we will plot the criterion for a few different "differences in objective function value" to see the effect it has on acceptance probability.

The complete example is listed below.

```
# explore metropolis acceptance criterion for simulated annealing
from math import exp
from matplotlib import pyplot
# total iterations of algorithm
iterations = 100
# initial temperature
initial_temp = 10
# array of iterations from 0 to iterations - 1
iterations = [i for i in range(iterations)]
# temperatures for each iterations
temperatures = [initial_temp/float(i + 1) for i in iterations]
# metropolis acceptance criterion
differences = [0.01, 0.1, 1.0]
for d in differences:
	metropolis = [exp(-d/t) for t in temperatures]
	# plot iterations vs metropolis
	label = 'diff=%.2f' % d
	pyplot.plot(iterations, metropolis, label=label)
# inalize plot
```

```
pyplot.xlabel('Iteration')
pyplot.ylabel('Metropolis Criterion')
pyplot.legend()
pyplot.show()
```

Running the example calculates the metropolis acceptance criterion for each algorithm iteration using the temperature shown for each iteration (shown in the previous section). The plot has three lines for three differences between the new worse solution and the current working solution.

We can see that the worse the solution is (the larger the difference), the less likely the model is to accept the worse solution regardless of the algorithm iteration, as we might expect. We can also see that in all cases, the likelihood of accepting worse solutions decreases with algorithm iteration.

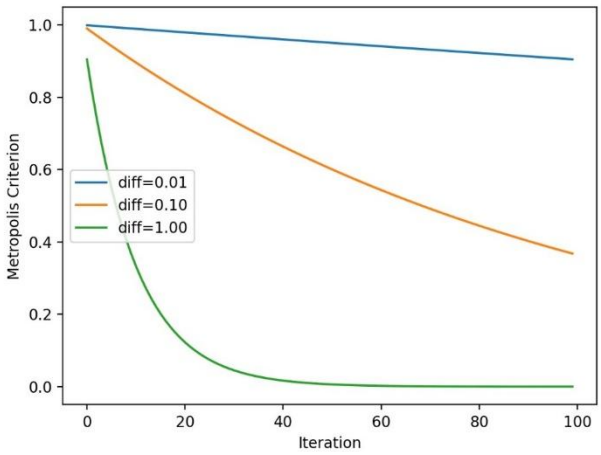


Figure 3: Line Plot of Metropolis Acceptance Criterion vs. Algorithm Iteration for Simulated Annealing

Now that we are more familiar with the behavior of the temperature and metropolis acceptance criterion over time, let's apply simulated annealing to our test problem. First, we will seed the pseudorandom number generator.

This is not required in general, but in this case, I want to ensure we get the same results (same sequence of random numbers) each time we run the algorithm so we can plot the results later.

```
...
# seed the pseudorandom number generator
seed(1)
```

Next, we can define the configuration of the search.

In this case, we will search for 1,000 iterations of the algorithm and use a step size of 0.1. Given that we are using a Gaussian function for generating the step, this means that about 99 percent of all steps taken will be within a distance of (0.1 * 3) of a given point, e.g. three standard deviations.

We will also use an initial temperature of 10.0. The search procedure is more sensitive to the annealing schedule than the initial temperature, as such, initial temperature values are almost arbitrary.

```
...
n_iterations = 1000
# define the maximum step size
step_size = 0.1
# initial temperature
temp = 10
```

Next, we can perform the search and report the results.

```
...
# perform the simulated annealing search
best, score = simulated_annealing(objective, bounds, n_iterations,
step_size, temp)
print('Done!')
print('f(%s) = %f' % (best, score))
```

Trying this all together, the complete example is listed below.

```
# simulated annealing search of a one-dimensional objective function
from numpy import asarray
from numpy import exp
from numpy.random import randn
from numpy.random import rand
from numpy.random import seed

# objective function
def objective(x):
	return x[0]**2.0

# simulated annealing algorithm
def simulated_annealing(objective, bounds, n_iterations, step_size,
temp):
	# generate an initial point
	best = bounds[:, 0] + rand(len(bounds)) * (bounds[:, 1] -
bounds[:, 0])
	# evaluate the initial point
	best_eval = objective(best)
	# current working solution
	curr, curr_eval = best, best_eval
	# run the algorithm
	for i in range(n_iterations):
		# take a step
		candidate = curr + randn(len(bounds)) * step_size
```

```
            # evaluate candidate point
            candidate_eval = objective(candidate)
            # check for new best solution
            if candidate_eval < best_eval:
                  # store new best point
                  best, best_eval = candidate, candidate_eval
                  # report progress
                  print('>%d f(%s) = %.5f' % (i, best, best_eval))
            # difference between candidate and current point evaluation
            diff = candidate_eval - curr_eval
            # calculate temperature for current epoch
            t = temp / float(i + 1)
            # calculate metropolis acceptance criterion
            metropolis = exp(-diff / t)
            # check if we should keep the new point
            if diff < 0 or rand() < metropolis:
                  # store the new current point
                  curr, curr_eval = candidate, candidate_eval
      return [best, best_eval]

# seed the pseudorandom number generator
seed(1)
# define range for input
bounds = asarray([[-5.0, 5.0]])
# define the total iterations
n_iterations = 1000
# define the maximum step size
step_size = 0.1
# initial temperature
temp = 10
# perform the simulated annealing search
best, score = simulated_annealing(objective, bounds, n_iterations,
step_size, temp)
print('Done!')
print('f(%s) = %f' % (best, score))
```

Running the example reports the progress of the search including the iteration number, the input to the function, and the response from the objective function each time an improvement was detected.

At the end of the search, the best solution is found and its evaluation is reported.

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

In this case, we can see about 20 improvements over the 1,000 iterations of the algorithm and a solution that is very close to the optimal input of 0.0 that evaluates to f(0.0) = 0.0.

Next, we can perform the search and report the results.

```
>34 f([-0.78753544]) = 0.62021
>35 f([-0.76914239]) = 0.59158
>37 f([-0.68574854]) = 0.47025
>39 f([-0.64797564]) = 0.41987
>40 f([-0.58914623]) = 0.34709
>41 f([-0.55446029]) = 0.30743
>42 f([-0.41775702]) = 0.17452
>43 f([-0.35038542]) = 0.12277
>50 f([-0.15799045]) = 0.02496
>66 f([-0.11089772]) = 0.01230
>67 f([-0.09238208]) = 0.00853
>72 f([-0.09145261]) = 0.00836
>75 f([-0.05129162]) = 0.00263
>93 f([-0.02854417]) = 0.00081
>144 f([0.00864136]) = 0.00007
>149 f([0.00753953]) = 0.00006
>167 f([-0.00640394]) = 0.00004
>225 f([-0.00044965]) = 0.00000
>503 f([-0.00036261]) = 0.00000
>512 f([0.00013605]) = 0.00000
Done!
f([0.00013605]) = 0.000000
```

It can be interesting to review the progress of the search as a line plot that shows the change in the evaluation of the best solution each time there is an improvement.

We can update the simulated_annealing() to keep track of the objective function evaluations each time there is an improvement and return this list of scores..

```
# simulated annealing algorithm
def simulated_annealing(objective, bounds, n_iterations, step_size,
temp):
	# generate an initial point
	best = bounds[:, 0] + rand(len(bounds)) * (bounds[:, 1] -
bounds[:, 0])
	# evaluate the initial point
	best_eval = objective(best)
	# current working solution
	curr, curr_eval = best, best_eval
	# run the algorithm
	for i in range(n_iterations):
		# take a step
		candidate = curr + randn(len(bounds)) * step_size
		# evaluate candidate point
		candidate_eval = objective(candidate)
		# check for new best solution
		if candidate_eval < best_eval:
			# store new best point
			best, best_eval = candidate, candidate_eval
			# keep track of scores
			scores.append(best_eval)
			# report progress
			print('>%d f(%s) = %.5f' % (i, best, best_eval))
```

```
            # difference between candidate and current point evaluation
            diff = candidate_eval - curr_eval
            # calculate temperature for current epoch
            t = temp / float(i + 1)
            # calculate metropolis acceptance criterion
            metropolis = exp(-diff / t)
            # check if we should keep the new point
            if diff < 0 or rand() < metropolis:
                  # store the new current point
                  curr, curr_eval = candidate, candidate_eval
      return [best, best_eval, scores]
```

We can then create a line plot of these scores to see the relative change in objective function for each improvement found during the search.

```
...
# line plot of best scores
pyplot.plot(scores, '.-')
pyplot.xlabel('Improvement Number')
pyplot.ylabel('Evaluation f(x)')
pyplot.show()
```

Tying this together, the complete example of performing the search and plotting the objective function scores of the improved solutions during the search is listed below..

```
# simulated annealing search of a one-dimensional objective function
from numpy import asarray
from numpy import exp
from numpy.random import randn
from numpy.random import rand
from numpy.random import seed
from matplotlib import pyplot

# objective function
def objective(x):
      return x[0]**2.0

# simulated annealing algorithm
def simulated_annealing(objective, bounds, n_iterations, step_size,
temp):
      # generate an initial point
      best = bounds[:, 0] + rand(len(bounds)) * (bounds[:, 1] -
bounds[:, 0])
      # evaluate the initial point
      best_eval = objective(best)
      # current working solution
      curr, curr_eval = best, best_eval
      scores = list()
      # run the algorithm
      for i in range(n_iterations):
            # take a step
```

```
            candidate = curr + randn(len(bounds)) * step_size
            # evaluate candidate point
            candidate_eval = objective(candidate)
            # check for new best solution
            if candidate_eval < best_eval:
                  # store new best point
                  best, best_eval = candidate, candidate_eval
                  # keep track of scores
                  scores.append(best_eval)
                  # report progress
                  print('>%d f(%s) = %.5f' % (i, best, best_eval))
            # difference between candidate and current point evaluation
            diff = candidate_eval - curr_eval
            # calculate temperature for current epoch
            t = temp / float(i + 1)
            # calculate metropolis acceptance criterion
            metropolis = exp(-diff / t)
            # check if we should keep the new point
            if diff < 0 or rand() < metropolis:
                  # store the new current point
                  curr, curr_eval = candidate, candidate_eval
      return [best, best_eval, scores]

# seed the pseudorandom number generator
seed(1)
# define range for input
bounds = asarray([[-5.0, 5.0]])
# define the total iterations
n_iterations = 1000
# define the maximum step size
step_size = 0.1
# initial temperature
temp = 10
# perform the simulated annealing search
best, score, scores = simulated_annealing(objective, bounds,
n_iterations, step_size, temp)
print('Done!')
print('f(%s) = %f' % (best, score))
# line plot of best scores
pyplot.plot(scores, '.-')
pyplot.xlabel('Improvement Number')
pyplot.ylabel('Evaluation f(x)')
pyplot.show()
```

Running the example performs the search and reports the results as before.

A line plot is created showing the objective function evaluation for each improvement during the hill climbing search. We can see about 20 changes to the objective function evaluation during the search with large changes initially and very small to imperceptible changes towards the end of the search as the algorithm converged on the optima.

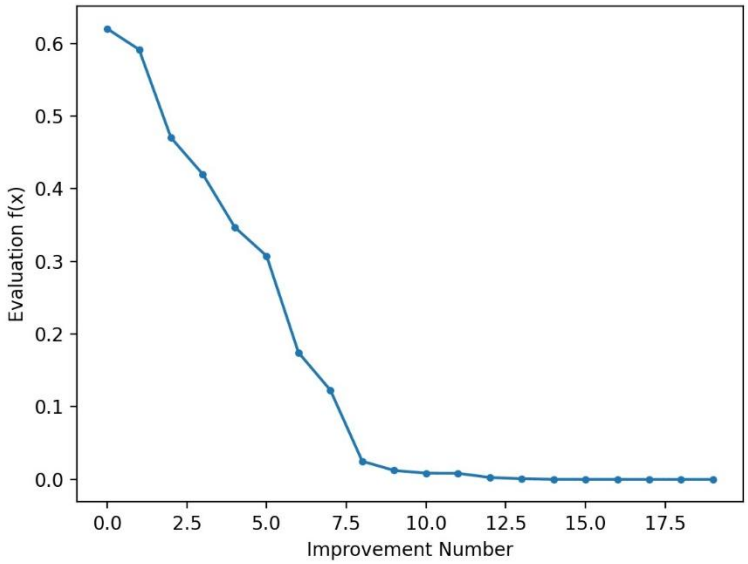


Figure 4: Line Plot of Objective Function Evaluation for Each Improvement During the Simulated Annealing Search

3) Graded Lab Tasks:

Note: The instructor can design graded lab activities according to the level of difficulty and complexity of the solved lab activities. The lab tasks assigned by the instructor should be evaluated in the same lab

Lab Task 1

Use simulated annealing to optimize x^3 objective function with the bounds [-1, 1].

Lab 06

Genetic Algoritm

Objective:

The objective of this lab is to understand the method of solving both constraind and unconstrained optimization problems based on a natural slelection process that mimics biological evolution. The lab will provide hands-on experience of solving realworld problem.

Activity Outcomes:

After completing this hands - on practice the student will know about following topics

- Genetic algorithm is a stochastic optimization algorithm inspired by evolution.
- How to implement the genetic algorithm from scratch in Python.
- How to apply the genetic algorithm for robotic control.

Instructor Note:

As pre-lab activity, read Chapter 08 from the text book Joshi, Prateek. *Artificial intelligence with python*. Packt Publishing Ltd, 2017.

1)Useful concepts:

Understanding evolutionary and genetic algorithms: A genetic algorithm is a type of evolutionary algorithm. So, in order to understand genetic algorithms, we need to discuss evolutionary algorithms. An evolutionary algorithm is a meta heuristic optimization algorithm that applies the principles of evolution to solve problems. The concept of evolution is similar to the one we find in nature. We directly use the problem's functions and variables to arrive at the solution. But in a genetic algorithm, any given problem is encoded in bit patterns that are manipulated by the algorithm.

The underlying idea in all evolutionary algorithms is that we take a population of individuals and apply the natural selection process. We start with a set of randomly selected individuals and then identify the strongest among them. The strength of each individual is determined using a fitness function that's predefined. In a way, we use the survival of the fittest approach. We then take these selected individuals and create the next generation of individuals by recombination and mutation. We will discuss the concepts of recombination and mutation in the next section. For now, let's think of these techniques as mechanisms to create the next generation by treating the selected individuals as parents. Once we execute recombination and mutation, we create a new set of individuals who will compete with the old ones for a place in the next generation. By discarding the weakest individuals and replacing them with offspring, we are increasing the overall fitness level of the population. We continue to iterate until the desired overall fitness is achieved. A genetic algorithm is an evolutionary algorithm where we use a heuristic to find a string of bits that solves a problem. We continuously iterate on a population to arrive at a solution. We do this by generating new populations containing stronger individuals. We apply probabilistic operators such as selection, crossover, and mutation in order to generate the next generation of individuals. The

individuals are basically strings, where every string is the encoded version of a potential solution. A fitness function is used that evaluates the fitness measure of each string telling us how well suited it is to solve this problem. This fitness function is also referred to as an evaluation function. Genetic algorithms apply operators that are inspired from nature, which is why the nomenclature is closely related to the terms found in biology.

Fundamental concepts in genetic algorithms: In order to build a genetic algorithm, we need to understand several key concepts and terminology. These concepts are used extensively throughout the field of genetic algorithms to build solutions to various problems. One of the most important aspects of genetic algorithms is the randomness. In order to iterate, it relies on the random sampling of individuals. This means that the process is non-deterministic. So, if you run the same algorithm multiple times, you might end up with different solutions.

Let's talk about population. A population is a set of individuals that are possible candidate solutions. In a genetic algorithm, we do not maintain a single best solution at any given stage. It maintains a set of potential solutions, one of which is the best. But the other solutions play an important role during the search. Since we have a population of solutions, it is less likely that will get stuck in a local optimum. Getting stuck in the local optimum is a classic problem faced by other optimization techniques. Now that we know about population and the stochastic nature of genetic algorithms, let's talk about the operators. In order to create the next generation of individuals, we need to make sure that they come from the strongest individuals in the current generation. Mutation is one of the ways to do it. A genetic algorithm makes random changes to one or more individuals of the current generation to yield a new candidate solution. This change is called mutation. Now this change might make that individual better or worse than existing individuals. The next concept here is recombination, which is also called crossover. This is directly related to the role of reproduction in the evolution process. A genetic algorithm tries to combine individuals from the current generation to create a new solution. It combines some of the features of each parent individual to create this offspring. This process is called crossover. The goal is to replace the weaker individuals in the current generation with offspring generated from stronger individuals in the population. In order to apply crossover and mutation, we need to have selection criteria. The concept of selection is inspired by the theory of natural selection. During each iteration, the genetic algorithm performs a selection process. The strongest individuals are chosen using this selection process and the weaker individuals are terminated. This is where the survival of the fittest concept comes into play. The selection process is carried out using a fitness function that computes the strength of each individual.

Generating a bit pattern with predefined parameters Now that we know how a genetic algorithm works, let's see how to use it to solve some problems. We will be using a Python package called DEAP. You can find all the details about it at http://deap.readthedocs.io/en/master. Let's go ahead and install it by running the following command on your Terminal:

```
$ pip3 install deap
```

Now that the package is installed, let's quickly test it. Go into the Python shell by typing the following on your Terminal:

```
$ python3
```

Once you are inside, type the following:

```
>>> import deap
```

If you do not see an error message, we are good to go. In this section, we will solve a variant of the One Max problem. The One Max problem is about generating a bit string that contains the maximum number of ones. It is a simple problem, but it's very helpful in getting familiar with the library as well as

understanding how to implement solutions using genetic algorithms. In our case, we will try to generate a bit string that contains a predefined number of ones. You will see that the underlying structure and part of the code is similar to the example used in the DEAP library. Create a new Python file and import the following:

```
import random
from deap import base, creator, tools
```

Let's say we want to generate a bit pattern of length 75, and we want it to contain 45 ones. We need to define an evaluation function that can be used to target this objective:

```
# Evaluation function
def eval_func(individual):
target_sum = 45
return len(individual) - abs(sum(individual) - target_sum),
```

If you look at the formula used in the preceding function, you can see that it reaches its maximum value when the number of ones is equal to 45. The length of each individual is 75. When the number of ones is equal to 45, the return value would be 75. We now need to define a function to create the toolbox. Let's define a creator object for the fitness function and to keep track of the individuals. The Fitness class used here is an abstract class and it needs the weights attribute to be defined. We are building a maximizing fitness using positive weights:

```
# Create the toolbox with the right parameters
def create_toolbox(num_bits):
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)
```

The first line creates a single objective maximizing fitness named FitnessMax. The second line deals with producing the individual. In a given process, the first individual that is created is a list of floats. In order to produce this individual, we must create an Individual class using the creator. The fitness attribute will use FitnessMax defined earlier. A toolbox is an object that is commonly used in DEAP It is used to store various functions along with their arguments. Let's create this object:

```
# Initialize the toolbox
toolbox = base.Toolbox()
```

We will now start registering various functions to this toolbox. Let's start with the random number generator that generates a random integer between 0 and 1. This is basically to generate the bit strings:

```
# Generate attributes
toolbox.register("attr_bool", random.randint, 0, 1)
```

Let's register the individual function. The method initRepeat takes three arguments – a container class for the individual, a function used to fill the container, and the number of times we want the function to repeat itself:

```
# Initialize structures
toolbox.register("individual",  tools.initRepeat,  creator.Individual,
toolbox.attr_bool, num_bits)
```

We need to register the population function. We want the population to be a list of individuals

```
# Define the population to be a list of individuals
toolbox.register("population",         tools.initRepeat,         list,
toolbox.individual)
```

We now need to register the genetic operators. Register the evaluation function that we defined earlier, which will act as our fitness function. We want the individual, which is a bit pattern, to have 45 ones:

```
# Register the evaluation operator
toolbox.register("evaluate", eval_func)
```

Register the crossover operator named mate using the cxTwoPoint method:

```
# Register the crossover operator
toolbox.register("mate", tools.cxTwoPoint)
```

Register the mutation operator named mutate using mutFlipBit. We need to specify the probability of each attribute to be mutated using indpb:

```
# Register a mutation operator
toolbox.register("mutate", tools.mutFlipBit, indpb=0.05)
```

Register the selection operator using selTournament. It specifies which individuals will be selected for breeding:

```
# Operator for selecting individuals for breeding
toolbox.register("select", tools.selTournament, tournsize=3)
return toolbox
```

This is basically the implementation of all the concepts we discussed in the preceding section. A toolbox generator function is very common in DEAP and we will use it throughout this chapter. So it's important to spend some time to understand how we generated this toolbox. Define the main function by starting with the length of the bit pattern:

```
if __name__ == "__main__":
# Define the number of bits
num_bits = 75
```

Create a toolbox using the function we defined earlier:

```
# Create a toolbox using the above parameter
toolbox = create_toolbox(num_bits)
```

We need to seed the random number generator so that we get repeatable results:

```
# Seed the random number generator
random.seed(7)
```

Create an initial population of, say, 500 individuals using the method available in the toolbox object. Feel free to change this number and experiment with it:

```
# Create an initial population of 500 individuals
population = toolbox.population(n=500)
```

Define the probabilities of crossing and mutating. Again, these are parameters that are defined by the user. So you can change these parameters and see how they affect the result:

```
# Define probabilities of crossing and mutating
probab_crossing, probab_mutating = 0.5, 0.2
```

Define the number of generations that we need to iterate until the process is terminated. If you increase the number of generations, you are giving it more freedom to improve the strength of the population:

```
# Define the number of generations
num_generations = 60
```

Evaluate all the individuals in the population using the fitness functions:

```
print('\nStarting the evolution process')
# Evaluate the entire population
fitnesses = list(map(toolbox.evaluate, population))
for ind, fit in zip(population, fitnesses):
ind.fitness.values = fit
```

Start iterating through the generations:

```
print('\nEvaluated', len(population), 'individuals')
# Iterate through generations
for g in range(num_generations):
print("\n===== Generation", g)
```

In each generation, select the next generation individuals using the selection operator that we registered to the toolbox earlier:

```
# Select the next generation individuals
offspring = toolbox.select(population, len(population))
```

Clone the selected individuals:

```
# Clone the selected individuals
offspring = list(map(toolbox.clone, offspring))
```

Apply crossover and mutation on the next generation individuals using the probability values defined earlier. Once it's done, we need to reset the fitness values:

```
# Apply crossover and mutation on the offspring
for child1, child2 in zip(offspring[::2], offspring[1::2]):
      # Cross two individuals
      if random.random() < probab_crossing:
            toolbox.mate(child1, child2)
            # "Forget" the fitness values of the children
            del child1.fitness.values
            del child2.fitness.values
```

Apply mutation to the next generation individuals using the corresponding probability value that we defined earlier. Once it's done, reset the fitness value:

```
# Apply mutation
      for mutant in offspring:
            # Mutate an individual
            if random.random() < probab_mutating:
            toolbox.mutate(mutant)
```

```
            del mutant.fitness.values
```

Evaluate the individuals with invalid fitness values:

```
# Evaluate the individuals with an invalid fitness
      invalid_ind = [ind for ind in offspring if not ind.fitness.valid]
            fitnesses = map(toolbox.evaluate, invalid_ind)
            for ind, fit in zip(invalid_ind, fitnesses):
            ind.fitness.values      =      fit      print('Evaluated',
            len(invalid_ind), 'individuals')
```

Replace the population with the next generation individuals:

```
# The population is entirely replaced by the offspring
population[:] = offspring
```

Print the stats for the current generation to see how it's progressing:

```
            # Gather all the fitnesses in one list and print the stats
            fits = [ind.fitness.values[0] for ind in population]
            length = len(population)
            mean = sum(fits) / length
             sum2 = sum(x*x for x in fits)
            std = abs(sum2 / length - mean**2)**0.5
            print('Min =', min(fits), ', Max =', max(fits))
            print('Average  =',  round(mean,  2),  ',  Standard  deviation
      =', round(std, 2))
            print("\n==== End of evolution")

            Print the final output:
            best_ind = tools.selBest(population, 1)[0]
            print('\nBest individual:\n', best_ind)
            print('\nNumber of ones:', sum(best_ind))
```

The full code is given in the file bit_counter.py. If you run the code, you will see iterations printed to your Terminal. At the start, you will see something like the following:

```
Starting the evolution process

Evaluated 500 individuals

===== Generation 0
Evaluated 297 individuals
Min = 58.0 , Max = 75.0
Average = 70.43 , Standard deviation = 2.91

===== Generation 1
Evaluated 303 individuals
Min = 63.0 , Max = 75.0
Average = 72.44 , Standard deviation = 2.16

===== Generation 2
Evaluated 310 individuals
Min = 65.0 , Max = 75.0
Average = 73.31 , Standard deviation = 1.6

===== Generation 3
Evaluated 273 individuals
Min = 67.0 , Max = 75.0
Average = 73.76 , Standard deviation = 1.41
```

At the end, you will see something like the following that indicates the end of the evolution:

```
===== Generation 57
Evaluated 306 individuals
Min = 68.0 , Max = 75.0
Average = 74.02 , Standard deviation = 1.27

===== Generation 58
Evaluated 276 individuals
Min = 69.0 , Max = 75.0
Average = 74.15 , Standard deviation = 1.18

===== Generation 59
Evaluated 288 individuals
Min = 69.0 , Max = 75.0
Average = 74.12 , Standard deviation = 1.24

==== End of evolution

Best individual:
 [1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1
, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0
, 1, 0, 0, 0, 1]

Number of ones: 45
```

2) Solved Activities:

Sr.no	*Allocated time*	*Level of complexity*	*CLO mapping*
1	*60 mins*	*Medium*	*CLO-4*

Activity 1

Step-by-step Implementation of GA in Numpy Python.

Let us start implementing GA. At first, let us create a list of the 6 inputs and a variable to hold the number of weights as follows:

```
# Inputs of the equation.
equation inputs = [4,-2,3.5,5,-11,-4.7]
# Number of the weights we are looking to optimize.
num_weights = 6
```

The next step is to define the initial population. Based on the number of weights, each chromosome (solution or individual) in the population will definitely have 6 genes, one gene for each weight. But the question is how many solutions per the population? There is no fixed value for that and we can select the value that fits well with our problem. But we could leave it generic so that it can be changed in the code. Next, we create a variable that holds the number of solutions per population, another to hold the size of the population, and finally, a variable that holds the actual initial population:

```
import numpysol_per_pop = 8# Defining the population size.

pop_size = (sol_per_pop,num_weights) # The population will have
sol_per_pop chromosome where each chromosome has num_weights genes.

#Creating the initial population.

new_population = numpy.random.uniform(low=-4.0, high=4.0,
size=pop_size)
```

After importing the numpy library, we are able to create the initial population randomly using the numpy.random.uniform function. According to the selected parameters, it will be of shape (8, 6). That is 8 chromosomes and each one has 6 genes, one for each weight. After running this code, the population is as follows:

```
[[-2.19134006 -2.88907857  2.02365737 -3.97346034  3.45160502
2.05773249][ 2.12480298  2.97122243  3.60375452  3.78571392
0.28776565  3.5170347 ][ 1.81098962  0.35130155  1.03049548 -
0.33163294  3.52586421  2.53845644][-0.63698911 -2.8638447
```

```
2.93392615 -1.40103767 -1.20313655  0.30567304][-1.48998583 -
1.53845766  1.11905299 -3.67541087  1.33225142  2.86073836][
1.14159503  2.88160332  1.74877772 -3.45854293  0.96125878
2.99178241][ 1.96561297  0.51030292  0.52852716 -1.56909315 -
2.35855588  2.29682254][ 3.00912373 -2.745417    3.27131287 -
0.72163167  0.7516408   0.00677938]]
```

Note that it is generated randomly and thus it will definitely change when get run again. After preparing the population, next is to follow the flowchart in figure 1. Based on the fitness function, we are going to select the best individuals within the current population as parents for mating. Next is to apply the GA variants (crossover and mutation) to produce the offspring of the next generation, creating the new population by appending both parents and offspring, and repeating such steps for a number of iterations/generations. The next code applies these steps:

```
import ga

num_generations = 5

num_parents_mating = 4
for generation in range(num_generations):
     # Measuring the fitness of each chromosome in the population.
     fitness = ga.cal_pop_fitness(equation_inputs, new_population)
    # Selecting the best parents in the population for mating.
     parents = ga.select_mating_pool(new_population, fitness,
                                     num_parents_mating)

     # Generating next generation using crossover.
     offspring_crossover = ga.crossover(parents,

offspring_size=(pop_size[0]-parents.shape[0], num_weights))

     # Adding some variations to the offsrping using mutation.
     offspring_mutation = ga.mutation(offspring_crossover)#
Creating the new population based on the parents and offspring.
     new_population[0:parents.shape[0], :] = parents
     new_population[parents.shape[0]:, :] = offspring_mutation
```

The current number of generations is 5. It is selected to be small for presenting results of all generations within the tutorial. There is a module named GA that holds the implementation of the algorithm.

The first step is to find the fitness value of each solution within the population using the *ga.cal_pop_fitness function*. The implementation of such function inside the GA module is as follows:

```
def cal_pop_fitness(equation_inputs, pop):
# Calculating the fitness value of each solution in the current population.
# The fitness function calculates the sum of products between each input and its corresponding weight.
fitness = numpy.sum(pop*equation_inputs, axis=1)
return fitness
```

The fitness function accepts both the equation inputs values (x1 to x6) in addition to the population. The fitness value is calculated as the sum of product (SOP) between each input and its corresponding gene (weight) according to our function. According to the number of solutions per population, there will be a number of SOPs. As we previously set the number of solutions to 8 in the variable named *sol_per_pop*, there will be 8 SOPs as shown below:

```
[-63.41070188 14.40299221 -42.22532674 18.24112489 -45.44363278 -37.00404311 15.99527402 17.0688537 ]
```

Note that the higher the fitness value the better the solution. After calculating the fitness values for all solutions, next is to select the best of them as parents in the mating pool according to the next function *ga.select_mating_pool*. Such function accepts the population, the fitness values, and the number of parents needed. It returns the parents selected. Its implementation inside the GA module is as follows:

```
def select_mating_pool(pop, fitness, num_parents):

# Selecting the best individuals in the current generation as
parents for producing the offspring of the next generation.

parents = numpy.empty((num_parents, pop.shape[1]))

for parent_num in range(num_parents):

max_fitness_idx = numpy.where(fitness == numpy.max(fitness))

max_fitness_idx = max_fitness_idx[0][0]

parents[parent_num, :] = pop[max_fitness_idx, :]

fitness[max_fitness_idx] = -99999999999

return parents
```

Based on the number of parents required as defined in the variable *num_parents_mating*, the function creates an empty array to hold them as in this line:

```
parents = numpy.empty((num_parents, pop.shape[1]))
```

Looping through the current population, the function gets the index of the highest fitness value because it is the best solution to be selected according to this line:

```
max_fitness_idx = numpy.where(fitness == numpy.max(fitness))
```

This index is used to retrieve the solution that corresponds to such fitness value using this line:

```
parents[parent_num, :] = pop[max_fitness_idx, :]
```

To avoid selecting such solution again, its fitness value is set to a very small value that is likely to not be selected again which is **-99999999999**. The **parents** array is returned finally which will be as follows according to our example:

```
[[-0.63698911 -2.8638447   2.93392615 -1.40103767 -1.20313655
0.30567304][ 3.00912373 -2.745417    3.27131287 -0.72163167
0.7516408   0.00677938][ 1.96561297  0.51030292  0.52852716 -
1.56909315 -2.35855588  2.29682254]
[ 2.12480298  2.97122243  3.60375452  3.78571392  0.28776565
3.5170347 ]]
```

Note that these three parents are the best individuals within the current population based on their fitness values which are 18.24112489, 17.0688537, 15.99527402, and 14.40299221, respectively.

Next step is to use such selected parents for mating in order to generate the offspring. The mating starts with the crossover operation according to the *ga.crossover* function. This function accepts the parents and the offspring size. It uses the offspring size to know the number of offspring to produce from such parents. Such a function is implemented as follows inside the GA module:

```
def crossover(parents, offspring_size):
offspring = numpy.empty(offspring_size)
# The point at which crossover takes place between two parents.
Usually, it is at the center.
crossover_point = numpy.uint8(offspring_size[1]/2)

for k in range(offspring_size[0]):
# Index of the first parent to mate.
parent1_idx = k%parents.shape[0]
# Index of the second parent to mate.
parent2_idx = (k+1)%parents.shape[0]
# The new offspring will have its first half of its genes taken
from the first parent.
offspring[k, 0:crossover_point] = parents[parent1_idx,
```

```
0:crossover_point]
# The new offspring will have its second half of its genes taken
from the second parent.
offspring[k, crossover_point:] = parents[parent2_idx,
crossover_point:]
return offspring
```

The function starts by creating an empty array based on the offspring size as in this line:

```
offspring = numpy.empty(offspring_size)
```

Because we are using single point crossover, we need to specify the point at which crossover takes place.

The point is selected to divide the solution into two equal halves according to this line:

```
crossover_point = numpy.uint8(offspring_size[1]/2)
```

Then we need to select the two parents to crossover. The indices of these parents are selected according to these two lines:

```
parent1_idx = k%parents.shape[0]
parent2_idx = (k+1)%parents.shape[0]
```

The parents are selected in a way similar to a ring. The first with indices 0 and 1 are selected at first to produce two offspring. If there still remaining offspring to produce, then we select the parent 1 with parent 2 to produce another two offspring. If we are in need of more offspring, then we select the next two parents with indices 2 and 3. By index 3, we reached the last parent. If we need to produce more offspring, then we select parent with index 3 and go back to the parent with index 0, and so on.

The solutions after applying the crossover operation to the parents are stored into the *offspring* variable and they are as follows:

```
[[-0.63698911 -2.8638447   2.93392615 -0.72163167  0.7516408
0.00677938][ 3.00912373 -2.745417    3.27131287 -1.56909315 -
```

```
2.35855588  2.29682254][ 1.96561297  0.51030292  0.52852716
3.78571392  0.28776565  3.5170347 ][ 2.12480298  2.97122243
3.60375452 -1.40103767 -1.20313655  0.30567304]]
```

Next is to apply the second GA variant, mutation, to the results of the crossover stored in the *offspring* variable using the *ga.mutation* function inside the GA module. Such function accepts the crossover offspring and returns them after applying uniform mutation. That function is implemented as follows:

```
def mutation(offspring_crossover):

# Mutation changes a single gene in each offspring randomly.

for idx in range(offspring_crossover.shape[0]):

# The random value to be added to the gene.

random_value = numpy.random.uniform(-1.0, 1.0, 1)

offspring_crossover[idx, 4] = offspring_crossover[idx, 4] +
random_value

return offspring_crossover
```

It loops through each offspring and adds a uniformly generated random number in the range from -1 to 1 according to this line:

```
random_value = numpy.random.uniform(-1.0, 1.0, 1)
```

Such random number is then added to the gene with index 4 of the offspring according to this line:

```
offspring_crossover[idx, 4] = offspring_crossover[idx, 4] +
random_value
```

Note that the index could be changed to any other index. The offspring after applying mutation are as follows:

```
[[-0.63698911 -2.8638447   2.93392615 -0.72163167  1.66083721
0.00677938][ 3.00912373 -2.745417    3.27131287 -1.56909315 -
1.94513681  2.29682254][ 1.96561297  0.51030292  0.52852716
3.78571392  0.45337472  3.5170347 ][ 2.12480298  2.97122243
3.60375452 -1.40103767 -1.5781162   0.30567304]]
```

Such results are added to the variable *offspring_crossover* and got returned by the function. At this point, we successfully produced 4 offspring from the 4 selected parents and we are ready to create the new population of the next generation. Note that GA is a random-based optimization technique. It tries to enhance the current solutions by applying some random changes to them. Because such changes are random, we are not sure that they will produce better solutions. For such reason, it is preferred to keep the previous best solutions (parents) in the new population. In the worst case when all the new offspring are worse than such parents, we will continue using such parents. As a result, we guarantee that the new generation will at least preserve the previous good results and will not go worse. The new population will have its first 4 solutions from the previous parents. The last 4 solutions come from the offspring created after applying crossover and mutation:

```
new_population[0:parents.shape[0], :] = parents
new_population[parents.shape[0]:, :] = offspring_mutation
```

By calculating the fitness of all solutions (parents and offspring) of the first generation, their fitness is as follows:

```
[  18.24112489  17.0688537  15.99527402  14.40299221  -8.46075629
31.73289712 6.10307563 24.08733441]
```

The highest fitness previously was **18.24112489** but now it is **31.7328971158**. That means that the random changes moved towards a better solution. This is GREAT. But such results could be enhanced by going through more generations. Below are the results of each step for another 4 generations:

```
Generation :  1Fitness values:[ 18.24112489  17.0688537
15.99527402  14.40299221  -8.46075629  31.73289712   6.10307563
24.08733441]Selected parents:[[ 3.00912373 -2.745417    3.27131287
-1.56909315 -1.94513681  2.29682254][ 2.12480298  2.97122243
3.60375452 -1.40103767 -1.5781162   0.30567304][-0.63698911 -
2.8638447   2.93392615 -1.40103767 -1.20313655  0.30567304][
3.00912373 -2.745417    3.27131287 -0.72163167  0.7516408
0.00677938]]Crossover result:[[ 3.00912373 -2.745417    3.27131287
-1.40103767 -1.5781162   0.30567304][ 2.12480298  2.97122243
3.60375452 -1.40103767 -1.20313655  0.30567304][-0.63698911 -
2.8638447   2.93392615 -0.72163167  0.7516408   0.00677938][
3.00912373 -2.745417    3.27131287 -1.56909315 -1.94513681
2.29682254]]Mutation result:[[ 3.00912373 -2.745417    3.27131287 -
1.40103767 -1.2392086   0.30567304][ 2.12480298  2.97122243
3.60375452 -1.40103767 -0.38610586  0.30567304][-0.63698911 -
2.8638447   2.93392615 -0.72163167  1.33639943  0.00677938][
3.00912373 -2.745417    3.27131287 -1.56909315 -1.13941727
2.29682254]]Best result after generation 1 :
34.1663669207Generation :  2Fitness values:[ 31.73289712
24.08733441  18.24112489  17.0688537   34.16636692  10.97522073  -
4.89194068  22.86998223]Selected Parents:[[ 3.00912373 -2.745417
3.27131287 -1.40103767 -1.2392086   0.30567304][ 3.00912373 -
2.745417    3.27131287 -1.56909315 -1.94513681  2.29682254][
2.12480298  2.97122243  3.60375452 -1.40103767 -1.5781162
0.30567304][ 3.00912373 -2.745417    3.27131287 -1.56909315 -
1.13941727  2.29682254]]Crossover result:[[ 3.00912373 -2.745417
3.27131287 -1.56909315 -1.94513681  2.29682254][ 3.00912373 -
2.745417    3.27131287 -1.40103767 -1.5781162   0.30567304][
2.12480298  2.97122243  3.60375452 -1.56909315 -1.13941727
2.29682254][ 3.00912373 -2.745417    3.27131287 -1.40103767 -
1.2392086   0.30567304]]Mutation result:[[ 3.00912373 -2.745417
3.27131287 -1.56909315 -2.20515009  2.29682254][ 3.00912373 -
2.745417    3.27131287 -1.40103767 -0.73543721  0.30567304][
2.12480298  2.97122243  3.60375452 -1.56909315 -0.50581509
2.29682254][ 3.00912373 -2.745417    3.27131287 -1.40103767 -
```

```
1.20089639  0.30567304]]Best result after generation 2:
34.5930432629Generation :  3Fitness values:[ 34.16636692
31.73289712  24.08733441  22.86998223  34.59304326  28.6248816
2.09334217  33.7449326 ]Selected parents:[[ 3.00912373 -2.745417
3.27131287 -1.56909315 -2.20515009  2.29682254][ 3.00912373 -
2.745417    3.27131287 -1.40103767 -1.2392086   0.30567304][
3.00912373 -2.745417    3.27131287 -1.40103767 -1.20089639
0.30567304][ 3.00912373 -2.745417    3.27131287 -1.56909315 -
1.94513681  2.29682254]]Crossover result:[[ 3.00912373 -2.745417
3.27131287 -1.40103767 -1.2392086   0.30567304][ 3.00912373 -
2.745417    3.27131287 -1.40103767 -1.20089639  0.30567304][
3.00912373 -2.745417    3.27131287 -1.56909315 -1.94513681
2.29682254][ 3.00912373 -2.745417    3.27131287 -1.56909315 -
2.20515009  2.29682254]]Mutation result:[[ 3.00912373 -2.745417
3.27131287 -1.40103767 -2.20744102  0.30567304][ 3.00912373 -
2.745417    3.27131287 -1.40103767 -1.16589294  0.30567304][
3.00912373 -2.745417    3.27131287 -1.56909315 -2.37553107
2.29682254][ 3.00912373 -2.745417    3.27131287 -1.56909315 -
2.44124005  2.29682254]]Best result after generation 3:
44.8169235189Generation :  4Fitness values[ 34.59304326
34.16636692  33.7449326   31.73289712  44.8169235233.35989464
36.46723397  37.19003273]Selected parents:[[ 3.00912373 -2.745417
3.27131287 -1.40103767 -2.20744102  0.30567304][ 3.00912373 -
2.745417    3.27131287 -1.56909315 -2.44124005  2.29682254][
3.00912373 -2.745417    3.27131287 -1.56909315 -2.37553107
2.29682254][ 3.00912373 -2.745417    3.27131287 -1.56909315 -
2.20515009  2.29682254]]Crossover result:[[ 3.00912373 -2.745417
3.27131287 -1.56909315 -2.37553107  2.29682254][ 3.00912373 -
2.745417    3.27131287 -1.56909315 -2.20515009  2.29682254][
3.00912373 -2.745417    3.27131287 -1.40103767 -2.20744102
0.30567304]]Mutation result:[[ 3.00912373 -2.745417    3.27131287 -
1.56909315 -2.13382082  2.29682254][ 3.00912373 -2.745417
3.27131287 -1.56909315 -2.98105233  2.29682254][ 3.00912373 -
2.745417    3.27131287 -1.56909315 -2.27638584  2.29682254][
3.00912373 -2.745417    3.27131287 -1.40103767 -1.70558545
0.30567304]]Best result after generation 4:  44.8169235189
```

After the above 5 generations, the best result now has a fitness value equal to **44.8169235189** compared to the best result after the first generation which is **18.24112489**. The best solution has the following weights:

```
[3.00912373 -2.745417    3.27131287 -1.40103767 -2.20744102
0.30567304]
```

Complete Python Implementation

```
import
numpy
      import ga


      """
      The y=target is to maximize this equation ASAP:
          y = w1x1+w2x2+w3x3+w4x4+w5x5+6wx6
          where (x1,x2,x3,x4,x5,x6)=(4,-2,3.5,5,-11,-4.7)
          What are the best values for the 6 weights w1 to w6?
          We are going to use the genetic algorithm for the best
      possible values after a number of generations.
      """


      # Inputs of the equation.
      equation_inputs = [4,-2,3.5,5,-11,-4.7]


      # Number of the weights we are looking to optimize.
      num_weights = 6


      """
      Genetic algorithm parameters:
          Mating pool size
          Population size
      """
      sol_per_pop = 8
      num_parents_mating = 4


      # Defining the population size.
      pop_size = (sol_per_pop,num_weights) # The population will have
      sol_per_pop chromosome where each chromosome has num_weights
      genes.
      #Creating the initial population.
      new_population = numpy.random.uniform(low=-4.0, high=4.0,
      size=pop_size)
      print(new_population)


      num_generations = 5
```

```
for generation in range(num_generations):
    print("Generation : ", generation)
    # Measing the fitness of each chromosome in the population.
    fitness = ga.cal_pop_fitness(equation_inputs,
new_population)


    # Selecting the best parents in the population for mating.
    parents = ga.select_mating_pool(new_population, fitness,
                                    num_parents_mating)


    # Generating next generation using crossover.
    offspring_crossover = ga.crossover(parents,

offspring_size=(pop_size[0]-parents.shape[0], num_weights))


    # Adding some variations to the offsrping using mutation.
    offspring_mutation = ga.mutation(offspring_crossover)


    # Creating the new population based on the parents and
offspring.
    new_population[0:parents.shape[0], :] = parents
    new_population[parents.shape[0]:, :] = offspring_mutation


    # The best result in the current iteration.
    print("Best result : ",
numpy.max(numpy.sum(new_population*equation_inputs, axis=1)))


# Getting the best solution after iterating finishing all
generations.
#At first, the fitness is calculated for each solution in the
final generation.
fitness = ga.cal_pop_fitness(equation_inputs, new_population)
# Then return the index of that solution corresponding to the
best fitness.
best_match_idx = numpy.where(fitness == numpy.max(fitness))
```

```
print("Best solution : ", new_population[best_match_idx, :])
print("Best solution fitness : ", fitness[best_match_idx])
```

The GA module is as follows:

```
import
numpy


    # This project is extended and a library called PyGAD is
    released to build the genetic algorithm.
    # PyGAD documentation: https://pygad.readthedocs.io
    # Install PyGAD: pip install pygad
    # PyGAD source code at GitHub:
    https://github.com/ahmedfgad/GeneticAlgorithmPython


    def cal_pop_fitness(equation_inputs, pop):
        # Calculating the fitness value of each solution in the
    current population.
        # The fitness function caulcuates the sum of products
    between each input and its corresponding weight.
        fitness = numpy.sum(pop*equation_inputs, axis=1)
        return fitness


    def select_mating_pool(pop, fitness, num_parents):
        # Selecting the best individuals in the current generation
    as parents for producing the offspring of the next generation.
        parents = numpy.empty((num_parents, pop.shape[1]))
        for parent_num in range(num_parents):
            max_fitness_idx = numpy.where(fitness ==
    numpy.max(fitness))
            max_fitness_idx = max_fitness_idx[0][0]
            parents[parent_num, :] = pop[max_fitness_idx, :]
            fitness[max_fitness_idx] = -99999999999
        return parents


    def crossover(parents, offspring_size):
        offspring = numpy.empty(offspring_size)
        # The point at which crossover takes place between two
    parents. Usually it is at the center.
```

```
    crossover_point = numpy.uint8(offspring_size[1]/2)


    for k in range(offspring_size[0]):
        # Index of the first parent to mate.
        parent1_idx = k%parents.shape[0]
        # Index of the second parent to mate.
        parent2_idx = (k+1)%parents.shape[0]
        # The new offspring will have its first half of its
genes taken from the first parent.
        offspring[k, 0:crossover_point] = parents[parent1_idx,
0:crossover_point]
        # The new offspring will have its second half of its
genes taken from the second parent.
        offspring[k, crossover_point:] = parents[parent2_idx,
crossover_point:]
    return offspring


def mutation(offspring_crossover):
    # Mutation changes a single gene in each offspring randomly.
    for idx in range(offspring_crossover.shape[0]):
        # The random value to be added to the gene.
        random_value = numpy.random.uniform(-1.0, 1.0, 1)
        offspring_crossover[idx, 4] = offspring_crossover[idx,
4] + random_value
    return offspring_crossover
```

3)Graded Lab Task

Note: The instructor can design graded lab activities according to the level of difficulty and complexity of the solved lab activities. The lab tasks assigned by the instructor should be evaluated in the same lab

Lab Task 1

Apply genetics algorithm to solve Traveling salesman problem (TSP)

Lab 07

Designing Robot Controller using Genetic Algoritm

Objective:

The objective of this lab is to introduce how to desing robot controller using genetics algorithm.The lab will provide hands-on experience of solving this problem in python.

Activity Outcomes:

After completing this hands - on practice the student will know about following topics

- How to apply the genetic algorithm for robotic control.

Instructor Note:

As pre-lab activity, read Chapter 08 from the text book Joshi, Prateek. *Artificial intelligence with python*. Packt Publishing Ltd, 2017.

1)Useful concepts:

In this lab, we will solve a real-world problem is to design a robotic controller. There are many techniques that can be used to solve this problem. Some of them include genetic algorithm (GA), particle swarm optimization and neural network (NN).

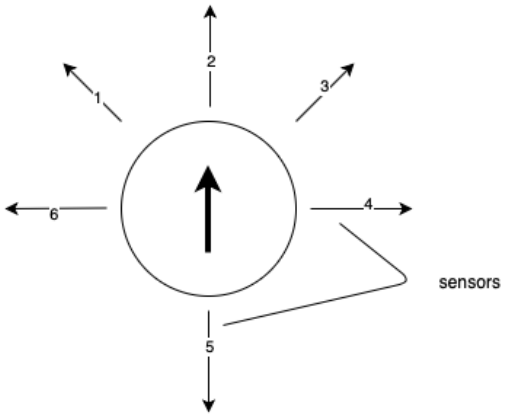
What we need to do is to apply an algorithm to the robotic as a method of designing robotic controllers that enable the robot to perform complex tasks and behaviors.

One of the capabilities of robots is to move from one point to another which called is autonomous navigation. Imagine that we have built a robot that can move goods around a warehouse. In this article, we will implement this capability using the Python language. How the robot can see its local environment? Yes, we would install sensors, which allow the robot can look around and we have given it wheels so it can navigate based on input from its sensors. The biggest problem is how we can link the sensor data to motor actions so that the robot can navigate the warehouse.

In general, we frequently use neural networks to successfully map robotic sensors to outputs by using reinforcement learning algorithms to the learning process. But we will use another way today, it is to use genetic algorithms. Typically, a genetic algorithm will evaluate a large population of individuals to find the best individuals for the next generation by using a fitness function that compute the performance of and individual based on certain predefined rules.

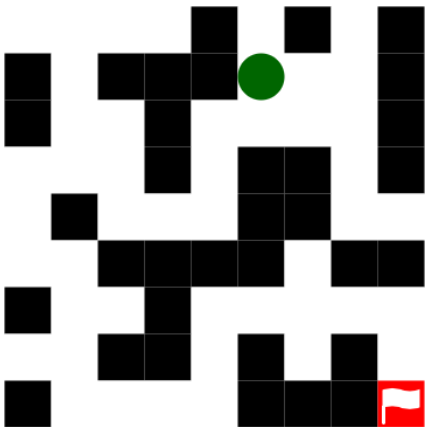
However, we will be facing a new challenge, physically evaluating each robot controller is not feasible for a large population due to the difficulty in physically testing each robotic controller and the time it would take to do so. For this reason, we will use our knowledge of genetic algorithms to design and implement a robotic controller and apply it to a virtual robot in a virtual environment.

The target: Our robot can take four actions: move one step forward, turn left, turn righ and do nothing.
The robot also has six sensors in the figure below.



- three on the front
- one on the left
- one on the right
- one on the back

The maze is comprised of walls that the robot can't cross and will have an outlined route. We are going to design a robotic controller that can use the robot sensors to navigate a robot successfully through a maze.



Mapping the sensor values to actions: As mentioned in the previous section, the robot will have four actions that can be represented in binary as follows:

- "00" — do nothing;
- "01" — move forward;
- "10" — turn left;
- "11" — turn right.

We also have six different sensors. To simplify the representation we limit the measurements to binary encoding, that is, values less than a threshold indicate obstacle detection and larger than threshold indicate a clear path. 6 sensors are giving us 2^6 = 64 possible combinations of sensor inputs. Since an action requires 2 bits, our controller requires 64*2 = 128 bits of storage to represent any possible input. Given that 128 bits are our requirement for representing instructions different combinations. But how should we organize the chromosome so we can encode and decode it?

We have a human readable list of inputs and outputs as follows:

- Sensor #1 (front): on
- Sensor #2 (front-left): off
- Sensor #3 (front-right): on
- Sensor #4 (left): off
- Sensor #5 (right): off
- Sensor #6 (back): off

We also have an instruction that is "turn left" with a value of 10 in binary. Our next is to take the six sensor values and encode those further.

000101 => 10

If we now convert the sensor values' bit string to decimal, we get the following:

5 => 10

So, we can use the sensor's decimal value as the position in the chromosome that represents a combination of sensor inputs as follows.

xx xx xx xx xx 10 xx xx xx xx (… 54 more pairs…)

As can be seen in the following figure, a combination of sensor values produces a binary output that describes how a typical chromosome can map the robot's sensor values to actions.

Sensors						Action	Chromosome
B	R	L	FR	FL	F		
0	0	0	0	0	0	11	**11**001001101001...11
0	0	0	0	0	1	00	11**00**1001101001...11
0	0	0	0	1	0	10	1100**10**01101001...11
0	0	0	0	1	1	01	110010**01**101001...11
0	0	0	1	0	0	10	11001001**10**1001...11
0	0	0	1	0	1	10	1100100110**10**01...11
0	0	0	1	1	0	01	110010011010**01**...11
1	1	1	1	1	1	11	11001001101001...**11**

This encoding scheme may seem obtuse at first and the chromosome is not human-readable but it has a couple of helpful properties.

- First, the chromosome can be manipulated as an array of bits which makes crossover, mutation, and other manipulations much easier.
- Secondly, every 128-bit value is a valid solution.

Implementation

Firstly, we need to create and initialize a maze to run the robot in (world.py file). The maze object we've created uses integers to represent different terrain types:

- 1 defines a wall;
- 2 is the starting position;
- 3 traces the best route through the maze;
- 4 is the goal position;

- and 0 is an empty position that the robot can travel over but isn't on the route to the goal.

We write a constructor to create a new maze from a double int array and implement public methods to get the start position, score a route through the maze.

2)Solved Lab Activities

Sr.no	*Allocated time*	*Level of complexity*	*CLO mapping*
1	*5 mins*	*Low*	*CLO-6*
2	*5 mins*	*Low*	*CLO-6*
3	*10 mins*	*Low*	*CLO-6*
4	*10 mins*	*Low*	*CLO-6*
5	*10 mins*	*Low*	*CLO-6*
6	*10 mins*	*Low*	*CLO-6*
7	*10 mins*	*Low*	*CLO-6*

Activity 1:

Defining World

World.py

```
class
World(object):

                    def __init__(self, world):
                        self.startPosition = [-1, -1]
                        self.world = world


                        # Get start position of maze
                        def getStartPosition(self):
                            # Check we already found start
                position
                            if self.startPosition[0] != -1 &
```

```
self.startPosition[1] != -1:
            return self.startPosition


        # Default return value
        startPosition = [0, 0]


        # Loop over rows
        for rowIndex in
range(len(self.world)):
            # Loop over columns
            for colIndex in
range(len(self.world[rowIndex])):
                # 2 is the type for start
position
                if
self.world[rowIndex][colIndex] == 2:
                    self.startPosition =
[colIndex, rowIndex]
                    return [colIndex,
rowIndex]


    # Gets value for position of maze
    def getPositionValue(self, x, y):
        if (x < 0 or y < 0 or x >=
len(self.world) or y >=
len(self.world[0])):
            return 1
        return self.world[y][x]


    def printWorld(self):
        colNum = 0
        rowNum = 0
        for row in self.world:
            colNum = 0
            for char in row:
```

```
                print(char, '\t', end="")
                colNum += 1
            print('\n')
            rowNum += 1


    # Check if position is wall
    def isWall(self, x, y):
        return self.getPositionValue(x, y)
== 1


    # Gets maximum index of x position
    def getMaxX(self):
        return len(self.world[0])-1


    # Gets maximum index of y position
    def getMaxY(self):
        return len(self.world)-1


    # Scores a maze route
    def scoreRoute(self, route):
        score = 0
        visited = [[False] *
(self.getMaxY()+1)
                   for i in
range(self.getMaxX()+1)]
        # Loop over route and score each
move
        for routeStep in route:
            step = routeStep
            if
(self.world[step[1]][step[0]] == 3 and
visited[step[1]][step[0]] == False):
                # Increase score for
correct move
                score += 1
```

```
                    # Remove reward
                    visited[step[1]][step[0]] =
True
            return score
```

Activity 2:

Defining Chromosome

Individual: An individual is represented by a single chromosome composed of multiple genes.

Individual.py

```
import random
class Individual:


    # Initializes individual with specific chromosome
    # def __init__(self, chromosome):
    #     self.fitness = -1
    #     self.chromosome = chromosome


    ''' Initializes random individual '''
    def __init__(self, chromosomeLength):
        self.fitness = -1
        self.chromosome = [None] * chromosomeLength
        for gene in range(chromosomeLength):
            if (0.5 < random.random()):
                self.setGene(gene, 1)
            else:
                self.setGene(gene, 0)


    ''' Gets individual's chromosome '''
    def getChromosome(self):
        return self.chromosome


    ''' Gets individual's chromosome length '''
    def getChromosomeLength(self):
```

```
        return len(self.chromosome)


    ''' Set gene at offset '''
    def setGene(self, offset, gene):
        self.chromosome[offset] = gene


    ''' Get gene at offset '''
    def getGene(self, offset):
        return self.chromosome[offset]


    ''' Store individual's fitness '''
    def setFitness(self, fitness):
        self.fitness = fitness


    ''' Gets individual's fitness '''
    def getFitness(self):
        return self.fitness


    ''' Display the chromosome as a string '''
    def toString(self):
        output = ''
        for gene in range(len(self.chromosome)):
            output += str(self.chromosome[gene])
        return output
```

Activity 3:

Defining Robot

Robot

Next, we need to create a Robot that can follow instructions and generate a route by executing those instructions.

Robot.py

```
'''
      for facing
      0 is north,
      1 is east,
      2 is south,
      and 3 is west
      '''
      from world import World
      class Robot:


          ''' Initalize a robot with controller '''
          def __init__(self, sensorActions, world, maxMoves):
              self.sensorActions =
      self.calcSensorActions(sensorActions)
              self.world = world
              startPos = self.world.getStartPosition()
              self.xPosition = startPos[0]
              self.yPosition = startPos[1]
              self.sensorVal = -1
              self.heading = 1
              self.maxMoves = maxMoves
              self.moves = 0
              self.route = []
              self.route.append(startPos)


          ''' Runs the robot's actions based on sensor inputs '''
          def run(self):
              while (True):
                  self.moves += 1


                  # Break if the robot stops moving
                  if self.getNextAction() == 0:
                      return

                  # Break if we reach a maximum number of moves
```

```
            if self.moves > self.maxMoves:
                return


            # Run action
            self.makeNextAction()

    ''' Runs the next action '''
    def makeNextAction(self):
        # If move forward
        if self.getNextAction() == 1:
            currentX = self.xPosition
            currentY = self.yPosition


            # Move depending on current direction
            if 0 == self.heading:
                self.yPosition += -1
                if self.yPosition < 0:
                    self.yPosition = 0

            if 1 == self.heading:
                self.xPosition += 1
                if self.xPosition > self.world.getMaxX():
                    self.xPosition = self.world.getMaxX()


            if 2 == self.heading:
                self.yPosition += 1
                if self.yPosition > self.world.getMaxY():
                    self.yPosition = self.world.getMaxY()

            if 3 == self.heading:
                self.xPosition += -1
                if self.xPosition < 0:
                    self.xPosition = 0


            # We can't move here
```

```
            if (self.world.isWall(self.xPosition,
self.yPosition) == True):
                self.xPosition = currentX
                self.yPosition = currentY


            else:
                if (currentX != self.xPosition or currentY !=
self.yPosition):
                    self.route.append(self.getPosition())


        elif self.getNextAction() == 2:
            if 0 == self.heading:
                self.heading = 1
            elif 1 == self.heading:
                self.heading = 2
            elif 2 == self.heading:
                self.heading = 3
            elif 3 == self.heading:
                self.heading = 0

        elif self.getNextAction() == 3:
            if 0 == self.heading:
                self.heading = 3
            elif 1 == self.heading:
                self.heading = 0
            elif 2 == self.heading:
                self.heading = 1
            elif 3 == self.heading:
                self.heading = 2


        # Reset sensor value
        self.sensorVal = -1

    ''' Get next action depending on sensor mapping '''
    def getNextAction(self):
        return self.sensorActions[self.getSensorValue()]
```

```
    def getSensorValue(self):
        # If sensor value has already been calculated
        if self.sensorVal > -1:
            return self.sensorVal


        frontSensor = frontLeftSensor = frontRightSensor =
leftSensor = rightSensor = backSensor = False


        # Find which sensors have been activated
        if self.getHeading() == 0:
            frontSensor = self.world.isWall(self.xPosition,
self.yPosition-1)
            frontLeftSensor = self.world.isWall(self.xPosition-
1, self.yPosition-1)
            frontRightSensor =
self.world.isWall(self.xPosition+1, self.yPosition-1)
            leftSensor = self.world.isWall(self.xPosition-1,
self.yPosition)
            rightSensor = self.world.isWall(self.xPosition+1,
self.yPosition)
            backSensor = self.world.isWall(self.xPosition,
self.yPosition+1)
        elif self.getHeading() == 1:
            frontSensor = self.world.isWall(self.xPosition+1,
self.yPosition)
            frontLeftSensor =
self.world.isWall(self.xPosition+1, self.yPosition-1)
            frontRightSensor =
self.world.isWall(self.xPosition+1, self.yPosition+1)
            leftSensor = self.world.isWall(self.xPosition,
self.yPosition-1)
            rightSensor = self.world.isWall(self.xPosition,
self.yPosition+1)
            backSensor = self.world.isWall(self.xPosition-1,
self.yPosition)
        elif self.getHeading() == 2:
```

```
            frontSensor = self.world.isWall(self.xPosition,
self.yPosition+1)
            frontLeftSensor =
self.world.isWall(self.xPosition+1, self.yPosition+1)
            frontRightSensor =
self.world.isWall(self.xPosition-1, self.yPosition+1)
            leftSensor = self.world.isWall(self.xPosition+1,
self.yPosition)
            rightSensor = self.world.isWall(self.xPosition-1,
self.yPosition)
            backSensor = self.world.isWall(self.xPosition,
self.yPosition-1)
        else:
            frontSensor = self.world.isWall(self.xPosition-1,
self.yPosition)
            frontLeftSensor = self.world.isWall(self.xPosition-
1, self.yPosition+1)
            frontRightSensor =
self.world.isWall(self.xPosition-1, self.yPosition-1)
            leftSensor = self.world.isWall(self.xPosition,
self.yPosition+1)
            rightSensor = self.world.isWall(self.xPosition,
self.yPosition-1)
            backSensor = self.world.isWall(self.xPosition+1,
self.yPosition)


        # Calculate sensor value
        sensorVal = 0
        if frontSensor == True:
            sensorVal += 1
        if frontLeftSensor == True:
            sensorVal += 2
        if frontRightSensor == True:
            sensorVal += 4
        if leftSensor == True:
            sensorVal += 8
        if rightSensor == True:
            sensorVal += 16
        if backSensor == True:
```

```
            sensorVal += 32
        self.sensorVal = sensorVal
        return sensorVal


    ''' Map robot's sensor data to actions from binary string
'''
    def calcSensorActions(self, sensorActionsStr):
        # How many actions are there?
        numActions = int(len(sensorActionsStr)/2)
        sensorActions = [None] * numActions


        # Loop through actions
        for sensorValue in range(numActions):
            # Get sensor action
            sensorAction = 0
            if sensorActionsStr[sensorValue*2] == 1:
                sensorAction += 2
            if sensorActionsStr[sensorValue*2+1] == 1:
                sensorAction += 1
            # Add to sensor-action map
            sensorActions[sensorValue] = sensorAction

        return sensorActions


    ''' Get robot's position '''
    def getPosition(self):
        return [self.xPosition, self.yPosition]


    ''' Get robot's heading '''
    def getHeading(self):
        return self.heading


    ''' Returns robot's complete route around the maze '''
```

```
        def getRoute(self):
            return self.route


        ''' Returns route in printable format '''
        def printRoute(self):
            route = ""
            for routeStep in self.route:
                step = [None] * routeStep
                route += "{" + step[0] + "," + step[1] + "}"
            return route
```

Activity 5

Defining Population

The population refers to a group of chromosomes.

Population.py

```
from individual import Individual
import random


class Population:

    ''' Initializes population of individuals '''

    def __init__(self, populationSize, chromosomeLength=0):
        # Initializes blank population of individuals
        if chromosomeLength == 0:
            self.population = [None] * populationSize
        else:
            # Initialize the population as an array of individuals
            self.population = [None] * populationSize
            for individualCount in range(populationSize):
                # Create an individual, initializing its chromosome
to the given length
                individual = Individual(chromosomeLength)
                self.population[individualCount] = individual

    ''' Get individuals from the population '''
```

```
    def getIndividuals(self):
        return self.population

    ''' Find an individual in the population by its fitness '''

    def getFittest(self, offset):

        # Order population by fitness
        self.population.sort(key=lambda x: x.getFitness(),
reverse=True)

        # Return the fittest individual
        return self.population[offset]

    ''' Set population's group fitness '''

    def setPopulationFitness(self, fitness):
        self.populationFitness = fitness

    ''' Get population's group fitness '''

    def getPopulationFitness(self):
        return self.populationFitness

    ''' Get population's size '''

    def size(self):
        return len(self.population)

    ''' Set individual at offset '''

    def setIndividual(self, offset, individual):
        self.population[offset] = individual
        return individual

    ''' Get individual at offset '''

    def getIndividual(self, offset):
        return self.population[offset]

    ''' Shuffles the population in-place '''

    def shuffle(self):
        random.shuffle(self.population)
```

Activity 6:

Implementing GA Algoritm

We implement methods to calculate fitness, select individuals, crossover and do mutation.

ga.py:

```
import random
from population import Population
from robot import Robot
from individual import Individual


class GA():

    def __init__(self, populationSize, mutationRate, crossoverRate,
elitismCount, tournamentSize):
        self.populationSize = populationSize
        self.mutationRate = mutationRate
        self.crossoverRate = crossoverRate
        self.elitismCount = elitismCount
        self.tournamentSize = tournamentSize

    def initPopulation(self, chromosomeLength):
        print("Initialize population")
        population = Population(self.populationSize,
chromosomeLength)
        return population

    ''' Calculate fitness for an individual '''

    def calcFitness(self, individual, world):
        # Get individual's chromosome
        chromosome = individual.getChromosome()

        # Get fitness
        robot = Robot(chromosome, world, 100)
        robot.run()
        fitness = world.scoreRoute(robot.getRoute())
        # Store fitness
        individual.setFitness(fitness)
        return fitness
```

```
    ''' Evaluate the whole population '''

    def evalPopulation(self, population, world):
        populationFitness = 0
        for individual in population.getIndividuals():
            populationFitness += self.calcFitness(individual, world)
        population.setPopulationFitness(populationFitness)

    ''' Check if population has met termination condition '''

    def isTerminationConditionMet(self, generationsCount,
maxGenerations):
        return (generationsCount > maxGenerations)

    ''' Selects parent for crossover using tournament selection '''
    def selectParent(self, population):
        # Create tournament
        tournament = Population(self.tournamentSize)

        # Add random individuals to the tournament
        population.shuffle()
        for i in range(self.tournamentSize):
            tournamentIndividual = population.getIndividual(i)
            tournament.setIndividual(i, tournamentIndividual)

        # Return the best
        return tournament.getFittest(0)

    ''' Apply mutation to population '''
    def mutatePopulation(self, population):
        # Initialize new population
        newPopulation = Population(self.populationSize)
        for populationIndex in range(population.size()):
            individual = population.getFittest(populationIndex)
            for geneIndex in range(individual.getChromosomeLength()):
                # Skip mutation if this is an elite individual
                if (populationIndex >= self.elitismCount):
                    # Does this gene need mutation?
                    if (self.mutationRate > random.random()):
                        # Get new gene
                        newGene = 1
                        if (individual.getGene(geneIndex) == 1):
                            newGene = 0
                        # Mutate gene
                        individual.setGene(geneIndex, newGene)
```

```
            # Add individual to population
            newPopulation.setIndividual(populationIndex, individual)

        # Return mutated population
        return newPopulation

    ''' Crossover population using single point crossover '''
    def crossoverPopulation(self, population):
        # Create new population
        newPopulation = Population(population.size())
        # Loop over current population by fitness
        for populationIndex in range(population.size()):
            parent1 = population.getFittest(populationIndex)
            # Apply crossover to this individual?
            if (self.crossoverRate > random.random()) and
(populationIndex >= self.elitismCount):
                # Initialize offspring
                offspring = Individual(parent1.getChromosomeLength())

                # Find second parent
                parent2 = self.selectParent(population)

                # Get random swap point
                swapPoint = (int) (random.random() *
(parent1.getChromosomeLength() + 1))

                # Loop over genome
                for geneIndex in
range(parent1.getChromosomeLength()):
                    # Use half of parent1's genes and half of
parent2's genes
                    if (geneIndex < swapPoint):
                        offspring.setGene(geneIndex,
parent1.getGene(geneIndex))
                    else:
                        offspring.setGene(geneIndex,
parent2.getGene(geneIndex))

                # Add offspring to new population
                newPopulation.setIndividual(populationIndex,
offspring)
            else:
                # Add individual to new population without applying
crossover
```

```
                newPopulation.setIndividual(populationIndex, parent1)
        return newPopulation
```

Activity 7

Implementing Robot Controller

Finally, we can write a class that actually executes the algorithm. Create another new file called main.py as follows.

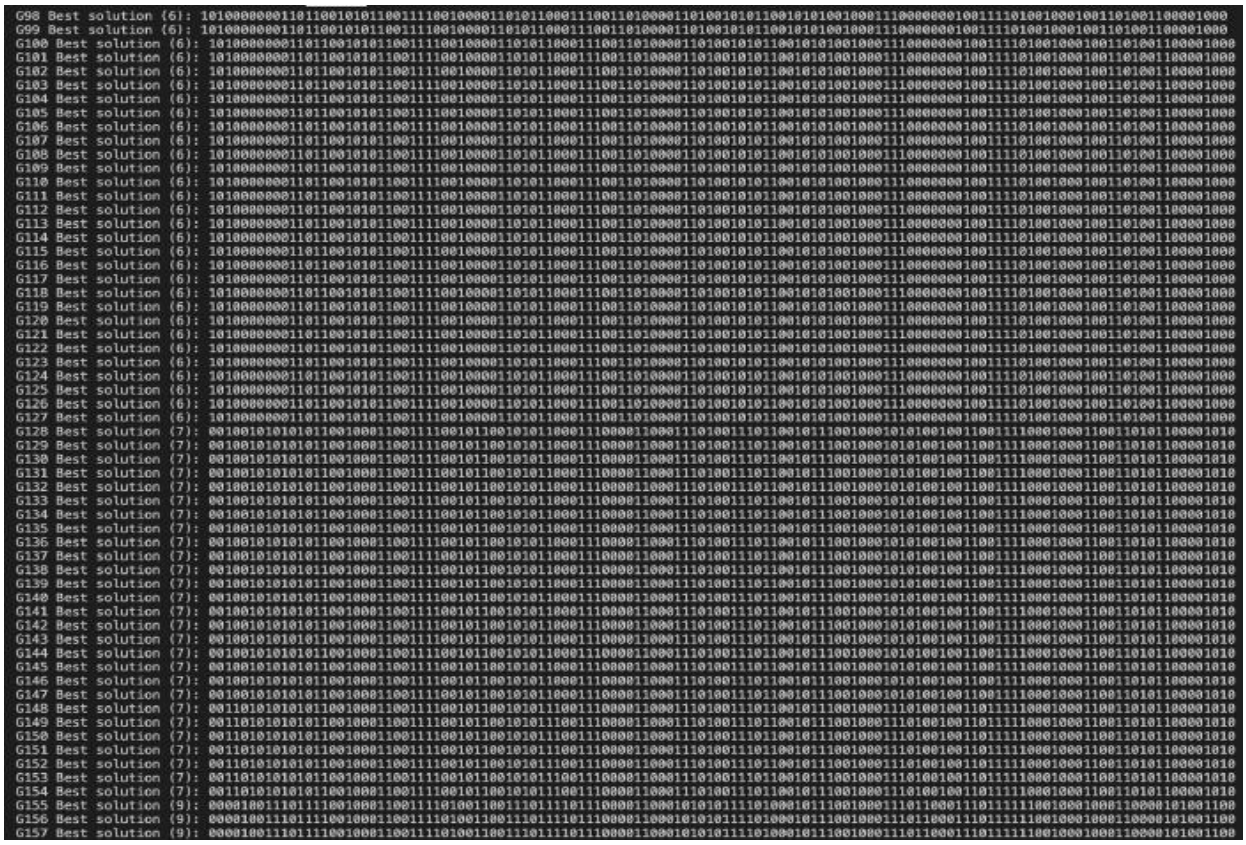
main.py

```
from world import World
from ga import GA

'''
As a reminder:
0 = Empty
1 = Wall
2 = Starting position
3 = Route
4 = Goal position
'''

maxGenerations = 1000

def runGA():
    myWorld = World([
        [0, 0, 0, 0, 1, 0, 1, 3, 1],
        [1, 0, 1, 1, 1, 0, 2, 3, 1],
        [1, 0, 0, 1, 3, 3, 3, 3, 1],
        [3, 3, 3, 1, 3, 1, 1, 0, 1],
        [3, 1, 3, 3, 3, 1, 1, 0, 0],
        [3, 3, 1, 1, 1, 1, 0, 1, 1],
        [1, 3, 0, 1, 3, 3, 3, 3, 3],
        [0, 3, 1, 1, 3, 1, 0, 1, 3],
        [1, 3, 3, 3, 3, 1, 1, 1, 4],
    ])

    ga = GA(20, 0.05, 0.9, 2, 10)
    population = ga.initPopulation(128)
    ga.evalPopulation(population, myWorld)

    # Keep track of current generation
```

```
    generation = 1
    while ga.isTerminationConditionMet(generation, maxGenerations) ==
False:
        # Print fittest individual from population
        fittest = population.getFittest(0)

        print("G" + str(generation) + " Best solution (" +
str(fittest.getFitness()) + "): " + fittest.toString())

        # Apply crossover
        population = ga.crossoverPopulation(population)

        # Apply mutation
        population = ga.mutatePopulation(population)

        # Evaluate population
        ga.evalPopulation(population, myWorld)

        # Increment the current generation
        generation +=1
    print("Stopped after " + str(maxGenerations) + " generations.")
    fittest = population.getFittest(0)
    print("Best solution (" + str(fittest.getFitness()) + "): " +
fittest.toString())

if __name__ == '__main__':
    runGA()
```

To run the program places all files in one directory with the specified names and run “python3 main.py in cmd”

Output



3) Graded Lab Tasks

Note: The instructor can design graded lab activities according to the level of difficulty and complexity of the solved lab activities. The lab tasks assigned by the instructor should be evaluated in the same lab

Lab Task 1

Extend the above robot controller program for five action commands:

- *do nothing;*
- *move forward;*
- *move back;*
- *turn left;*
- *turn right;*

Lab 08

Introduction to Keras

Objective:

The objective of this lab is to introduce Keras by implementing diabetes prediction model.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Load Data.
- Define Keras Model.
- Compile Keras Model.
- Fit Keras Model.
- Evaluate Keras Model.
- Tie It All Together.
- Make Predictions

Instructor Note:

As pre-lab activity, read Chapter 2 of book "Gulli, Antonio, and Sujit Pal. Deep learning with Keras. Packt Publishing Ltd, 2017."

1) Useful Concepts

Introduction: Keras is a powerful and easy-to-use free open source Python library for developing and evaluating deep learning models. It wraps the efficient numerical computation libraries Theano and TensorFlow and allows you to define and train neural network models in just a few lines of code. In this tutorial, you will discover how to create your first deep learning neural network model in Python using Keras. Models in Keras are defined as a sequence of layers. We create a Sequential model and add layers one at a time until we are happy with our network architecture.

Loading data using Keras : The first step is to define the functions and classes we intend to use in this tutorial. We will use the NumPy library to load our dataset and we will use two classes from the Keras library to define our model. The imports required are listed below.

Syntax :

```
# first neural network with keras tutorial
from numpy import loadtxt
from keras.models import Sequential
from keras.layers import Dense
```

We can now load our dataset.

In this Keras tutorial, we are going to use the Pima Indians onset of diabetes dataset. This is a standard machine learning dataset from the UCI Machine Learning repository. It describes patient medical record data for Pima Indians and whether they had an onset of diabetes within five years.

As such, it is a binary classification problem (onset of diabetes as 1 or not as 0). All of the input variables that describe each patient are numerical. This makes it easy to use directly with neural networks that expect numerical input and output values, and ideal for our first neural network in Keras.

The dataset is available from here:

Dataset CVS file: https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv

Dataset details: https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.names

Download the dataset and place it in your local working directory, the same location as your python file.

Save it with the filename:

Syntax :

```
pima-indians-diabetes.csv
```

Take a look inside the file, you should see rows of data like the following:

Syntax :

```
6,148,72,35,0,33.6,0.627,50,1
1,85,66,29,0,26.6,0.351,31,0
8,183,64,0,0,23.3,0.672,32,1
1,89,66,23,94,28.1,0.167,21,0
0,137,40,35,168,43.1,2.288,33,1
```

We can now load the file as a matrix of numbers using the NumPy function loadtxt().

There are eight input variables and one output variable (the last column). We will be learning a model to map rows of input variables (X) to an output variable (y), which we often summarize as y = f(X).

The variables can be summarized as follows:

Input Variables (X):

1. Number of times pregnant

2. Plasma glucose concentration a 2 hours in an oral glucose tolerance test

3. Diastolic blood pressure (mm Hg)
4. Triceps skin fold thickness (mm)
5. 2-Hour serum insulin (mu U/ml)
6. Body mass index (weight in kg/(height in m)^2)
7. Diabetes pedigree function
8. Age (years)

Output Variables (y):

1. Class variable (0 or 1)

Once the CSV file is loaded into memory, we can split the columns of data into input and output variables.

The data will be stored in a 2D array where the first dimension is rows and the second dimension is columns, e.g. [rows, columns].

We can split the array into two arrays by selecting subsets of columns using the standard NumPy slice operator or ":" We can select the first 8 columns from index 0 to index 7 via the slice 0:8. We can then select the output column (the 9th variable) via index 8.

Syntax :

```
...
# load the dataset
dataset = loadtxt('pima-indians-diabetes.csv',
delimiter=',')
# split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
```

Define model in Keras: The first thing to get right is to ensure the input layer has the right number of input features. This can be specified when creating the first layer with the input_dim argument and setting it to 8 for the 8 input variables.

How do we know the number of layers and their types?

This is a very hard question. There are heuristics that we can use and often the best network structure is found through a process of trial and error experimentation. Generally, you need a network large enough to capture the structure of the problem.

In this example, we will use a fully-connected network structure with three layers.

Fully connected layers are defined using the Dense class. We can specify the number of neurons or nodes in the layer as the first argument, and specify the activation function using the activation argument.

We will use the rectified linear unit activation function referred to as ReLU on the first two layers and the Sigmoid function in the output layer.

It used to be the case that Sigmoid and Tanh activation functions were preferred for all layers. These days, better performance is achieved using the ReLU activation function. We use a sigmoid on the output layer to ensure our network output is between 0 and 1 and easy to map to either a probability of class 1 or snap to a hard classification of either class with a default threshold of 0.5.

We can piece it all together by adding each layer:

- The model expects rows of data with 8 variables (the input_dim=8 argument)
- The first hidden layer has 12 nodes and uses the relu activation function.
- The second hidden layer has 8 nodes and uses the relu activation function.
- The output layer has one node and uses the sigmoid activation function.

Syntax :

```
...
# define the keras model
model = Sequential()
model.add(Dense(12, input_dim=8, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```

Compile Model in Keras: Now that the model is defined, we can compile it. Compiling the model uses the efficient numerical libraries under the covers (the so-called backend) such as Theano or TensorFlow. The backend automatically chooses the best way to represent the network for training and making predictions to run on your hardware, such as CPU or GPU or even distributed.

When compiling, we must specify some additional properties required when training the network. Remember training a network means finding the best set of weights to map inputs to outputs in our dataset.

We must specify the loss function to use to evaluate a set of weights, the optimizer is used to search through different weights for the network and any optional metrics we would like to collect and report during training.

In this case, we will use cross entropy as the loss argument. This loss is for a binary classification problems and is defined in Keras as “binary_crossentropy“. You can learn more about choosing loss

functions based on your problem here: https://machinelearningmastery.com/how-to-choose-loss-functions-when-training-deep-learning-neural-networks/

We will define the optimizer as the efficient stochastic gradient descent algorithm "adam". This is a popular version of gradient descent because it automatically tunes itself and gives good results in a wide range of problems. To learn more about the Adam version of stochastic gradient descent see the post: https://machinelearningmastery.com/adam-optimization-algorithm-for-deep-learning/

Finally, because it is a classification problem, we will collect and report the classification accuracy, defined via the metrics argument.

Syntax :

```
...
# compile the keras model
model.compile(loss='binary_crossentropy',
optimizer='adam', metrics=['accuracy'])
```

Fit/Train Model in Keras: We have defined our model and compiled it ready for efficient computation. Now it is time to execute the model on some data. We can train or fit our model on our loaded data by calling the fit() function on the model. Training occurs over epochs and each epoch is split into batches.

Epoch: One pass through all of the rows in the training dataset.

Batch: One or more samples considered by the model within an epoch before weights are updated.

One epoch is comprised of one or more batches, based on the chosen batch size and the model is fit for many epochs. For more on the difference between epochs and batches, see the post: https://machinelearningmastery.com/difference-between-a-batch-and-an-epoch/.

The training process will run for a fixed number of iterations through the dataset called epochs, that we must specify using the epochs argument. We must also set the number of dataset rows that are considered before the model weights are updated within each epoch, called the batch size and set using the batch_size argument.

For this problem, we will run for a small number of epochs (150) and use a relatively small batch size of 10.

These configurations can be chosen experimentally by trial and error. We want to train the model enough so that it learns a good (or good enough) mapping of rows of input data to the output classification. The model will always have some error, but the amount of error will level out after some point for a given model configuration. This is called model convergence.

Syntax :

```
...
# fit the keras model on the dataset
```

```
model.fit(X, y, epochs=150, batch_size=10)
```

This is where the work happens on your CPU or GPU.

No GPU is required for this example, but if you're interested in how to run large models on GPU hardware cheaply in the cloud, see this post: https://machinelearningmastery.com/develop-evaluate-large-deep-learning-models-keras-amazon-web-services/

Evaluate Model in Keras: We have trained our neural network on the entire dataset and we can evaluate the performance of the network on the same dataset.

This will only give us an idea of how well we have modeled the dataset (e.g. train accuracy), but no idea of how well the algorithm might perform on new data. We have done this for simplicity, but ideally, you could separate your data into train and test datasets for training and evaluation of your model.

You can evaluate your model on your training dataset using the evaluate() function on your model and pass it the same input and output used to train the model.

This will generate a prediction for each input and output pair and collect scores, including the average loss and any metrics you have configured, such as accuracy.

The evaluate() function will return a list with two values. The first will be the loss of the model on the dataset and the second will be the accuracy of the model on the dataset. We are only interested in reporting the accuracy, so we will ignore the loss value.

Syntax :

```
# evaluate the keras model
_, accuracy = model.evaluate(X, y)
print('Accuracy: %.2f' % (accuracy*100))
```

Tie it together in Keras: You have just seen how you can easily create your first neural network model in Keras.

Let's tie it all together into a complete code example.

Syntax :

```
# first neural network with keras tutorial
from numpy import loadtxt
from keras.models import Sequential
from keras.layers import Dense
# load the dataset
dataset = loadtxt('pima-indians-diabetes.csv',
delimiter=',')
# split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
```

```
# define the keras model
model = Sequential()
model.add(Dense(12, input_dim=8, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# compile the keras model
model.compile(loss='binary_crossentropy',
optimizer='adam', metrics=['accuracy'])
# fit the keras model on the dataset
model.fit(X, y, epochs=150, batch_size=10)
# evaluate the keras model
_, accuracy = model.evaluate(X, y)
print('Accuracy: %.2f' % (accuracy*100))
```

You can copy all of the code into your Python file and save it as "keras_first_network.py" in the same directory as your data file "pima-indians-diabetes.csv". You can then run the Python file as a script from your command line (command prompt) as follows:

Syntax :

```
python keras_first_network.py
```

Running this example, you should see a message for each of the 150 epochs printing the loss and accuracy, followed by the final evaluation of the trained model on the training dataset.

It takes about 10 seconds to execute on my workstation running on the CPU.

Ideally, we would like the loss to go to zero and accuracy to go to 1.0 (e.g. 100%). This is not possible for any but the most trivial machine learning problems. Instead, we will always have some error in our model. The goal is to choose a model configuration and training configuration that achieve the lowest loss and highest accuracy possible for a given dataset.

Syntax :

```
...
768/768 [==============================] - 0s 63us/step
- loss: 0.4817 - acc: 0.7708
Epoch 147/150
768/768 [==============================] - 0s 63us/step
- loss: 0.4764 - acc: 0.7747
Epoch 148/150
768/768 [==============================] - 0s 63us/step
- loss: 0.4737 - acc: 0.7682
Epoch 149/150
```

```
768/768 [==============================] - 0s 64us/step
- loss: 0.4730 - acc: 0.7747
Epoch 150/150
768/768 [==============================] - 0s 63us/step
- loss: 0.4754 - acc: 0.7799
768/768 [==============================] - 0s 38us/step
Accuracy: 76.56
```

Note, if you try running this example in an IPython or Jupyter notebook you may get an error.

The reason is the output progress bars during training. You can easily turn these off by setting verbose=0 in the call to the fit() and evaluate() functions, for example:

Syntax :

```
# fit the keras model on the dataset without progress bars
model.fit(X, y, epochs=150, batch_size=10, verbose=0)
# evaluate the keras model
_, accuracy = model.evaluate(X, y, verbose=0)
```

Note: Your results may vary given the stochastic nature of the algorithm or evaluation procedure, or differences in numerical precision. Consider running the example a few times and compare the average outcome.

What score did you get?
Post your results in the comments below.

Neural networks are a stochastic algorithm, meaning that the same algorithm on the same data can train a different model with different skill each time the code is run. This is a feature, not a bug. You can learn more about this in the post:

Making Model Predictions in Keras: We can adapt the above example and use it to generate predictions on the training dataset, pretending it is a new dataset we have not seen before.

Making predictions is as easy as calling the predict() function on the model. We are using a sigmoid activation function on the output layer, so the predictions will be a probability in the range between 0 and 1. We can easily convert them into a crisp binary prediction for this classification task by rounding them. For example:

Syntax :

```
...
# make probability predictions with the model
predictions = model.predict(X)
```

```
# round predictions
rounded = [round(x[0]) for x in predictions]
```

Alternately, we can convert the probability into 0 or 1 to predict crisp classes directly, for example:

Syntax :

```
...
# make class predictions with the model
predictions = (model.predict(X) > 0.5).astype(int)
```

The complete example below makes predictions for each example in the dataset, then prints the input data, predicted class and expected class for the first 5 examples in the dataset.

```
# first neural network with keras make predictions
from numpy import loadtxt
from keras.models import Sequential
from keras.layers import Dense
# load the dataset
dataset = loadtxt('pima-indians-diabetes.csv',
delimiter=',')
# split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
# define the keras model
model = Sequential()
model.add(Dense(12, input_dim=8, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
# compile the keras model
model.compile(loss='binary_crossentropy',
optimizer='adam', metrics=['accuracy'])
# fit the keras model on the dataset
model.fit(X, y, epochs=150, batch_size=10, verbose=0)
# make class predictions with the model
predictions = (model.predict(X) > 0.5).astype(int)
# summarize the first 5 cases
for i in range(5):
      print('%s => %d (expected %d)' % (X[i].tolist(),
predictions[i], y[i]))
```

Running the example does not show the progress bar as before as we have set the verbose argument to 0. After the model is fit, predictions are made for all examples in the dataset, and the input rows and predicted class value for the first 5 examples is printed and compared to the expected class value.
We can see that most rows are correctly predicted. In fact, we would expect about 76.9% of the rows to be correctly predicted based on our estimated performance of the model in the previous section.

```
[6.0, 148.0, 72.0, 35.0, 0.0, 33.6, 0.627, 50.0] => 0
(expected 1)
[1.0, 85.0, 66.0, 29.0, 0.0, 26.6, 0.351, 31.0] => 0
(expected 0)
```

```
[8.0, 183.0, 64.0, 0.0, 0.0, 23.3, 0.672, 32.0] => 1
(expected 1)
[1.0, 89.0, 66.0, 23.0, 94.0, 28.1, 0.167, 21.0] => 0
(expected 0)
[0.0, 137.0, 40.0, 35.0, 168.0, 43.1, 2.288, 33.0] => 1
(expected 1)
```

2) Solved Lab Activities

Sr.no	*Allocated time*	*Level of complexity*	*CLO mapping*
1	*5 mins*	*Low*	*CLO-4,5*
2	*5 mins*	*Medium*	*CLO-4,5*
3	*10 mins*	*Medium*	*CLO-4,5*

Activity 1

Loading data using Keras

Syntax :

```
# first neural network with keras tutorial
from numpy import loadtxt
from keras.models import Sequential
from keras.layers import Dense
pima-indians-diabetes.csv
...
# load the dataset
dataset = loadtxt('pima-indians-diabetes.csv',
delimiter=',')
# split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
```

Activity 2

Define, Compile and Train Model in Keras

Syntax :

```
...
# define the keras model
model = Sequential()
model.add(Dense(12, input_dim=8, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

...
# compile the keras model
model.compile(loss='binary_crossentropy',
```

```
optimizer='adam', metrics=['accuracy'])

...
# fit the keras model on the dataset
model.fit(X, y, epochs=150, batch_size=10)
```

Activity 3

Evaluate Model in Keras

Syntax :

```
# evaluate the keras model
_, accuracy = model.evaluate(X, y)
print('Accuracy: %.2f' % (accuracy*100))

...
768/768 [==============================] - 0s 63us/step -
loss: 0.4817 - acc: 0.7708
Epoch 147/150
768/768 [==============================] - 0s 63us/step -
loss: 0.4764 - acc: 0.7747
Epoch 148/150
768/768 [==============================] - 0s 63us/step -
loss: 0.4737 - acc: 0.7682
Epoch 149/150
768/768 [==============================] - 0s 64us/step -
loss: 0.4730 - acc: 0.7747
Epoch 150/150
768/768 [==============================] - 0s 63us/step -
loss: 0.4754 - acc: 0.7799
768/768 [==============================] - 0s 38us/step
Accuracy: 76.56
```

3)Graded Lab Tasks

Note: The instructor can design graded lab activities according to the level of difficulty and complexity of the solved lab activities. The lab tasks assigned by the instructor should be evaluated in the same lab

Lab Task 1

Build a neural network model in keras to solve a classification problem from Kaggle or UCI machine learning repository.

Lab 09

Medical Image Classification using Keras

Objective:

The objective of this lab will learn the students about the Construction of CNN model for detection of pneumonia in x-rays using Keras.

Activity Outcomes:

The activities provide hands - on practice with the following topics

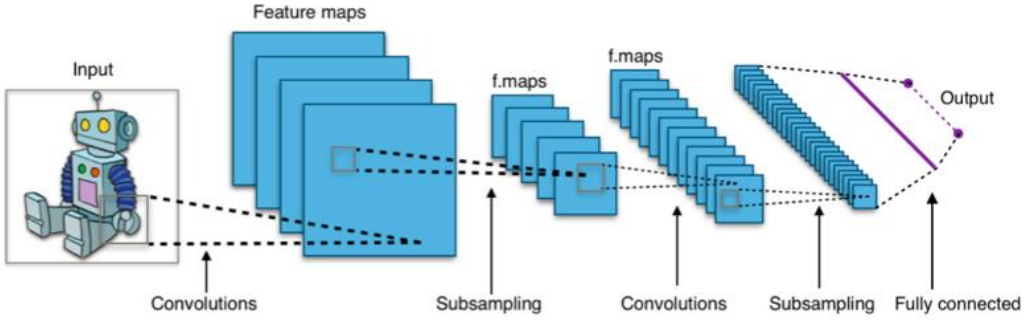
- Load Data.
- Define Keras Model.
- Compile Keras Model.
- Fit Keras Model.
- Evaluate Keras Model.
- Tie It All Together.
- Make Predictions

Instructor Note:

As pre-lab activity, read Chapter 3 of book "Gulli, Antonio, and Sujit Pal. Deep learning with Keras. Packt Publishing Ltd, 2017."

1)UsefulConcepts:

CNN architecture is based on layers of convolution. The convolution layers receive input and transform the data from the image and pass it as input to the next layer. The transformation is known as the operation of convolution. We need to define the number of filters for each convolution layer. These filters detect patterns such as edges, shapes, curves, objects, textures, or even colors. The more sophisticated patterns or objects it detects are more deeply layered. In essence, filters are image kernels that we can define as 3x3 or 4x4, which is a small matrix applied to an image as a whole. We will use Pooling layer together with Convolution layer as well as the goal is to down-sample an input representation (image), decrease its dimensionality by retaining the maximum value (activated features) in the sub regions binding. The number of pixels moving across the input matrix is called Stride. When the stride is 1 we move the filter to 1 pixel at a time. When the stride is 2 then we move the filter to 2 pixels at a time, and so on. Larger filter sizes and strides may be used to reduce the size of a large image to a moderate size.



CNN architecture

The Dataset: The dataset that we are going to use for the image classification is Chest X-Ray images, which consists of 2 categories, Pneumonia and Normal. This dataset was published by Paulo Breviglieri, a revised version of Paul Mooney's most popular dataset. This updated version of the dataset has a more balanced distribution of the images in the validation set and the testing set. The data set is organised into 3 folders (train, test, val) and contains subfolders for each image category Opacity(viz. Pneumonia) & Normal.The activities provide hands - on practice with the following total number of observations (images) 5,856:

- Training observations: 4,192 (1,082 normal cases, 3,110 lung opacity cases)
- Validation observations: 1,040 (267 normal cases, 773 lung opacity cases)
- Testing observations: 624 (234 normal cases, 390 lung opacity cases)

First, we will extract the dataset directly from Kaggle using the Kaggle API. To do this, we need to create an API token that is located in the Account section under the Kaggle API tab. Click on 'Create a new API token' and a json file will be downloaded.

Run the following lines of codes to instal the needed libraries and upload the json file.

```
! pip install -q kaggle
from google.colab import files
files.upload()
! mkdir ~/.kaggle
! cp kaggle.json ~/.kaggle/
! chmod 600 ~/.kaggle/kaggle.json
```

https://miro.medium.com/max/1332/1*fIF8cA0x_FdPjnItvI21Kw.png

When prompted to 'Choose Files,' upload the downloaded json file. Running the next line of code is going to download the dataset. To get the dataset API command to download the dataset, click the 3 dots in the data section of the Kaggle dataset page and click the 'Copy API command' button and paste it with the !

```
! kaggle datasets download -d pcbreviglieri/pneumonia-xray-images
```

Since I use Google Colab to run this project, the dataset zip file is downloaded to the Sample Data Folder. Now, by running the next lines of codes, we unzip folders and files to the desired target folder using the zipfile library.

```
import zipfilezf = "/content/pneumonia-xray-images.zip"
target_dir = "/content/dataset/cnn/pneumonia_revamped"zfile =
zipfile.ZipFile(zf)
zfile.extractall(target_dir)
```

Now that our dataset is ready, let's get rolling!

Initialize

In this part of the code, we will define the directory path, import some needed libraries, and define some common constant parameters that we will often use in later parts of the project.

Let’s take a look at our dataset directory tree.

Tree Directory:

```
content
    └──dataset
        └──cnn
            └──pneumonia_revamped
                ├──test
                │   ├──Normal
                │   │   ├──image1.jpg
                │   │   └──image2.jpg
                │   └──Opacity
                │       ├──image1.jpg
                │       └──image2.jpg
                ├──train
                ├──Normal
                │   │ ├──image1.jpg
                │   │ └──image2.jpg
                │   └──Opacity
                │       ├──image1.jpg
                │       └──image2.jpg
                └──val
                    ├──Normal
                    │   ├──image1.jpg
                    │   └──image2.jpg
                    └──Opacity
                        ├──image1.jpg
                        └──image2.jpg
```

Figure 5: Dataset Directory

Syntax :

```
#Some Basic Imports

import matplotlib.pyplot as plt     #For Visualization
import numpy as np                    #For handling arrays
import pandas as pd                          # For handling
data

#Define Directories for train, test & Validation Set
train_path =
```

```
'/content/dataset/cnn/pneumonia_revamped/train'
test_path = '/content/dataset/cnn/pneumonia_revamped/test'
valid_path = '/content/dataset/cnn/pneumonia_revamped/val'

#Define some often used standard parameters
#The batch refers to the number of training examples
utilized in one #iteration
batch_size = 16

#The dimension of the images we are going to define is
500x500
img_height = 500
img_width = 500
```

The dimension size of 500 or more than 500 with batch size greater than 16 may result in a crash as the RAM gets completely used in such cases. A lower dimension size with greater batch size is one of the options to try.

Data Augmentation: We will increase the size of the image training dataset artificially by performing some Image Augmentation technique.

Image Augmentation expands the size of the dataset by creating a modified version of the existing training set images that helps to increase dataset variation and ultimately improve the ability of the model to predict new images.

```
from tensorflow.keras.preprocessing.image import
ImageDataGenerator# Create Image Data Generator for Train Set
image_gen = ImageDataGenerator(
                                    rescale = 1./255,
                                    shear_range = 0.2,
                                    zoom_range = 0.2,
                                    horizontal_flip = True,
                                )# Create Image Data Generator for
Test/Validation Set
test_data_gen = ImageDataGenerator(rescale = 1./255)
```

Using the tensorflow.keras.preprocessing.image library, for the Train Set, we created an Image Data Generator that randomly applies defined parameters to the train set and for the Test & Validation set, we're just going to rescale them to avoid manipulating the test data beforehand.

1. rescale —Each digital image is created by a pixel with a value between 0 and 255. 0 in black, 255 in white. So rescale the scales array of the original image pixel values to be between [0,1] which makes the images contribute more equally to the overall loss.

Otherwise, higher pixel range image results in greater loss and a lower learning rate should be used, lower pixel range image would require a higher learning rate.

2. shear_range — The shape of the image is the transformation of the shear. It fixes one axis and stretches the image at a certain angle known as the angle of the shear.

3. zoom_range — The image is enlarged by a zoom of less than 1.0. The image is more than 1.0 zoomed out of the picture.

4. horizontal_flip —Some images are flipped horizontally at random

5. vertical_flip — Some images are flipped vertically at random

6. roataion_range — Randomly, the image is rotated by some degree in the range 0 to 180.

7. width_shift_range — Shifts the image horizontally.

8. height_shift_range — Shifts the image vertically.

9. brightness_range — brightness of 0.0 corresponds to absolutely no brightness, and 1.0 corresponds to maximum brightness

10. fill_mode — Fills the missing value of the image to the nearest value or to the wrapped value or to the reflecting value.

These transformation techniques are applied randomly to the images, except for the rescale. All images have been rescaled.

Loading the Images:

The Image Data Generator has a class known as flow from directory to read the images from folders containing images. Returns the Directory Iterator

```
typetensorflow.python.keras.preprocessing.image.DirectoryIterator.
```

```
train   =    image_gen.flow_from_directory(
      train_path,
      target_size=(img_height, img_width),
      color_mode='grayscale',
      class_mode='binary',
      batch_size=batch_size
      )
```

```
 test = test_data_gen.flow_from_directory(
      test_path,
      target_size=(img_height, img_width),
      color_mode='grayscale',
      shuffle=False,

#setting shuffle as False just so we can later compare it with
predicted values without having indexing problem
      class_mode='binary',
      batch_size=batch_size
      )

valid = test_data_gen.flow_from_directory(
      valid_path,
      target_size=(img_height, img_width),
      color_mode='grayscale',
      class_mode='binary',
      batch_size=batch_size
      )
```

Found 4192 images belonging to 2 classes. Found 624 images belonging to 2 classes. Found 1040 images belonging to 2 classes.

Parameters with Definition:

1. directory — The first parameter used is the path of the train, test & validation folder that we defined earlier.

2. target_size — The target size is the size of your input images, each image will be resized to this size. We have defined the target size earlier as 500 x 500.

3. color_mode —If the image is either black and white or grayscale set to "grayscale" or if the image has three colour channels set to "rgb." We're going to work with the grayscale, because it's the X-Ray images.

4. batch_size — Number of images to be generated by batch from the generator. We defined the batch size as 16 earlier. We choose 16 because the size of the images is too large to handle the RAM.

5. class_mode — Set "binary" if you only have two classes to predict, if you are not set to "categorical," if you develop an Autoencoder system, both input and output are

likely to be the same image, set to “input” in this case. Here we’re going to set it to binary because we’ve only got 2 classes to predict

```
plt.figure(figsize=(12, 12))
for i in range(0, 10):
           plt.subplot(2, 5, i+1)
           for X_batch, Y_batch in train:
                       image = X_batch[0]
                        dic = {0:’NORMAL’, 1:’PNEUMONIA’}
                         plt.title(dic.get(Y_batch[0]))
                         plt.axis(’off’)

plt.imshow(np.squeeze(image),cmap=’gray’,interpolation=’nearest’)
break
plt.tight_layout()
plt.show()
```

Defining Convolutional Neural Network Model Architecture:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import
Dense,Conv2D,Flatten,MaxPooling2D
from tensorflow.keras.callbacks import
EarlyStopping,ReduceLROnPlateau
```

Things to note before starting to build a CNN model:-

1. Always begin with a lower filter value such as 32 and begin to increase it layer wise.
2. Construct the model with a layer of Conv2D followed by a layer of MaxPooling.
3. The kernel_size is preferred to be odd number like 3x3.
4. Tanh, relu, etc. can be used for activation function, but relu is the most preferred activation function.
5. input_shape takes in image width & height with last dimension as color channel.
6. Flattening the input after CNN layers and adding ANN layers.

7. Use activation function as softmax for the last layer If the problem is more than 2 classes, define units as the total number of classes and use sigmoid for binary classification and set unit to 1.

Note :- You can always experiment with these hyperparameters as there is no fixed value on which we can settle.

```
cnn = Sequential()

cnn.add(Conv2D(32, (3, 3), activation="relu",
input_shape=(img_width, img_height, 1)))
cnn.add(MaxPooling2D(pool_size = (2, 2)))

cnn.add(Conv2D(32, (3, 3), activation="relu",
input_shape=(img_width, img_height, 1)))
cnn.add(MaxPooling2D(pool_size = (2, 2)))

cnn.add(Conv2D(32, (3, 3), activation="relu",
input_shape=(img_width, img_height, 1)))
cnn.add(MaxPooling2D(pool_size = (2, 2)))

cnn.add(Conv2D(64, (3, 3), activation="relu",
input_shape=(img_width, img_height, 1)))
cnn.add(MaxPooling2D(pool_size = (2, 2)))
cnn.add(Conv2D(64, (3, 3), activation="relu",
input_shape=(img_width, img_height, 1)))
cnn.add(MaxPooling2D(pool_size = (2, 2)))

cnn.add(Flatten())

cnn.add(Dense(activation = 'relu', units = 128))
cnn.add(Dense(activation = 'relu', units = 64))
cnn.add(Dense(activation = 'sigmoid', units = 1))

cnn.compile(optimizer = 'adam', loss = 'binary_crossentropy',
metrics = ['accuracy'])
```

Now we've developed the CNN model, let's see in depth what's going on here.

```
cnn.summary()
Model: "sequential_1"
_________________________________________________________________ Layer (type)
Output Shape              Param #
```

```
=================================================================
conv2d_3 (Conv2D)           (None, 498, 498, 32)      320
_________________________________________________________________ max_pooling2d_3
(MaxPooling2 (None, 249, 249, 32)      0
_________________________________________________________________ conv2d_4
(Conv2D)           (None, 247, 247, 32)      9248
_________________________________________________________________ max_pooling2d_4
(MaxPooling2 (None, 123, 123, 32)      0
_________________________________________________________________ conv2d_5
(Conv2D)           (None, 121, 121, 32)      9248
_________________________________________________________________ max_pooling2d_5
(MaxPooling2 (None, 60, 60, 32)        0
_________________________________________________________________ conv2d_6
(Conv2D)           (None, 58, 58, 64)        18496
_________________________________________________________________ max_pooling2d_6
(MaxPooling2 (None, 29, 29, 64)        0
_________________________________________________________________ conv2d_7
(Conv2D)           (None, 27, 27, 64)        36928
_________________________________________________________________ max_pooling2d_7
(MaxPooling2 (None, 13, 13, 64)        0
_________________________________________________________________ flatten_1 (Flatten)
(None, 10816)             0
_________________________________________________________________ dense_2 (Dense)
(None, 128)               1384576
_________________________________________________________________ dense_3 (Dense)
(None, 64)                8256
_________________________________________________________________ dense_4 (Dense)
(None, 1)                 65
================================================================= Total
params: 1,467,137 Trainable params: 1,467,137 Non-trainable params: 0
```

Defining Model Compile:

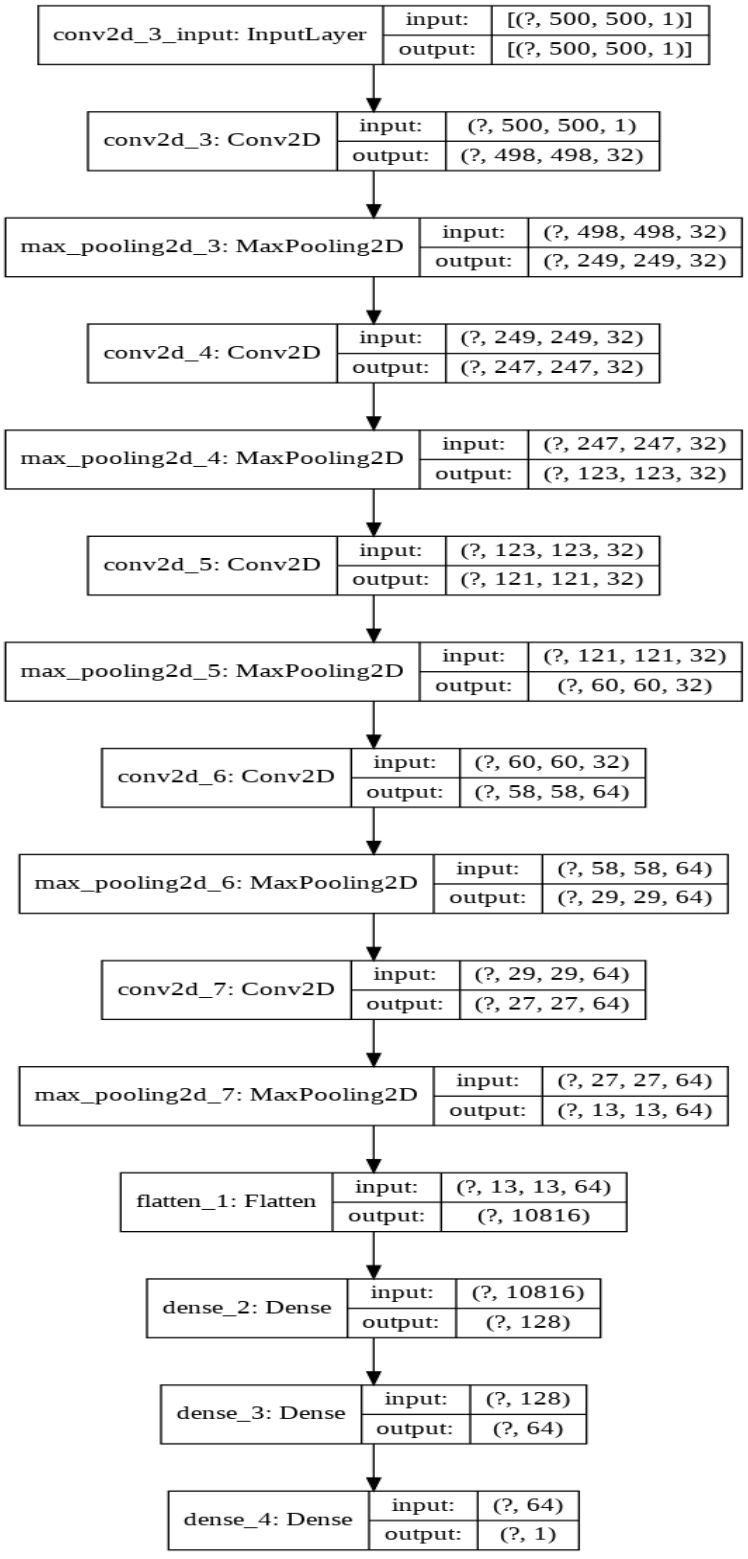
- Learning Rate — while training the aim for stochastic gradient descent is to minimize loss among actual and predicted values of training set. Path to minimize loss takes several steps. Adam is an adaptive learning rate method, which means, it computes individual learning rates for different parameters.

- loss function — Since it is a binary classification, we will use binary crossentropy during training for evaluation of losses. We would have gone for categorical crossentropy if there were more than 4 classes.

- metrics — accuracy — Calculate how often actual labels are equal to predictions. It will measure the loss and accuracy of training and validation.

Visualize CNN model:

```
from tensorflow.keras.utils import
plot_modelplot_model(cnn,show_shapes=True, show_layer_names=True,
rankdir='TB', expand_nested=True)
```



Plot CNN:

Training and Evaluating the model:

EarlyStopping is called to stop the epochs based on some metric(monitor) and conditions (mode, patience) . It helps to avoid overfitting the model. Over here we are telling to stop based on val_loss metric, we need it to be minimum. patience says that after a minimum val_loss is achieved then after that in next iterations if the val_loss increases in any the 3 iterations then the the training will stop at that epoch.

Reduce learning rate when a metric has stopped improving. Models often benefit from reducing the learning rate by a factor of 2–10 once learning stagnates. This callback monitors a quantity and if no improvement is seen for a 'patience' number of epochs, the learning rate is reduced.

```
early = EarlyStopping(monitor="val_loss", mode="min",
patience=3)learning_rate_reduction =
ReduceLROnPlateau(monitor='val_loss', patience = 2,
verbose=1,factor=0.3, min_lr=0.000001)callbacks_list = [ early,
learning_rate_reduction]
```

Assigning Class Weights:

It is good practice to assign class weights for each class. It emphasizes the weight of the minority class in order for the model to learn from all classes equally.

```
from sklearn.utils.class_weight import compute_class_weight
weights = compute_class_weight('balanced',
np.unique(train.classes), train.classes)
cw = dict(zip( np.unique(train.classes), weights))
print(cw)
```

```
{0: 1.9371534195933457, 1: 0.6739549839228296}
```

The parameters we are passing to model.fit are train set, epochs as 25, validation set used to calculate val_loss and val_accuracy, class weights and callback list.

```
cnn.fit(train,epochs=25, validation_data=valid, class_weight=cw,
callbacks=callbacks_list)
```

Evaluating model:

Let's visualize the progress of all metrics throughout the total epochs lifetime

```
pd.DataFrame(cnn.history.history).plot()
```

The accuracy we are getting on Test dataset is of 91%

```
test_accu = cnn.evaluate(test)print('The testing accuracy is
:',test_accu[1]*100, '%')
```

39/39 [==============================] — 50s 1s/step — loss: 0.3132 — accuracy: 0.9119 The testing accuracy is : 91.18589758872986 %

Let's predict the test dataset and look at some of the performance measurement metrics in detail to evaluate our model.

```
preds = cnn.predict(test,verbose=1)
```

39/39 [==============================] — 46s 1s/step

Since the activation function of the last layer is sigmoid, the model gives prediction in the 0 to 1 range and not an exact classification as 0 or 1. So we categorise all the values in the 0.5 to 1 range as 0 and less than 0.5 as 1. Note(0 denotes a normal case and 1 denotes a case of pneumonia)

```
predictions = preds.copy()
predictions[predictions <= 0.5] = 0
predictions[predictions > 0.5] = 1
```

Let's visualize some of the predicted images with percentage %

```
test.reset()
x=np.concatenate([test.next()[0] for i in range(test.__len__())])
y=np.concatenate([test.next()[1] for i in range(test.__len__())])
print(x.shape)
print(y.shape)#this little code above extracts the images from test
Data iterator without shuffling the sequence# x contains image
array and y has labels dic = {0:'NORMAL', 1:'PNEUMONIA'}
plt.figure(figsize=(20,20))
for i in range(0+228, 9+228):
  plt.subplot(3, 3, (i-228)+1)
  if preds[i, 0] >= 0.5:
      out = ('{:.2%} probability of being Pneumonia
case'.format(preds[i][0]))


  else:
      out = ('{:.2%} probability of being Normal case'.format(1-
```

```
preds[i][0]))plt.title(out+"\n Actual case : "+ dic.get(y[i]))
  plt.imshow(np.squeeze(x[i]))
  plt.axis('off')
plt.show()
```

This code block gives a percentage prediction of the individual image that can be loaded directly from your drive by specifying its path.

We have to re-create all the data preprocessing steps over here after importing the image as we had done previously to feed the test set into the model to get prediction. For pre-processing we need to import tensorflow.keras.preprocessing.image class.

1. Import image and define dimensions as (500,500) and color channel as grayscale.
2. Convert image to array, rescale it by dividing it 255 and expand dimension by axis = 0 as our model takes 4 dimensions as seen earlier.
3. Finally let's predict the case!

Task:

Let's do some field testing on this model with my X-ray

Link to my Colab Notebook for this project : colab.research.google.com

2) Solved Lab Activities:

Sr.no	*Allocated time*	*Level of complexity*	*CLO mapping*
1	*20 mins*	*Medium*	*CLO-5*
2	*20 mins*	*High*	*CLO-5*
3	*20 mins*	*High*	*CLO-5*

Activity 1

Importing libraries and setting environment

Syntax :

```
#Some Basic Imports

import matplotlib.pyplot as plt     #For Visualization
```

```
import numpy as np                      #For handling arrays
import pandas as pd                           # For handling
data

#Define Directories for train, test & Validation Set
train_path =
'/content/dataset/cnn/pneumonia_revamped/train'
test_path = '/content/dataset/cnn/pneumonia_revamped/test'
valid_path = '/content/dataset/cnn/pneumonia_revamped/val'

#Define some often used standard parameters
#The batch refers to the number of training examples
utilized in one #iteration
batch_size = 16

#The dimension of the images we are going to define is
500x500
img_height = 500
img_width = 500

from tensorflow.keras.preprocessing.image import
ImageDataGenerator# Create Image Data Generator for Train
Set
image_gen = ImageDataGenerator(
                                    rescale = 1./255,
                                    shear_range = 0.2,
                                    zoom_range = 0.2,
                                    horizontal_flip = True,
                                 )# Create Image Data
Generator for Test/Validation Set
test_data_gen = ImageDataGenerator(rescale = 1./255)

typetensorflow.python.keras.preprocessing.image.DirectoryIt
erator.

 train   =    image_gen.flow_from_directory(
      train_path,
      target_size=(img_height, img_width),
      color_mode='grayscale',
      class_mode='binary',
      batch_size=batch_size
      )
```

```
 test = test_data_gen.flow_from_directory(
      test_path,
      target_size=(img_height, img_width),
      color_mode='grayscale',
      shuffle=False,

#setting shuffle as False just so we can later compare it
with predicted values without having indexing problem
      class_mode='binary',
      batch_size=batch_size
      )

valid = test_data_gen.flow_from_directory(
      valid_path,
      target_size=(img_height, img_width),
      color_mode='grayscale',
      class_mode='binary',
      batch_size=batch_size
      )

plt.figure(figsize=(12, 12))
for i in range(0, 10):
           plt.subplot(2, 5, i+1)
           for X_batch, Y_batch in train:
                        image = X_batch[0]
                         dic = {0:'NORMAL', 1:'PNEUMONIA'}
                          plt.title(dic.get(Y_batch[0]))
                          plt.axis('off')

plt.imshow(np.squeeze(image),cmap='gray',interpolation='nea
rest')
break
plt.tight_layout()
plt.show()
```

Activity 2

Defining Model

```
cnn = Sequential()

cnn.add(Conv2D(32, (3, 3), activation="relu",
input_shape=(img_width, img_height, 1)))
cnn.add(MaxPooling2D(pool_size = (2, 2)))

cnn.add(Conv2D(32, (3, 3), activation="relu",
input_shape=(img_width, img_height, 1)))
cnn.add(MaxPooling2D(pool_size = (2, 2)))

cnn.add(Conv2D(32, (3, 3), activation="relu",
input_shape=(img_width, img_height, 1)))
cnn.add(MaxPooling2D(pool_size = (2, 2)))

cnn.add(Conv2D(64, (3, 3), activation="relu",
input_shape=(img_width, img_height, 1)))
cnn.add(MaxPooling2D(pool_size = (2, 2)))
cnn.add(Conv2D(64, (3, 3), activation="relu",
input_shape=(img_width, img_height, 1)))
cnn.add(MaxPooling2D(pool_size = (2, 2)))

cnn.add(Flatten())

cnn.add(Dense(activation = 'relu', units = 128))
cnn.add(Dense(activation = 'relu', units = 64))
cnn.add(Dense(activation = 'sigmoid', units = 1))

cnn.compile(optimizer = 'adam', loss = 'binary_crossentropy',
metrics = ['accuracy'])


from tensorflow.keras.utils import
plot_modelplot_model(cnn,show_shapes=True, show_layer_names=True,
rankdir='TB', expand_nested=True)
```

Activity 3

Training and Evaluating Model

```
early = EarlyStopping(monitor=”val_loss”, mode=”min”,
patience=3)learning_rate_reduction =
ReduceLROnPlateau(monitor=’val_loss’, patience = 2,
verbose=1,factor=0.3, min_lr=0.000001)callbacks_list = [ early,
learning_rate_reduction]
```

```
from sklearn.utils.class_weight import compute_class_weight
weights = compute_class_weight('balanced',
np.unique(train.classes), train.classes)
cw = dict(zip( np.unique(train.classes), weights))
print(cw)

cnn.fit(train,epochs=25, validation_data=valid, class_weight=cw,
callbacks=callbacks_list)

pd.DataFrame(cnn.history.history).plot()

test_accu = cnn.evaluate(test)print('The testing accuracy is
:',test_accu[1]*100, '%')
```

Activity 4

Visualizing Images

```
test.reset()
x=np.concatenate([test.next()[0] for i in range(test.__len__())])
y=np.concatenate([test.next()[1] for i in range(test.__len__())])
print(x.shape)
print(y.shape)#this little code above extracts the images from test
Data iterator without shuffling the sequence# x contains image
array and y has labels dic = {0:'NORMAL', 1:'PNEUMONIA'}
plt.figure(figsize=(20,20))
for i in range(0+228, 9+228):
  plt.subplot(3, 3, (i-228)+1)
  if preds[i, 0] >= 0.5:
      out = ('{:.2%} probability of being Pneumonia
case'.format(preds[i][0]))


  else:
      out = ('{:.2%} probability of being Normal case'.format(1-
```

```
preds[i][0]))plt.title(out+"\n Actual case : "+ dic.get(y[i]))
  plt.imshow(np.squeeze(x[i]))
  plt.axis('off')
plt.show()
```

3)Graded Lab Tasks

Note: The instructor can design graded lab activities according to the level of difficulty and complexity of the solved lab activities. The lab tasks assigned by the instructor should be evaluated in the same lab

Lab Task 1

Build and train a CNN model in Keras on Tuberculosis (TB) Chest X-ray Database available in Keras.

Lab 10

Sentiment Analysis using Keras

Objective:

The objective of this lab is to introduce students to Sentiment Analysis with Deep Learning and Keras. In this lab, students will learn to easily build, train and validate a Neural Network in Keras with the help of examples and learning tasks.

Activity Outcomes:

The activities provide hands-on practice with the following topics

- Data Exploration and Preprocessing
- Model building using Keras.
- Predicting Sentiment.

Instructor Note:

As pre-lab activity, read Chapter 5&6 of book "Gulli, Antonio, and Sujit Pal. Deep learning with Keras. Packt Publishing Ltd, 2017."

1) Useful Concepts

Sentiment analysis, also known as opinion mining is a natural language processing (NLP) technique used to determine whether data is positive, negative, or neutral. Sentiment analysis looks at the emotions expressed in a text. It is commonly used to analyze customer feedback, survey responses, and product reviews. There are several different approaches to sentiment analysis here you will learn a few of them.

1) **Data exploration:** Data exploration is the first step of data analysis used to explore and visualize data to uncover insights from the start or identify areas or patterns to dig into more.

2) **Data Preprocessing:** Data preprocessing is the process of transforming raw data into an understandable format. It also eliminates any unwanted aspect of data.

3) **Tokenization:** Tokenization is breaking the raw text into small chunks. It breaks down the raw text into words, sentences called tokens.

4) **Padding:** The padding adds zeros to the sentence to acquire the required length while not harming the meaning of the sentence.

5) **Vectors:** Vectors also known as Tensors are simply 1-dimensional arrays for data representation and operations.

6) **Activation Functions:** An activation function in a neural network defines how the weighted sum of the input is transformed into an output from a node or nodes in a layer of the network

7) **Word Embedding:** Neural Networks don't understand text data. To deal with the issue, you need to convert text into numbers. A word embedding is a learned representation for text where words that have the same meaning have a similar representation. It is this approach to representing words and documents

2) Solved Lab activities

Sr.No	Allocated Time	Level of Complexity	CLO Mapping
1	20 mins	High	CLO-5
2	20 mins	High	CLO-5
3	10 mins	Medium	CLO-5
4	10 mins	Low	CLO-5

Activity 1:

In this activity, you will download and preprocess the "Movie review dataset". The dataset is freely available on the given link.

https://raw.githubusercontent.com/jbrownlee/Datasets/master/review_polarity.tar.gz

Solution:

```
"""
importing stopwords from the natural language tool kit (NLTK) to remove
stopwords from the dataset. Stopwords do not add any meaning so to
save time and computation we omit them from the data
(example of stopwords are: "a", "the", "is", "are" etc )

"""
from nltk.corpus import stopwords

"""To process sentences we also need string library"""
import string library

""" Function to load doc into memory. """

def load_doc(filename):
    #open the file as read only
    file = open(filename, 'r')
    # read all text
    text = file.read()
    #close the file
```

```
    file.close()
    return text

""" turn a doc into clean token """
def clean_doc(doc):
    # split into tokens by white space
    tokens = doc.split()
    # remove punctuation from each token
    table = str.maketrans('', '', string.punctuation)
    tokens = [w.translate(table) for w in tokens]
    # remove remaining tokens that are not alphabetic
    tokens = [word for word in tokens in word.isalpha()]
    # filter out stop words
    stop_words = set(stopwords.words('english'))
    tokens = [w for w in tokens if not w in stop_words]
    # filter out short tokens
    tokens = [word for word in tokens if len(word) > 1]
    return tokens

# load a single document from dataset to test the functions
filename = 'txt_sentoken/pos/cv000_29590.txt'
text = load_doc(filename)
tokens = clean_doc(text)
print(tokens)
```

Output:
The following output should be expected

```
...
'creepy', 'place', 'even', 'acting', 'hell', 'solid', 'dreamy', 'depp', 't
urning', 'typically', 'strong', 'performance', 'deftly', 'handling', 'brit
ish', 'accent', 'ians', 'holm', 'joe', 'goulds', 'secret', 'richardson', '
dalmatians', 'log', 'great', 'supporting', 'roles', 'big', 'surprise', 'gr
aham', 'cringed', 'first', 'time', 'opened', 'mouth', 'imagining', 'attemp
t', 'irish', 'accent', 'actually', 'wasnt', 'half', 'bad', 'film', 'howeve
r', 'good', 'strong', 'violencegore', 'sexuality', 'language', 'drug', 'co
ntent']
```

Activity 2

The following part is the continuation of the code above to define the vocabulary for the model.

Solution:

```
""" Adding new libraries. """
from string import punctuation
from os import listdir
from collections import Counter


""" load doc and add to vocab """""
def add_doc_to_vocab(filename, vocab):
    # load doc
    doc = load_doc(filename)
    # clean doc
    tokens = clean_doc(doc)
    # update countes
    vocab.update(tokens)

""" Load all the doc in a directory """
def process_docs(directory, vocab, is_trian):
    # walk through all files in the folder
    for filename in listdir(directory):
        # skip any reviews in the test set
        if is_trian and filename.startswith('cv9'):
            continue
        if not is_trian and not filename.startswith('cv9'):
            continue
        # create the full path of the file to open
        path = directory + '/' + filename
        # add doc to vocab
        add_doc_to_vocab(path, vocab)

# define vocab
vocab = Counter()
# add all docs to vocab

"""add the neg review file path instead of 'txt_sentoken/neg' """
process_docs('txt_sentoken/neg', vocab, True)

"""add the pos review file path instead of 'txt_senttoken/pos'"""
process_docs('txt_sentoken/pos', vocab, True)
# print the size of the vocab
```

```
print(len(vocab))
# print the top words in the vocab
print(vocab.most_common(50))
```

Output:

```
44276
[('film', 7983), ('one', 4946), ('movie', 4826), ('like', 3201), ('even',
2262), ('good', 2080), ('time', 2041), ('story', 1907), ('films', 1873), (
'would', 1844), ('much', 1824), ('also', 1757), ('characters', 1735), ('ge
t', 1724), ('character', 1703), ('two', 1643), ('first', 1588), ('see', 15
57), ('way', 1515), ('well', 1511), ('make', 1418), ('really', 1407), ('li
ttle', 1351), ('life', 1334), ('plot', 1288), ('people', 1269), ('could',
1248), ('bad', 1248), ('scene', 1241), ('movies', 1238), ('never', 1201),
('best', 1179), ('new', 1140), ('scenes', 1135), ('man', 1131), ('many', 1
130), ('doesnt', 1118), ('know', 1092), ('dont', 1086), ('hes', 1024), ('g
reat', 1014), ('another', 992), ('action', 985), ('love', 977), ('us', 967
), ('go', 952), ('director', 948), ('end', 946), ('something', 945), ('sti
ll', 936)]
```

Saving the vocabulary:

This is a short function to save vocabulary you just created.

```
# save list to file
def save_list(lines, filename):
    # convert lines to a single blob of text
    data = '\n'.join(lines)
    # open file
    file = open(filename, 'w')
    # write text
    file.write(data)
    # close file
    file.close()

# save tokens to a vocabulary file
save_list(tokens, 'vocab.txt')
```

Activity 3

In this activity, you will learn a word embedding and get things ready for the model.

Solution:

```
from numpy import array
# kearas tokenizer
from keras.preprocessing.text import Tokenizer
# for padding sequences
from keras.preprocessing.sequence import pad_sequences
# Sequential is a linear stack of layers
from keras.models import Sequential
# A deeply connected Neural Network
from keras.layers import Dense
# to flatten the input
from keras.layers import Flatten
# for Embedding input
from keras.layers import Embedding
# used to create a convolution kernel that is convolved with the layer inp
ut
# over a single spatial dimension to produce a tensor of outputs.
from keras.layers.convolutional import Conv1D
# Used to downsample the input representation by taking the maximum value
# from a size of a window.
from keras.layers.convolutional import MaxPooling1D

""" The updated 'clean_doc' function is listed below.
    You can update the function written above.

"""
# turn a doc into clean tokens
def clean_doc(doc, vocab):
    # split into tokens by white space
    tokens = doc.split()
    # remove punctuation from each token
    table = str.maketrans('', '', punctuation)
    tokens = [w.translate(table) for w in tokens]
    # filter out tokens not in vocab
    tokens = [w for w in tokens if w in vocab]
    tokens = ' '.join(tokens)
    return tokens

""" The 'process_docs()' funtion is also update.
    You can update the function above.
```

```
    """
# load all docs in a directory
def process_docs(directory, vocab, is_trian):
    documents = list()
    # walk through all files in the folder
    for filename in listdir(directory):
        # skip any reviews in the test set
        if is_trian and filename.startswith('cv9'):
            continue
        if not is_trian and not filename.startswith('cv9'):
            continue
        # create the full path of the file to open
        path = directory + '/' + filename
        # load the doc
        doc = load_doc(path)
        # clean doc
        tokens = clean_doc(doc, vocab)
        # add to list
        documents.append(tokens)
    return documents


# load the vocabulary
"""
 you should have a vocab.txt file in your output just copy and paste
 its address below in place of 'vocab.txt'

"""
vocab_filename = 'vocab.txt'
vocab = load_doc(vocab_filename)
vocab = vocab.split()
vocab = set(vocab)

""" load all training reviews.
    Note that the value 'True' indicates training reviews.

    you need to update 'txt_sentoken/pos' and 'txt_sentence/neg' according

    to yourself.

    """

positive_docs = process_docs('txt_sentoken/pos', vocab, True)
negative_docs = process_docs('txt_sentoken/neg', vocab, True)
train_docs = negative_docs + positive_docs
```

```
# create the tokenizer
tokenizer = Tokenizer()
# fit the tokenizer on the documents
tokenizer.fit_on_texts(train_docs)

# sequence encode
encoded_docs = tokenizer.texts_to_sequences(train_docs)

""" for efficient computation we need to have uniform input size.
    We either need to truncate the sequence to the smallest size or
    pad to larget sentence. In this case, we will pad the sentences.
    """
# pad sequences
max_length = max([len(s.split()) for s in train_docs])
Xtrain = pad_sequences(encoded_docs, maxlen=max_length, padding='post')

""" Finally we need to define training labels. '0' for nagative review and
    '1' for positive review. """
ytrain = array([0 for _ in range(900)] + [1 for _ in range(900)])

# load all test reviews. Note that 'False' defines test set.
positive_docs = process_docs('txt_sentoken/pos', vocab, False)
negative_docs = process_docs('txt_sentoken/neg', vocab, False)
test_docs = negative_docs + positive_docs

# sequence encode
encoded_docs = tokenizer.texts_to_sequences(test_docs)

# pad sequences for the test dataset
Xtest = pad_sequences(encoded_docs, maxlen=max_length, padding='post')
# define test labels
ytest = array([0 for _ in range(100)] + [1 for _ in range(100)])

# define vocabulary size (largest integer value)
vocab_size = len(tokenizer.word_index) + 1
```

Activity 4:

Finally, we will define our model.

Solution:

```
# define model
model = Sequential()
model.add(Embedding(vocab_size, 100, input_length=max_length))
```

```
model.add(Conv1D(filters=32, kernel_size=8, activation='relu'))
model.add(MaxPooling1D(pool_size=2))
model.add(Flatten())
model.add(Dense(10, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
print(model.summary())
```

output:

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 embedding (Embedding)       (None, 1317, 100)         2576800

 conv1d (Conv1D)             (None, 1310, 32)          25632

 max_pooling1d (MaxPooling1D  (None, 655, 32)          0
 )

 flatten (Flatten)           (None, 20960)             0

 dense (Dense)               (None, 10)                209610

 dense_1 (Dense)             (None, 1)                 11

=================================================================
Total params: 2,812,053
Trainable params: 2,812,053
Non-trainable params: 0
_________________________________________________________________
None
```

```
# compile network
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accu
racy'])
# fit network
model.fit(Xtrain, ytrain, epochs=10, verbose=2)
# evaluate
loss, acc = model.evaluate(Xtest, ytest, verbose=0)
print('Test Accuracy: %f' % (acc*100))
```

Output:

The following should be the output and accuracy can vary.

```
...
Epoch 6/10
2s - loss: 0.0013 - acc: 1.0000
```

```
Epoch 7/10
2s - loss: 8.4573e-04 - acc: 1.0000
Epoch 8/10
2s - loss: 5.8323e-04 - acc: 1.0000
Epoch 9/10
2s - loss: 4.3155e-04 - acc: 1.0000
Epoch 10/10
2s - loss: 3.3083e-04 - acc: 1.0000
Test Accuracy: 84.500000
...
Epoch 6/10
2s - loss: 0.0013 - acc: 1.0000
Epoch 7/10
2s - loss: 8.4573e-04 - acc: 1.0000
Epoch 8/10
2s - loss: 5.8323e-04 - acc: 1.0000
Epoch 9/10
2s - loss: 4.3155e-04 - acc: 1.0000
Epoch 10/10
2s - loss: 3.3083e-04 - acc: 1.0000
Test Accuracy: 84.500000
```

Activity 5

In this short tutorial, you will learn a simple method for sentiment analysis using Huggingface Transformer library.

Solution:

```
# use pip to install transformers from huggingface
!pip install transformers
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
classifier("I loved Star Wars so much!")
```

Output:

The following is the output you should see.

```
[{'label': 'POSITIVE', 'score': 0.99}
```

3)Graded Lab Tasks

Note: The instructor can design graded lab activities according to the level of difficulty and complexity of the solved lab activities. The lab tasks assigned by the instructor should be evaluated in the same lab

Lab Task 1

Download your own dataset for sentiment analysis and apply the model above. The links for dataset repositories are provided here. Kaggle has a large dataset repository while you can also search for public datasets from GOOGLE Dataset Search
https://www.kaggle.com
Google Public Data Explorer

Lab Task 2

Make the following changes and report any change in results.

- *Change the number of filters and kernels.*
- *Add or subtract dense layer to the network to see if it makes any difference in results.*
- *Change the optimizer i.e 'adam'*
- *Change the number of epochs*

Note:
A working link of the above code is provided below in case of any issues faced.
https://colab.research.google.com/drive/1xs6MF6r5mjnR4qiXzlGQh_UyqJvn1rAM?usp=sharing

Lab 11

OpenCV

Objective:

Students will learn the Image-processing from Basics to Advance, like operations on Images, using a set of Opencv-programs.

Activity Outcomes:

After completing this tutorial, you will know about the following opencv image operations:

1. Read & Save Images
2. Image resizing
3. Image Rotation
4. Drawing Functions
5. Canny Edge Detection
6. Image Filters
7. Image Threshold
8. Contours
9. Erosion & Dilation

Instructor Note:

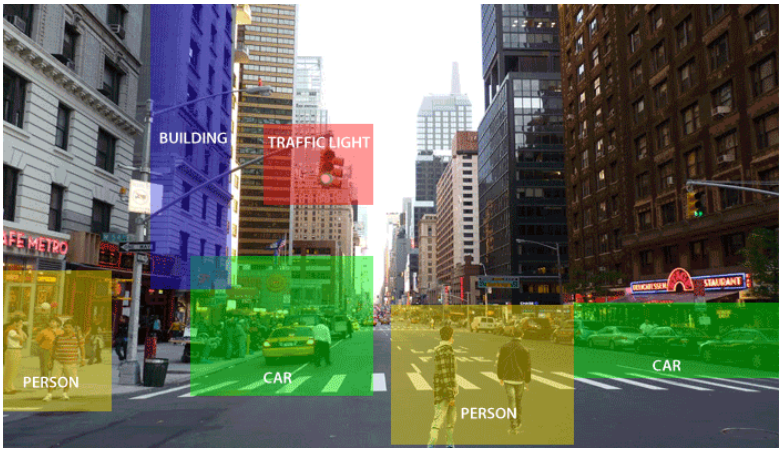
As pre-lab activity, read Chapter 13 from the text book Joshi, Prateek. *Artificial intelligence with python*. Packt Publishing Ltd, 2017.

1)Useful Concepts

OpenCV is one of the most popular computer vision libraries. In OpenCV, the CV is an abbreviation form of a computer vision, which is defined as a field of study that helps computers to understand the content of the digital images such as photographs and videos. It is a huge open-source library for computer vision, machine learning, and image processing. OpenCV supports a wide variety of programming languages like Python, C++, Java, etc. It can process images and videos to identify objects, faces, or even the handwriting of a human. When it is integrated with various libraries, such as Numpy which is a highly optimized library for numerical operations, then the number of weapons increases in your Arsenal i.e whatever operations one can do in Numpy can be combined with OpenCV.

Computer Vision: The purpose of computer vision is to understand the content of the images. It extracts the description from the pictures, which may be an object, a text description, and three-dimension model, and so on. For example, cars can be facilitated with computer vision, which will be able to identify and

different objects around the road, such as traffic lights, pedestrians, traffic signs, and so on, and acts accordingly.

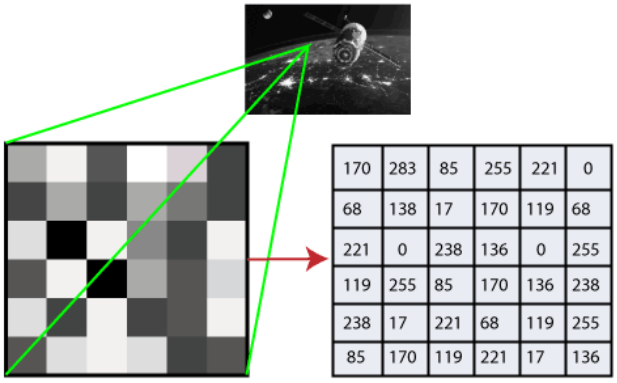


Computer vision allows the computer to perform the same kind of tasks as humans with the same efficiency. There are a two main task which are defined below:

- **Object Classification** - In the object classification, we train a model on a dataset of particular objects, and the model classifies new objects as belonging to one or more of your training categories.
- **Object Identification** - In the object identification, our model will identify a particular instance of an object

How does computer recognize the image?

Human eyes provide lots of information based on what they see. Machines are facilitated with seeing everything, convert the vision into numbers and store in the memory. Here the question arises how computer convert images into numbers. So the answer is that the pixel value is used to convert images into numbers. A pixel is the smallest unit of a digital image or graphics that can be displayed and represented on a digital display device.



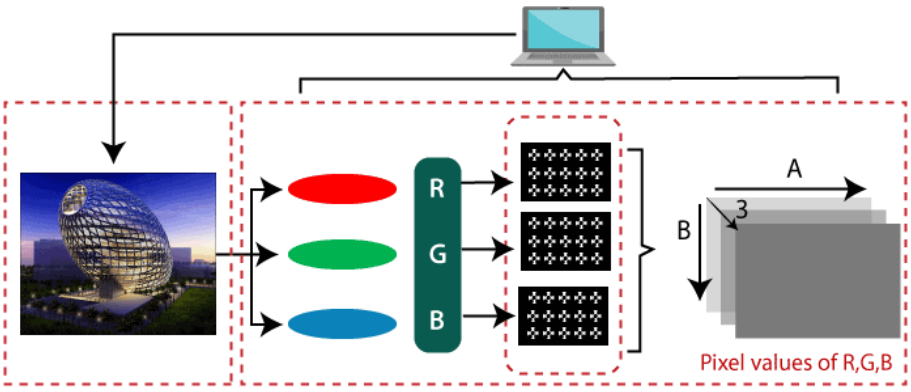
170	283	85	255	221	0
68	138	17	170	119	68
221	0	238	136	0	255
119	255	85	170	136	238
238	17	221	68	119	255
85	170	119	221	17	136

The picture intensity at the particular location is represented by the numbers. In the above image, we have shown the pixel values for a grayscale image consist of only one value, the intensity of the black color at that location.

There are two common ways to identify the images:

1. Grayscale: Grayscale images are those images which contain only two colors black and white. The contrast measurement of intensity is black treated as the weakest intensity, and white as the strongest intensity. When we use the grayscale image, the computer assigns each pixel value based on its level of darkness.

2. RGB: An RGB is a combination of the red, green, blue color which together makes a new color. The computer retrieves that value from each pixel and puts the results in an array to be interpreted.



Why OpenCV is used for Computer Vision?

- OpenCV is available for free of cost.
- Since the OpenCV library is written in C/C++, so it is quit fast. Now it can be used with Python.
- It require less RAM to usage, it maybe of 60-70 MB.
- Computer Vision is portable as OpenCV and can run on any device that can run on C.

2)Solved Lab Activities

Sr.no	*Allocated time*	*Level of complexity*	*CLO mapping*
Activity 1	***20 mins***	***Low***	***CLO-5***
Activity 2	***20 mins***	***Low***	***CLO-5***
Activity 3	***20 mins***	***Medium***	***CLO-5***
Activity 4	***20 mins***	***Medium***	***CLO-5***
Activity 4	***20 mins***	***Medium***	***CLO-5***

Activity 1:

Installation of OpenCV

A. Install OpenCV using Anaconda

The first step is to download the latest Anaconda graphic installer for Windows from it official site. Choose your bit graphical installer. You are suggested to install 3.7 or above working with Python 3.

After installing it, open the Anaconda prompt and type the following command.

```
conda install -c conda-forge opencv
```

Press the Enter button and it will download all the related OpenCV configuration.

B. Install OpenCV in the Windows via pip

OpenCV is a Python library so it is necessary to install Python in the system and install OpenCV using pip command:

```
pip install opencv-contrib-python --upgrade
```

We can install it without extra modules by the following command:

```
pip install opencv-python
```

Activity 2:

Reading Image in OpenCV

OpenCV imread function*:* The imread() function loads image from the specified file and returns it. The syntax is:

```
cv2.imread(filename[,flag])
```

Parameters:

filename: Name of the file to be loaded

flag: The flag specifies the color type of a loaded image:

- CV_LOAD_IMAGE_ANYDEPTH - If we set it as flag, it will return 16-bits/32-bits image when the input has the corresponding depth, otherwise convert it to 8-BIT.
- CV_LOAD_IMAGE_COLOR - If we set it as flag, it always return the converted image to the color one.
- C V_LOAD_IMAGE_GRAYSCALE - If we set it as flag, it always convert image into the grayscale.

The imread() function returns a matrix, if the image cannot be read because of unsupported file format, missing file, unsupported or invalid format. Currently, the following file formats are supported.

Let's consider the following example:

```
#importing the opencv module
```

```
1. #importing the opencv module
2. import cv2
3.
4. # using imread('path') and 0 denotes read as  grayscale imag
   e
5. img = cv2.imread(r'cat.jpeg',1)
6.
7. #This is using for display the image
8. cv2.imshow('image',img)
9.
10.     cv2.waitKey(3) # This is necessary to be required so t
   hat the image doesn't close immediately.
11.     #It will run continuously until the key press.
12.     cv2.destroyAllWindows()
```

Output: it will display the following image

OpenCV imwrite() function*:*This function is used to save an image to a specified file. The file extension defines the image format. The syntax is the following::

```
cv2.imwrite(filename, img[,params])
```

Parameters:
image- Image to be saved.
params- The following parameters are currently supported:

- For JPEG, quality can be from 0 to 100. The default value is 95.
- For PNG, quality can be the compress level from 0 to 9. The default value is 1.
- For PPM, PGM, or PBM, it can be a binary format flag 0 or 1. The default value is 1**.**

Let's consider the following example:

```
1. import cv2
2.
3. # read image as grey scale
4. img = cv2.imread(r'cat.jpeg', 1)
5.
6. # save image
7. status = cv2.imwrite(r'cat.jpeg', 0, img)
8. print("Image written to file-
   system : ", status)
```

Output:

```
Image written to file-system : True
```

Activity 2:

Image Resizing and Roatation

Image Resizing

Sometimes, it is necessary to transform the loaded image. In the image processing, we need to resize the image to perform the particular operation. Images are generally stored in Numpy ndarray(array). The ndarray.shape is used to obtain the dimension of the image. We can get the width, height, and numbers of the channels for each pixel by using the index of the dimension variable.

```
1. import cv2
2.
3. img = cv2.imread(r'cat.jpeg', 1)
4. scale = 60
5. width = int(img.shape[1] * scale / 100)
6. height = int(img.shape[0] * scale / 100)
7. dim = (width, height)
8. # resize image
9. resized = cv2.resize(img, dim, interpolation=cv2.INTER_AREA)


10.
```

```
    print('Resized Dimensions : ', resized.shape)

    cv2.imshow("Resized image", resized)
    cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:

```
Resized Dimensions :  (199, 300, 3)
```

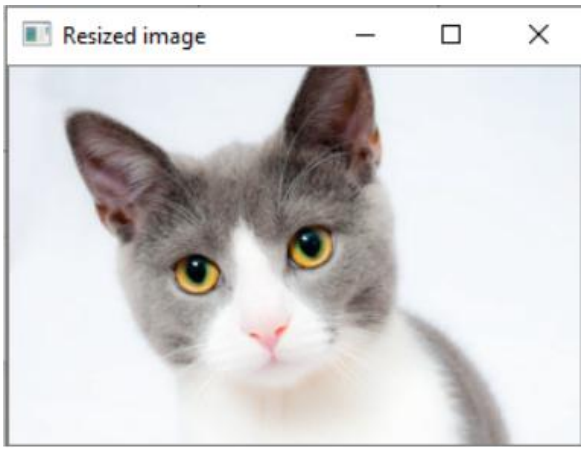


Image Rotation

The image can be rotated in various angles (90,180,270 and 360). OpenCV calculates the affine matrix that performs affine transformation, which means it does not preserve the angle between the lines or distances between the points, although it preserves the ratio of distances between points lying on the lines.

The syntax of the rotate image is the following:

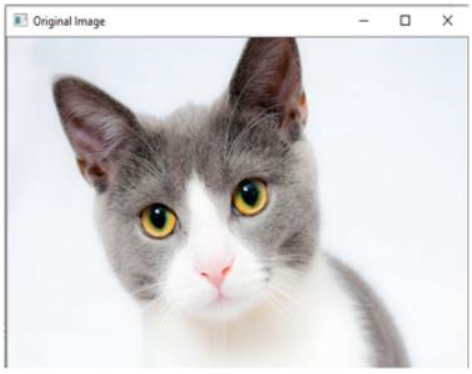
```
cv2.getRotationMatrix2D(center, angle, scale rotated =
cv2.warpAfifne(img,M,(w,h))
```

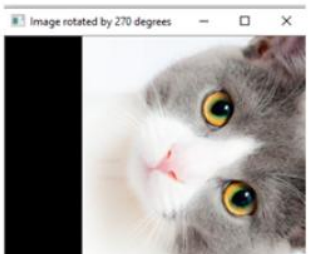
```
import cv2
# read image as greyscale
img = cv2.imread(r'cat.jpeg')
# get image height, width
(h, w) = img.shape[:2]
# calculate the center of the image
```

```
7. center = (w / 2, h / 2)
8.
9. angle90 = 90
10.     angle180 = 180
11.     angle270 = 270
12.
13.     scale = 1.0
14.
15.     # Perform the counterclockwise rotation holding at the
   center
16.     # 90 degrees
17.     M = cv2.getRotationMatrix2D(center, angle90, scale)
18.     rotated90 = cv2.warpAffine(img, M, (h, w))
19.
20.     # 180 degrees
21.     M = cv2.getRotationMatrix2D(center, angle180, scale)
22.     rotated180 = cv2.warpAffine(img, M, (w, h))
23.
24.     # 270 degrees
25.     M = cv2.getRotationMatrix2D(center, angle270, scale)
26.     rotated270 = cv2.warpAffine(img, M, (h, w))
27.
28.     cv2.imshow('Original Image', img)
29.     cv2.waitKey(0)  # waits until a key is pressed
30.     cv2.destroyAllWindows()  # destroys the window showing
   image
31.
32.     cv2.imshow('Image rotated by 90 degrees', rotated90)
33.     cv2.waitKey(0)  # waits until a key is pressed
34.     cv2.destroyAllWindows()  # destroys the window showing
   imag
35.
36.     cv2.imshow('Image rotated by 180 degrees', rotated180)
```

```
37.      cv2.waitKey(0)  # waits until a key is pressed
38.      cv2.destroyAllWindows()  # destroys the window showing
   image
39.
40.      cv2.imshow('Image rotated by 270 degrees', rotated270)

41.      cv2.waitKey(0)  # waits until a key is pressed
42.      cv2.destroyAllWindows()  # destroys the window showing
   image
43.
```

Output:





Activity 3:

OpenCV Drawing Functions

We can draw the various shapes on an image such as circle, rectangle, ellipse, polylines, convex, etc. It is used when we want to highlight any object in the input image. The OpenCV provides functions for each shape. Here we will learn about the drawing functions.

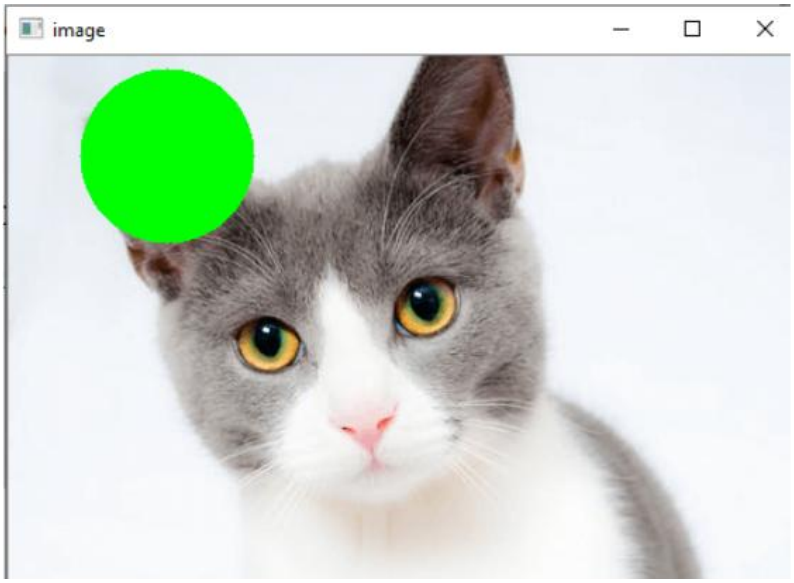
Cirlce

We can draw the circle on the image by using the cv2.circle() function. The syntax is the following:

```
cv2.circle(img, center, radius, color[,thickness [, lineType[,shift]]])
```

```
1. import numpy as np
2. import cv2
3. img = cv2.imread(r" cat.jpeg",1)
4. cv2.circle(img,(80,80), 55, (0,255,0), -1)
5. cv2.imshow('image',img)
6. cv2.waitKey(0)
7. cv2.destroyAllWindows()
```

Output:



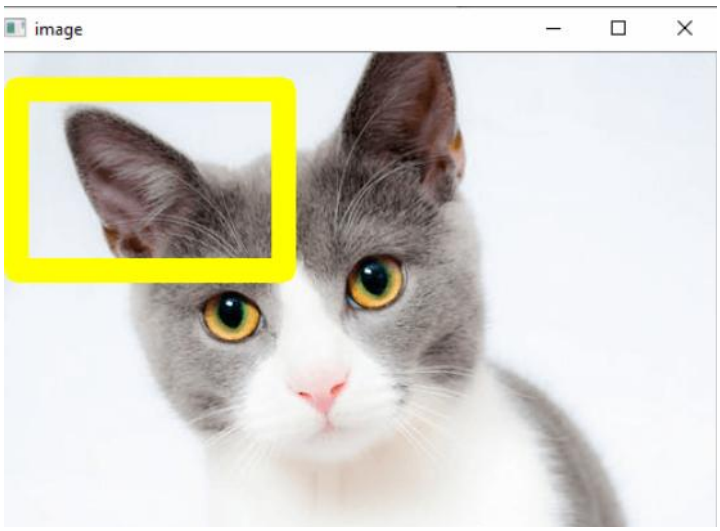
Rectangle

We can draw the circle on the image by using the cv2.circle() function. The syntax is the following:

```
cv2.rectangle(img, pt1, pt2, color[,
thickness[,lineType[,shift]]])
```

```
1. import numpy as np
2. import cv2
3. img = cv2.imread(r" cat.jpeg",1)
4. cv2.rectangle(img,(15,25),(200,150),(0,255,255),15)
5. cv2.imshow('image',img)
6. cv2.waitKey(0)
7. cv2.destroyAllWindows()
```

Output:



Canny Edge Detection

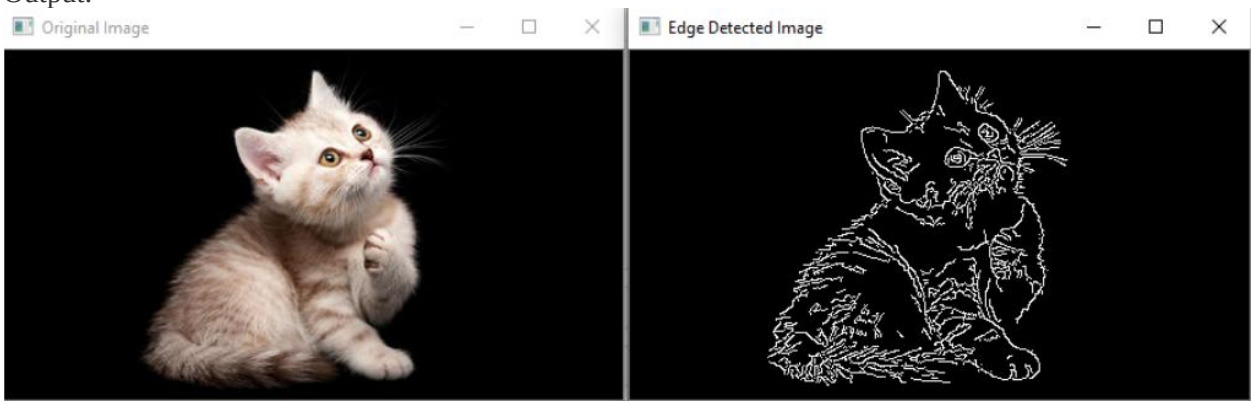
Edge detection is term where identify the boundary of object in image. We will learn about the edge detection using the canny edge detection technique. The syntax is canny edge detection function is given as:.

```
edges = cv2.Canny('/path/to/img', minVal, maxVal,
apertureSize, L2gradient)
```

```
1. import cv2
2. img = cv2.imread(r'cat_16x9.jpg')
3. edges = cv2.Canny(img, 100, 200)
4.
5. cv2.imshow("Edge Detected Image", edges)
6. cv2.imshow("Original Image", img)
```

```
7. cv2.waitKey(0)  # waits until a key is pressed
8. cv2.destroyAllWindows()  # destroys the window showing image
```

Output:



Activity 4:

OpenCV Image Filters

Image filtering is the process of modifying an image by changing its shades or color of the pixel. It is also used to increase brightness and contrast. In this tutorial, we will learn about several types of filters.

Box Filter

We can perform this filter using the boxfilter() function. It is similar to the averaging blur operation. The syntax of the function is given below:

```
cv2. boxfilter(src, dst, ddepth, ksize, anchor, normalize, bordertype)
```

```
1. import cv2
2. import numpy as np
3. # using imread('path') and 0 denotes read as  grayscale imag
   e
4. img = cv2.imread(r'baloon.jpg',1)
5. img_1 = cv2.boxFilter(img, 0, (7,7), img, (-1,-
   1), False, cv2.BORDER_DEFAULT)
6. #This is using for display the image
7. cv2.imshow('Image',img_1)
8. cv2.waitKey(3) # This is necessary to be required so that th
   e image doesn't close immediately.
9. #It will run continuously until the key press.
```

```
10.     cv2.destroyAllWindows()
```

Output:

Filter2D

It combines an image with the kernel. We can perform this operation on an image using the Filter2D() method. The syntax of the function is given below:

```
cv2.Filter2D(src, dst, kernel, anchor = (-1,-1))
```

```
1. import cv2
2. import numpy as np
3. from matplotlib import pyplot as plt
4. img = cv2.imread(r'baloon.jpg',1)
5.
6. kernel = np.ones((5,5),np.float32)/25
7. dst = cv2.filter2D(img,-1,kernel)
8. plt.subplot(121),plt.imshow(img),plt.title('Original')
9. plt.xticks([]), plt.yticks([])
10.     plt.subplot(122),plt.imshow(dst),plt.title('Filter2D'
   )
11.     plt.xticks([]), plt.yticks([])
12.     plt.show()
```

Output:

OpenCV Image Threshold

The basic concept of the threshold is that more simplify the visual data for analysis. When we convert the image into gray-scale, we have to remember that grayscale still has at least 255 values. The threshold is converted everything to white or black, based on the threshold value. Let's assume we want the threshold to be 125(out of 255), then everything that was under the 125 would be converted to 0 or black, and everything above the 125 would be converted to 255, or white. The syntax is as follows:

Box Filter

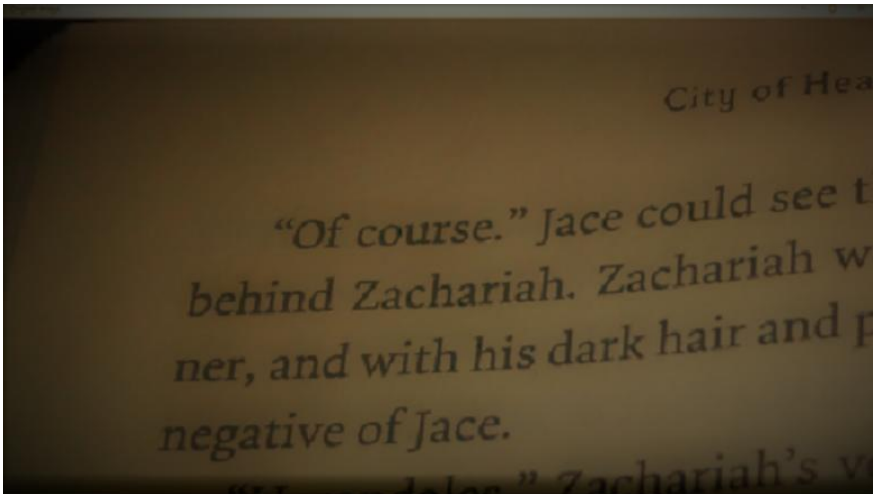
We can perform this filter using the boxfilter() function. It is similar to the averaging blur operation. The syntax of the function is given below:

```
retval,threshold = cv2.threshold(src, thresh, maxValue, cv2.THRESH_BINARY_INV)
```

OpenCV provides different styles of threshold that is used as fourth parameter of the function. These are the following:

0. cv2.THRESH_BINARY
1. cv2.THRESH_BINARY_INV
2. cv2.THRESH_TRUNC
3. cv2.THRESH_TOZERO
4. cv2.THRESH_TOZERO_INV

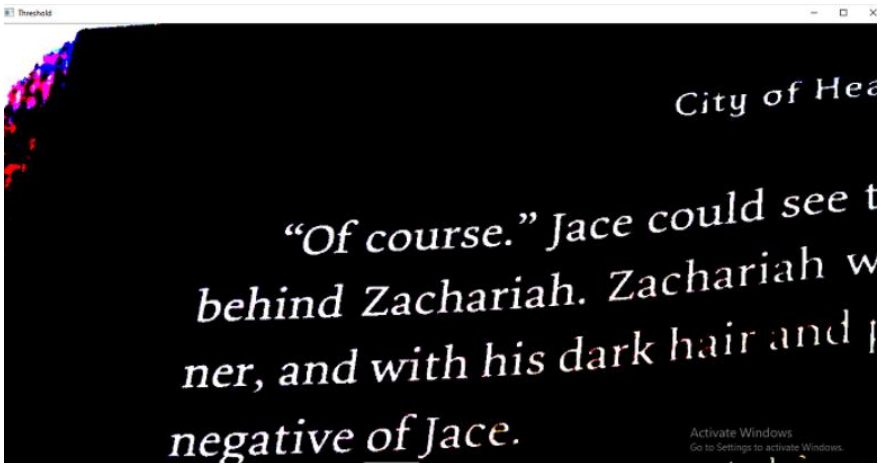
Let's take a sample input image



We have taken above image as an input. We describe how threshold actually works. The above image is slightly dim and little bit hard to read. Some parts are light enough to read, while other part is required more focus to read properly. Let's consider the following example:

```
1. import cv2
2. img  = cv2.imread(r'book1.jpg',1)
3. retval, threshold = cv2.threshold(img, 62, 255, cv2.THRESH_B
   INARY)
4. cv2.imshow("Original Image", img)
5. cv2.imshow("Threshold",threshold)
6. cv2.waitKey(0)
```

Output:



Activity 5:
OpenCV Contours, Erosion & Dilation

Contours are defined as a curve joining all the continuous points (along the boundary), having the same color or intensity. In the other, we find counter in a binary image, we focus to find the boundary in the binary image. The official definition is following:

To maintain accuracy, we should use the binary images. First, we apply the threshold or canny edge detection. In OpenCV, finding the contour in the binary image is the same as finding white object from a black background.
OpenCV provides findContours(), which is used to find the contour in the binary image. The syntax is following:

```
cv2. findContours (thes, cv2.RETR_TREE,
cv.CHAIN_APPROX_SIMPLE)
```

The findContours () accepts the three argument first argument is source image, second is contour retrieval mode, and the third is contours approximation.

Let's consider the following example:

```
1. import numpy as np
2. import cv2 as cv
3. im = cv.imread(r'binary.png')
4. imgray = cv.cvtColor(im, cv.COLOR_BGR2GRAY)
5. ret, thresh = cv.threshold(imgray, 127, 255, 0)
6. contours, hierarchy = cv.findContours(thresh, cv.RETR_TREE,
   cv.CHAIN_APPROX_SIMPLE)
```

OpenCV Erosion & Dilation
Erosion and Dilation are morphological image processing operations. OpenCV morphological image processing is a procedure for modifying the geometric structure in the image. In morphism, we find the shape and size or structure of an object. Both operations are defined for binary images, but we can also use them on a grayscale image. These are widely used in the following way:

- Removing Noise
- Identify intensity bumps or holes in the picture.
- Isolation of individual elements and joining disparate elements in image.

In this tutorial, we will explain the erosion and dilation briefly..

Dilation

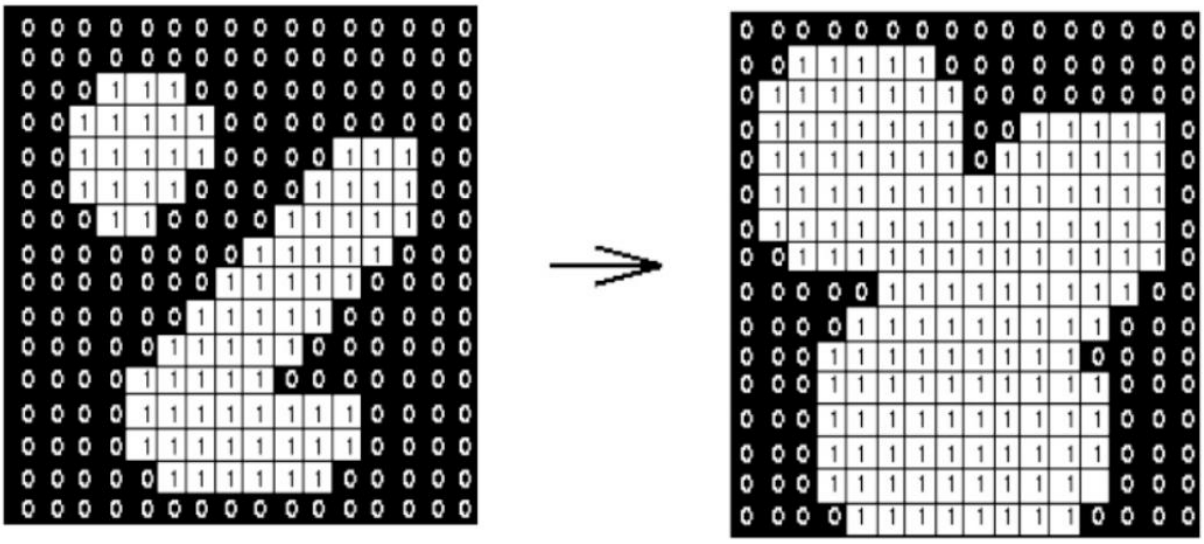
Dilation is a technique where we expand the image. It adds the number of pixels to the boundaries of objects in an image. The structuring element controls it. The structuring element is a matrix of 1's and 0's.

Structuring Element

The size and shape of the structuring element define how many numbers of the pixel should be added or removed from the objects in an image. It is a matrix of 1's and 0's. The center pixel of the image is called the origin.

It contains an image A with some kernel (B), which can have any shape or size, generally a square or circle. Here the kernel B has a defined anchor point. It is the center of the kernel.

In the next step, the kernel is overlapped over the image to calculate maximum pixel values. When the computation is completed, the image is replaced with an anchor at the center. The brighter areas increase in size that made increment in image size.



Effect of dilation using a 3×3 square structuring element

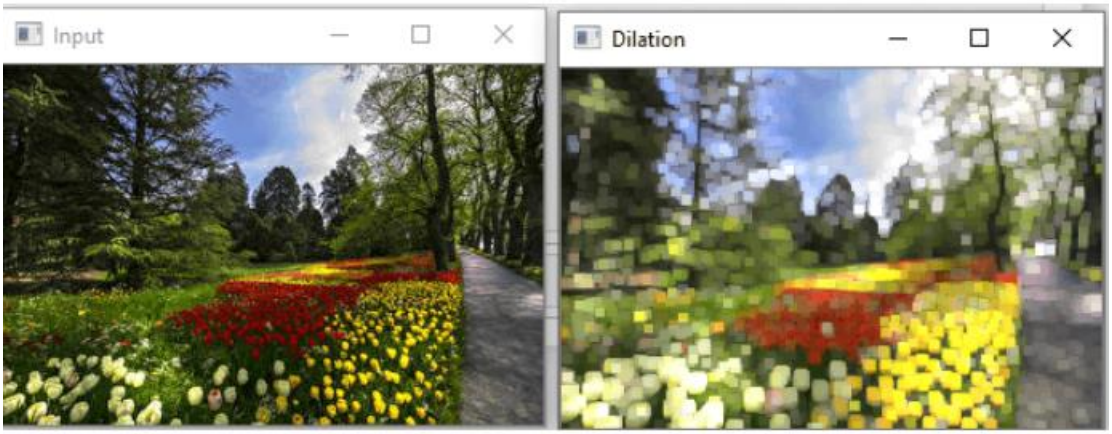
For example, the size of the object increases in the white shade; on the other side, the size of an object in black shades automatically decreases.

The dilation operation is performed by using the cv2.dilate() method. The syntax is given below:

```
cv2.dilate(src, dst, kernel)
```

```
1. import cv2
2. import numpy as np
3. img = cv2.imread(r'C:\Users\DEVANSH SHARMA\jtp_flower.jpg, 0
   )
4.
```

```
5. kernel = np.ones((5,5), np.uint8)
6. img_erosion = cv2.erode(img, kernel, iterations=1)
7. img_dilation = cv2.dilate(img, kernel, iterations=1)
8. cv2.imshow('Input', img)
9. cv2.imshow('Dilation', img_dilation)
10.     cv2.waitKey(0)
```



Erosion

Erosion is much similar to dilation. The difference is that the pixel value calculated minimum rather than the maximum in dilation. The image is replaced under the anchor point with that calculated minimum pixel. Unlikely dilation, the regions of darker shades increase. While it decreases in white shade or brighter side.

OpenCV provides cv2.erode() function to perform this operation. The syntax of the function is the following:.

```
cv2.erode(src, dst, kernel)
```

```
1. import cv2
2. import numpy as np
3. img = cv2.imread(r'C:\Users\DEVANSH SHARMA\baloon.jpg', 1)
4. kernel = np.ones((5,5), np.uint8)
5. img_erosion = cv2.erode(img, kernel, iterations=1)
6. img_dilation = cv2.dilate(img, kernel, iterations=1)
7. cv2.imshow('Input', img)
8. cv2.imshow('Erosion', img_erosion)
9. cv2.waitKey(0)
```

```
10.
```

The above program will give the following output. We can see the different between both images.

Erosion operation applied to the input image.

3)Graded Lab Task

Note: The instructor can design graded lab activities according to the level of difficulty and complexity of the solved lab activities. The lab tasks assigned by the instructor should be evaluated in the same lab

Lab task 1

Apply different filters on images and analyze the results

Lab 12

OpenCV: Image Smoothing and Template Matching

Objective:

Students will learn the Image-processing from Basics to Advance, like operations on Images, using a set of Opencv-programs.

Activity Outcomes:

- Image smoothing
- Template Matching

Instructor Note:

As pre-lab activity, read Chapter 13 from the text book Joshi, Prateek. *Artificial intelligence with python*. Packt Publishing Ltd, 2017.

1)Useful Concepts

This lab is an extended version of Lab 12.

2)Solved Lab Activities

Sr.no	***Allocated time***	***Level of complexity***	***CLO mapping***
1	***20 mins***	***Low***	***CLO-5***
2	***20 mins***	***Low***	***CLO-5***
3	***20 mins***	***Medium***	***CLO-5***
4	***20 mins***	***Medium***	***CLO-5***
5	***20 mins***	***Medium***	***CLO-5***

Activity 1:

OpenCV Image Smoothing

Image smoothing is an image processing technique used for removing the noise in an image. Blurring (smoothing) removes low-intensity edges and is also beneficial in hiding the details; for example, blurring is required in many cases, such as hiding any confidential information in an image.OpenCV provides mainly the following type of blurring techniques.

Here are a few of the methods that we are going to use for smoothing an image:

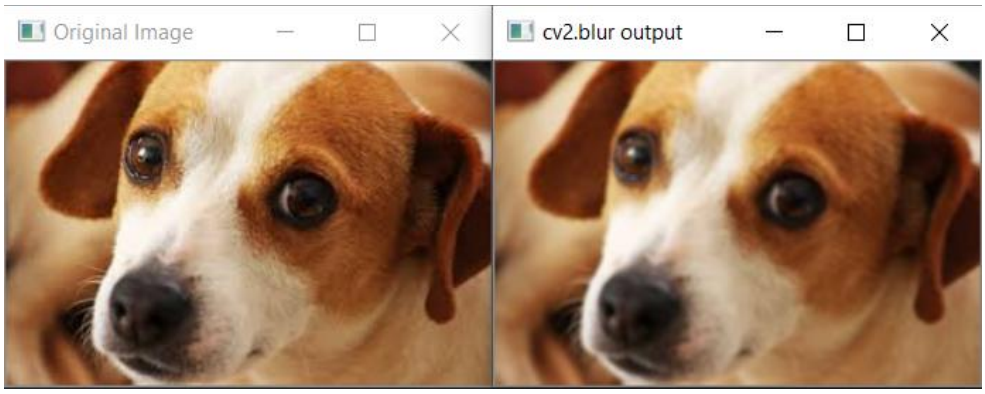
- OpenCV averaging
- OpenCV median Blur
- OpenCV Gaussian Blur
- OpenCV Bilateral Filter

OpenCV averaging: In this technique, we normalize the image with a box filter. It calculates the average of all the pixels which are under the kernel area(box filter) and replaces the value of the pixel at the center of the box filter with the calculated average. OpenCV provides the cv2.blur() to perform this operation. The syntax of cv2.blur() function is as follows.

```
 cv2.blur(src, ksize,anchor, borderType)

Parameters:

src: It is the image which is to be blurred.

ksize: A tuple representing the blurring kernel size.

anchor: It is a variable of type integer representing anchor
point and it’s default value Point is (-1, -1) which means that
the anchor is at the kernel center.

borderType:It represents the type of border to be used for the
output.
```

Here is an example:

```
1    import cv2
2    img = cv2.imread('dog.jpg')
3    cv2.imshow('Original Image',img)
4    cv2.imshow('cv2.blur output', cv2.blur(img, (3,3)))
5    cv2.waitKey(0)
6    cv2.destroyAllWindows()
```

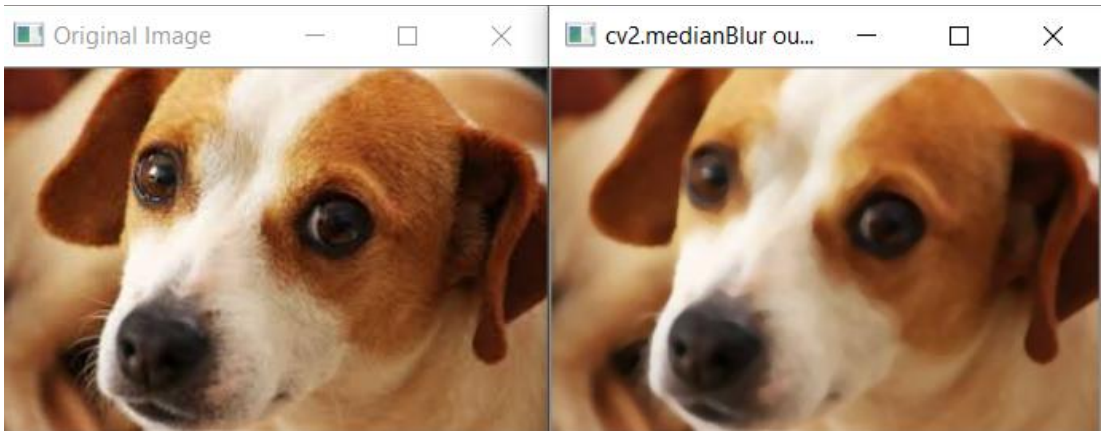


OpenCV median Blur : In this technique,the median of all the pixels under the kernel window is computed and the central pixel is replaced with this median value. It has one advantage over the Gaussian and box filters,that being the filtered value for the central element is always replaced by some pixel value in the image which is not the case in case of either Gaussian or box filters.OpenCV provides a function medianBlur() that can be used to easily implement this kind of smoothing.Here is the syntax:

```
cv2.medianBlur(src, dst, ksize)

Parameters:

src- It represents the source (input image).

dst - It represents the destination (output image).

ksize - It represents the size of the kernel.
```

Consider the following example:

```
1
2   import cv2
3   img = cv2.imread('dog.jpg')
4   cv2.imshow('Original Image',img)
5   cv2.imshow('cv2.medianBlur output', cv2.medianBlur(img,5))
6   cv2.waitKey(0)
    cv2.destroyAllWindows()
```



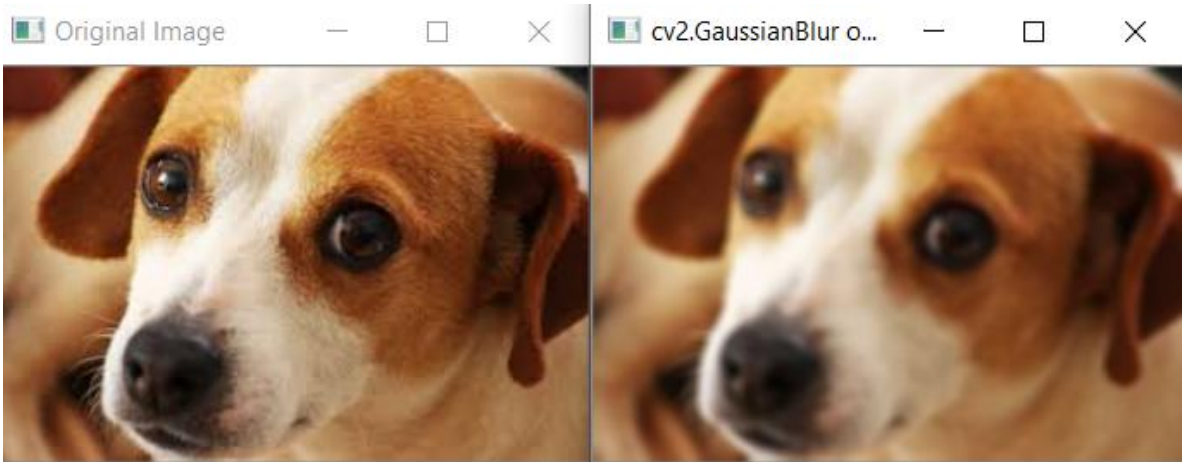
OpenCV Gaussian Blur: In this technique,a Gaussian function(kernel) instead of a box filter to blur the image.The width and height of the kernel needs to be specified and they should be positive and odd.We also have to specify the standard deviation in the directions X and Y and are represented by sigmaX and sigmaY respectively. If both sigmaX and sigmaY are given as zeros, they are calculated from the kernel size and if we only specify sigmaX,sigmaY is set to the same value.Gaussian blurring is highly effective when removing Gaussian noise from an image. In OpenCV we have a function GaussianBlur() to implement this technique easily. Here is the syntax:

```
 GaussianBlur(src, dst, ksize, sigmaX,sigmaY)

Parameters:

src - Input image which is to be blurred

dst - output image of the same size and type as src.

ksize - A Size object representing the size of the kernel.

sigmaX - A variable of the type double representing the
Gaussian kernel standard deviation in X direction.

sigmaY - A variable of the type double representing the
Gaussian kernel standard deviation in Y direction.
```

Here is an example:

```
1    import cv2
2    img = cv2.imread('dog.jpg')
```

```
3    cv2.imshow('Original Image',img)
4    cv2.imshow('cv2.GaussianBlur output', cv2.GaussianBlur(img, (5, 5), cv2.BORDER_DEFAULT
5    cv2.waitKey(0)
6    cv2.destroyAllWindows()
```



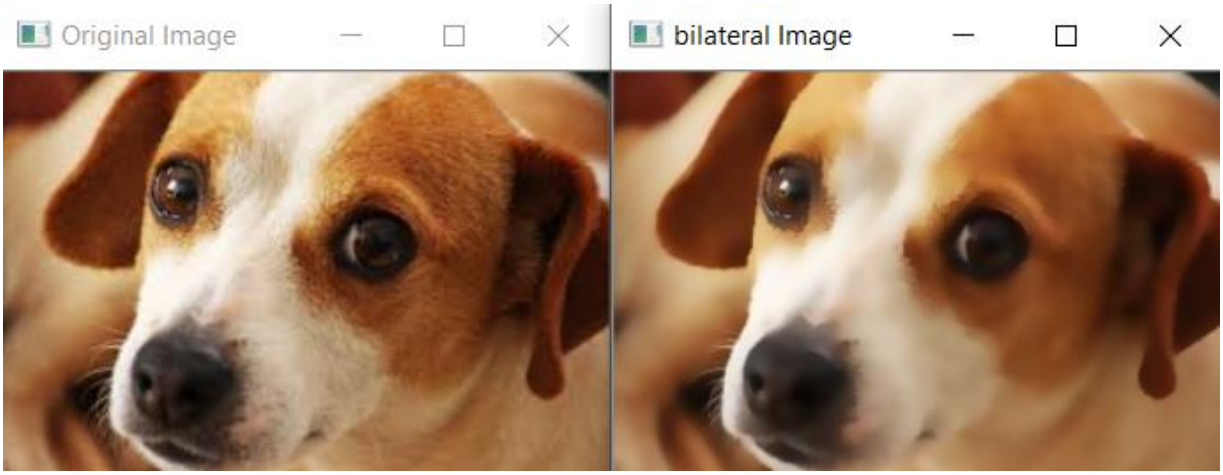
OpenCV Bilateral Filter: This method of noise removal is highly effective but is slower compared to other filters.The Gaussian filter blurred the edges too and that is not what we want,but this filter makes sure that only those pixels with similar intensities to the central pixel are considered for blurring,thus preserving the edges since pixels at edges will have large intensity variation.In OpenCV we have cv.bilateralFilter() method that can implement this filter.Here is the syntax:

```
 cv2.bilateralFilter(src, dst, d, sigmaColor,sigmaSpace,
borderType)

Parameters:

src Source 8-bit or floating-point, 1-channel or 3-channel
image.

dst Destination image of the same size and type as src .

d Diameter of each pixel neighborhood that is used during
filtering. If it is non-positive, it is computed from
sigmaSpace.

sigmaColor Filter sigma in the color space. A larger value of
the parameter means that farther colors within the pixel
neighborhood (see sigmaSpace) will be mixed together, resulting
in larger areas of semi-equal color.

sigmaSpace Filter sigma in the coordinate space. A larger value
of the parameter means that farther pixels will influence each
other as long as their colors are close enough (see sigmaColor
```

```
). When d>0, it specifies the neighborhood size regardless of
sigmaSpace. Otherwise, d is proportional to sigmaSpace.

borderType border mode used to extrapolate pixels outside of
the image, see the BorderTypes available here.
```

Here is an example:

```
import cv2
img = cv2.imread('dog.jpg')
cv2.imshow('Original Image',img)
cv2.imshow('bilateral Image', cv2.bilateralFilter(img,9,75,75))
cv2.waitKey(0)
cv2.destroyAllWindows()
```



OpenCV Image Threshold: Thresholding is a popular segmentation technique, used for separating an object considered as a foreground from its background.In this technique we assign pixel values in relation to the threshold value provided.This technique of thresholding is done on grayscale images,so initially, the image has to be converted in grayscale color space.Here we will discuss two different approaches taken when performing thresholding on an image:

- Simple Thresholding
- Adaptive Thresholding

Simple Thresholding: In this basic Thresholding technique, for every pixel, the same threshold value is applied. If the pixel value is smaller than the threshold, it is set to a certain value(usually zero) , otherwise, it is set to another value(usually maximum value) .There are various variations of this technique as shown below.
In OpenCV, we use cv2.threshold function to implement it.Here is the syntax:

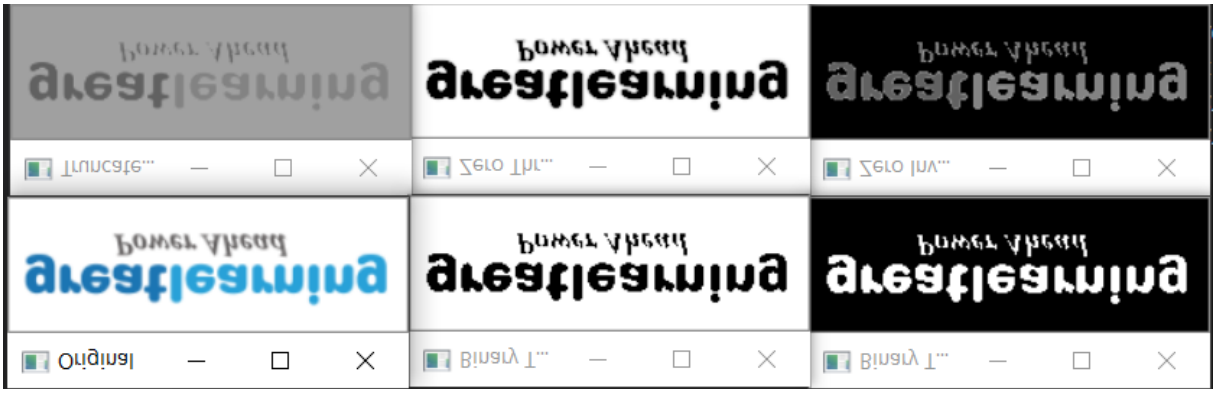
```
 cv2.threshold(source, thresholdValue, maxVal,
thresholdingTechnique)
```

```
Parameters:

-> source: Input Image array (must be in Grayscale).

-> thresholdValue: Value of Threshold below and above which
pixel values will change accordingly.

-> maxVal: Maximum value that can be assigned to a pixel.

-> thresholdingTechnique: The type of thresholding to be
applied.Here are various types of thresholding we can use



cv2.THRESH_BINARY: If  the pixel intensity is greater than the
threshold, the pixel value is set to 255(white), else it is set
to 0 (black).

cv2.THRESH_BINARY_INV: Inverted or Opposite case of
cv2.THRESH_BINARY.If  the pixel intensity is greater than the
threshold, the pixel value is set to 0(black), else it is set
to 255 (white).

cv.THRESH_TRUNC: If  the pixel intensity is greater than the
threshold,the pixel values are set to be the same as the
threshold. All other values remain the same.

cv.THRESH_TOZERO: Pixel intensity is set to 0, for all the
pixels intensity, less than the threshold value.All other pixel
values remain same

cv.THRESH_TOZERO_INV: Inverted or Opposite case of
cv2.THRESH_TOZERO.
```

Here is an example:

```
1     import cv2
2     import numpy as np
3
4     # path to input image is specified and
5     # image is loaded with imread command
6     image = cv2.imread('gl.png')
7
8     # to convert the image in grayscale
9     img = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
10
11      threshold=160
12      ret, thresh1 = cv2.threshold(img, threshold, 255, cv2.THRESH_BINARY)
13      ret, thresh2 = cv2.threshold(img, threshold, 255, cv2.THRESH_BINARY_INV)
14      ret, thresh3 = cv2.threshold(img, threshold, 255, cv2.THRESH_TRUNC)
15      ret, thresh4 = cv2.threshold(img, threshold, 255, cv2.THRESH_TOZERO)
16      ret, thresh5 = cv2.threshold(img, threshold, 255, cv2.THRESH_TOZERO_INV)
17
18      # the window showing output images
19      # with the corresponding thresholding
20      # techniques applied to the input images
21      cv2.imshow('Original',image)
22      cv2.imshow('Binary Threshold', thresh1)
23      cv2.imshow('Binary Threshold Inverted', thresh2)
24      cv2.imshow('Truncated Threshold', thresh3)
25      cv2.imshow('Zero Threshold', thresh4)
26      cv2.imshow('Zero Inverted', thresh5)
27
28      # De-allocate any associated memory usage
29      cv2.waitKey(0)
30      cv2.destroyAllWindows()
31
```



Adaptive Thresholding: In simple thresholding,the threshold value was global which means it was same for all the pixels in the image.But this may not be the best approach for thresholding as the different image sections can have different lightings.Thus we need Adaptive thresholding,which is the method where the threshold value is calculated for smaller regions and therefore, there will be different threshold values for different regions.In OpenCV we have adaptiveThreshold() function to implement this type of thresholding.Here is the syntax of this function:

```
 adaptiveThreshold(src, dst, maxValue, adaptiveMethod,
thresholdType, blockSize, C)
```

This method accepts the following parameters –

src – An object of the class Mat representing the source (input) image.

dst – An object of the class Mat representing the destination (output) image.

maxValue – A variable of double type representing the value that is to be given if pixel value is more than the threshold value.

adaptiveMethod – A variable of integer the type representing the adaptive method to be used. This will be either of the following two values:

cv.ADAPTIVE_THRESH_MEAN_C: The threshold value is the mean of the neighbourhood area minus the constant C.

cv.ADAPTIVE_THRESH_GAUSSIAN_C: The threshold value is a gaussian-weighted sum of the neighbourhood values minus the constant C.

thresholdType – A variable of integer type representing the type of threshold to be used.

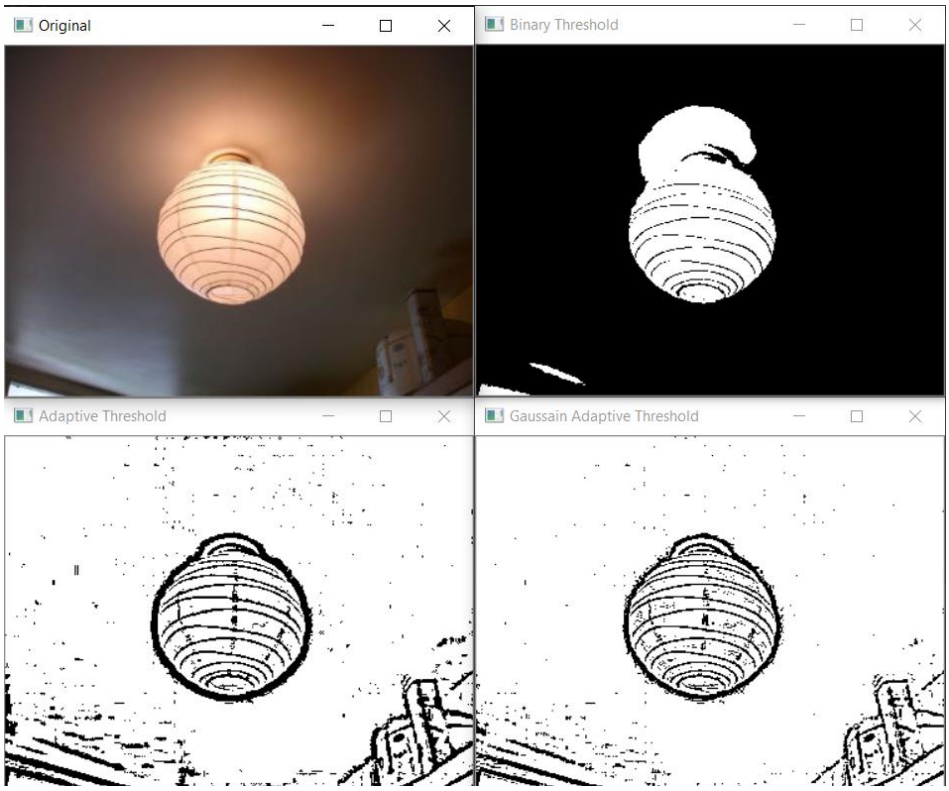
blockSize – A variable of the integer type representing size of the pixelneighborhood used to calculate the threshold value.

C – A variable of double type representing the constant used in the both methods (subtracted from the mean or weighted mean).

Here is an example:

```
import cv2
import numpy as np

# path to input image is specified and
# image is loaded with imread command
image = cv2.imread('lamp.jpg')

```

```
# to convert the image in grayscale
img = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

ret, th1 = cv2.threshold(img,160, 255, cv2.THRESH_BINARY)

th2 = cv2.adaptiveThreshold(img,255,cv2.ADAPTIVE_THRESH_MEAN_C,
            cv2.THRESH_BINARY,11,2)
th3 = cv2.adaptiveThreshold(img,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
            cv2.THRESH_BINARY,11,2)

cv2.imshow('Original',image)
cv2.imshow('Binary Threshold', th1)
cv2.imshow('Adaptive Threshold', th2)
cv2.imshow('Gaussain Adaptive Threshold', th3)

# De-allocate any associated memory usage
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Activity 2:

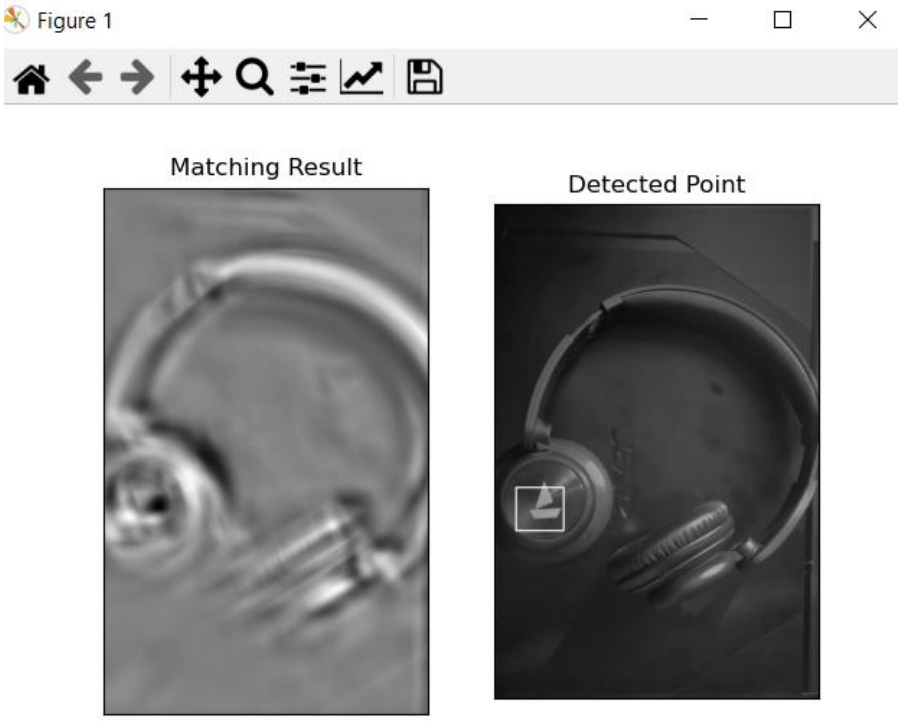
OpenCV Template Matching

OpenCV Template Matching: Template Matching is a method used for finding the location of a template image in a larger image. In OpenCV,we use a function cv.matchTemplate() for template matching. It simply slides the template image over the larger input image (as in 2D convolution) and compares the template image with the patch of input image under the template image.It returns a grayscale image, where each pixel denotes how much does the neighbourhood of that pixel match with the template.There are several comparison methods that can be implemented in OpenCV.
If input image is of size (WxH) and template image is of size (wxh), output image will have a size of (W-w+1, H-h+1).Upon getting results, the best matches can be found as global minimums (when TM_SQDIFF was used) or maximums (when TM_CCORR or TM_CCOEFF was used) using the minMaxLoc function. Take it as the top-left corner of the rectangle and take (w,h) as width and height of the rectangle. That rectangle is your region of template.
Here is the syntax of cv.matchTemplate():

```
 cv.matchTemplate(image, templ, method,mask)

Parameters:




image :Image where the search is running. It must be 8-bit or
32-bit floating-point.

templ :Searched template. It must be not greater than the
source image and have the same data type.

result Map of comparison results. It must be single-channel 32-
bit floating-point. If image is W×H and templ is w×h , then
result is (W−w+1)×(H−h+1) .

method:Parameter specifying the comparison method, see
TemplateMatchModes

mask: Optional
```

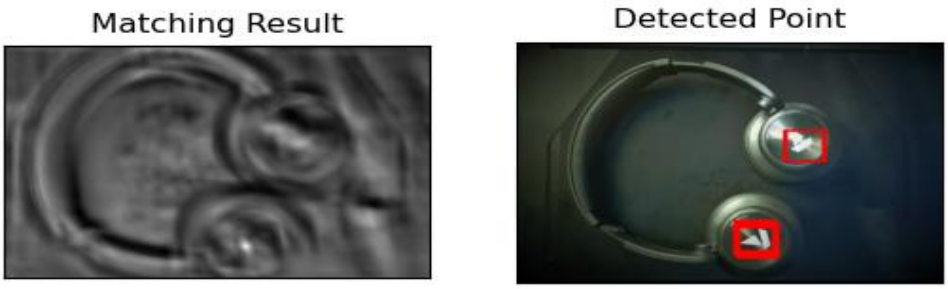
Here is an example in which we take this image as the template image:

```
1
2
3   import cv2 as cv
4   import numpy as np
5   from matplotlib import pyplot as plt
6   img = cv.imread('headphone1.jpeg',0)
7   img2 = img.copy()
8   template = cv.imread('logo1.jpeg',0)
9   w, h = template.shape[::-1]
10  # All the 6 methods for comparison in a list
11  # Apply template Matching
12  res = cv.matchTemplate(img,template,eval('cv.TM_CCOEFF'))
13  min_val, max_val, min_loc, max_loc = cv.minMaxLoc(res)
14  # If the method is TM_SQDIFF or TM_SQDIFF_NORMED, take minimum
15  top_left = max_loc
16  bottom_right = (top_left[0] + w, top_left[1] + h)
17  cv.rectangle(img,top_left, bottom_right, 255, 2)
18  plt.subplot(121),plt.imshow(res,cmap = 'gray')
19  plt.title('Matching Result'), plt.xticks([]), plt.yticks([])
20  plt.subplot(122),plt.imshow(img,cmap = 'gray')
    plt.title('Detected Point'), plt.xticks([]), plt.yticks([])
    plt.show()
```



Template Matching with Multiple Objects: In the above example, we searched for template images that occurred only once in the image. Suppose a particular object occurs multiple times in a particular image. In this scenario, we will use the thresholding as cv2.minMaxLoc() just gives the location of one template image and it won't give all locations of the template images. Consider the following example.

```
import cv2 as cv
import numpy as np
from matplotlib import pyplot as plt
img2=cv.imread('headohone2.jpeg',1)
img_gray = cv.imread('headohone2.jpeg',0)
template = cv.imread('logo1.jpeg',0)
w, h = template.shape[::-1]
res = cv.matchTemplate(img_gray,template,eval('cv.TM_CCOEFF_NORMED'))
print(res)
threshold = 0.52
loc = np.where( res >= threshold)
for pt in zip(*loc[::-1]):
    cv.rectangle(img2, pt, (pt[0] + w, pt[1] + h), (255,0,0), 1)
plt.subplot(121),plt.imshow(res,cmap = 'gray')
plt.title('Matching Result'), plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(img2,cmap = 'gray')
plt.title('Detected Point'), plt.xticks([]), plt.yticks([])
plt.show()
```



3) Graded Lab Task:

Note: The instructor can design graded lab activities according to the level of difficulty and complexity of the solved lab activities. The lab tasks assigned by the instructor should be evaluated in the same lab

Lab Task 1

Build a face recognition system using template matching in OpenCV